

definegeometry2d

November 24, 2009

13:47

Contents

1	A program for Defining 2-D Geometries	1
2	Introduction and Background	2
3	Symmetry	4
4	How To Use This Code	5
5	Input File	7
6	Keyword Reference	12
7	Other Considerations	21
8	The Main Program	24
9	Main Subroutine	27
10	Read and process DG file	45
11	Read wall file	51
12	Convert mesh data into polygons	53
13	Set up mesh elements	55
14	Specify and break up a new polygon	58
15	Set up trivial end zones	63
16	Set up toroidally facing sectors	66
17	Set up sector based diagnostics, including those composed of auxiliary strata	69
18	Read mesh file from UEDGE	71
19	Compute minimum and maximum coordinates	72
20	Print wall coordinates to a file for user examination	73
21	Read input lines specifying a polygon	75

22 Read data specifying a diagnostic	93
23 Fill in data for this zone	96
24 Define universal cell	98
25 Find the centroid of a triangle	101
26 Compute the center of a quadrilateral	103
27 Compute area of a closed polygon	103
28 Reverse order of polygon points	104
29 Find endpoints in a set of elements	105
30 Convert elements to a polygon	106
31 Count the number of continuous sections in set of elements	108
32 Search mesh edges to find the ones corresponding to the input DG label	110
33 Attempt to find the closest point on the two input edges to the input point	111
34 Find closest point in a set of elements, <i>edge_elements</i> , to the point provided, <i>poly_p</i>	113
35 Set up sectors	115
36 Print data from Triangle	119
37 C interface routine to Triangle	122
38 INDEX	127

1 A program for Defining 2-D Geometries

This program began as an extension and generalization of the logic used in *readgeometry*. The approach used has been rather successful, permitting a wide variety of geometries to be described in a straightforward manner. While the code can be run using just a human generated text input file, with perhaps some auxiliary data in other text files, truly complicated cases can be handled when the code is run in conjunction with other geometry packages. Currently, a nearly complete coupling to the DG code, developed in Garching, gives the user what is effectively a graphical interface to the problem at hand. Alternatively, the plasma mesh used by the UEDGE code can be read in via a single command; a plasma mesh in the Sonnet format can be read in a similar manner (Sonnet format grids are also compatible with DG). `definegeometry2d` can now handle nearly symmetric 3-D problems; the essential restriction is that all of the objects in the problem must be describeable as 2-D figures swept through a range of the ignorable coordinate. The intermediate specification of the geometry purely in terms of polygons is written out to a netCDF file. This file can then be exported to other applications, such as AVS/Express, providing a precise description of the geometry for use in plotting DEGAS 2 results (as opposed to the approximate approach used in `geomtesta`).

A version of DG is now distributed as part of DEGAS 2. See the additional documentation in the DEGAS 2 User's Manual and in the manual provided with DG.

Note that this program makes extensive use of Jonathan Shewchuck's **Triangle** package to break polygons up into triangles. The user needs to download its source code from:

`http://www.cs.cmu.edu/~quake/triangle.html`

and arrange for the `Makefile` to find it (the default location is `$HOME/Triangle`).

2 Introduction and Background

This code takes a human-readable input file that, together with the other file(s) it references, describes a geometry you want to use in **DEGAS 2**. The purpose of this code is to transform that description into **DEGAS 2**'s internal description of the geometry.

The hierarchy of geometry-related objects in **DEGAS 2** is, from the lowest level to the highest level, is:

1. surface,
2. cell,
3. polygon,
4. zone.

Individual cells are composed of surfaces, as discussed in the documentation for the internal geometry (e.g., see *geometry.web*). The next level up (in two dimensions) is a polygon. This program will call various routines to break up (non-convex) polygons into cells. Finally, a zone may consist of one or more polygons; properties are constant across a zone. For example, a single zone might be used to represent the vacuum region around the plasma which is comprised of several (possibly disconnected) polygons. Or, the volume between two plasma flux surfaces in which density and temperature are constant might be a single zone. Note that polygons are used only within external interfaces such as this code. Cells and surfaces are essentially used only by the code itself; the user only deals with them in debugging. Zones are used primarily for the specification of input and output data.

The most common sources of errors in this code are associated with the violation of two basic restrictions on **DEGAS 2** geometries:

1. The problem volume (everything inside the “universal cell”) must be completely filled with user-defined zones.
2. Each surface not lying along the boundary of the universal cell must have user-defined zones on both sides of it.

If *definegeometry2d* reports an error, you have most likely violated one (or both) of these two constraints.

Another important component of the geometry are the “sectors”; they are defined at interfaces between adjacent zones of different types to facilitate tracking or between zones of the same type for diagnostic purposes. Flights are temporarily halted at sectors during tracking. If the next zone along the track represents, say, a solid material, the main code hands that flight off to the routines that effect the interaction of the current particle with that material. While stopped at that sector, scores to corresponding “diagnostics” (groups of sectors) are also tabulated. The essential identifying information of a sector are its surface and zone numbers. Currently, sectors are defined on both sides of that interface, one for each of the two adjacent zones. Both will refer to the same surface, although the surfaces will have opposite orientations.

For additional information on sectors see the documentation for the sectors class (in `src/sector.hweb` or `Doc/classes.pdf`) and the “Diagnostic Sectors” section of the User’s Manual.

This code automatically creates sectors at the interfaces between plasma / vacuum zones (i.e., zones in which flights propagate) and solid / exit zones (i.e., zones in which flights do not propagate). A sector between solid and plasma zones is defined to be a “target” sector and one between a solid and vacuum zone is a “wall” sector. The adjoining sectors are referred to as “plasma” and “vacuum” sectors, respectively. For further details, see the sector class described in *sector.hweb*.

Sectors are also labeled by a “stratum” number and a “segment” number. These labels facilitate subsequent user reference to the sectors. Since these two labels are just that, they can in principle be chosen almost arbitrarily (although the choice should be such that a given a stratum, segment pair does uniquely identify a sector). Because the process of identifying the default sectors in a problem is based on an examination of all solid and exit polygons, the stratum numbers associated with those polygons are used to label the resulting sectors *on both sides*. Consequently, stratum numbers assigned to vacuum and plasma polygons are currently not used. Default sectors defined at the edges of a *sonnet_mesh* or *uedge_mesh* the mesh will carry the stratum label from the adjoining solid polygon.

Each side of a polygon gives gives rise to a different surface. The sector’s “segment” number is derived from the location of that side in the polygon. Again, that segment number is used to label both sectors constructed at that interface.

The user can also specify “auxiliary sectors” that coincide with arbitrary segments of a polygon and even include segments from multiple polygons. The user can then define a diagnostic or diagnostics from either default or auxiliary sectors.

3 Symmetry

The `definegeometry2d` code is intended to handle 2-D and nearly symmetric 3-D planar and cylindrical geometries (for a more detailed description of geometry symmetry in DEGAS 2, see the “Symmetry and Coordinate Systems” section of the User’s Manual). In the 2-D cases, one coordinate is ignored (y in plane symmetry; ϕ in cylindrical symmetry). Tracking is done in 3-D, but the zones constructed from the polygons established in `definegeometry2d` extend the full range of the ignored coordinate, providing no resolution in that direction.

In the nearly symmetric 3-D cases, the 3-D zones are again constructed by sweeping the polygons through the y or ϕ coordinate. The difference is that those zones are further subdivided by $y = \text{constant}$ or $\phi = \text{constant}$ planes. The result is a “bread slice” (in the rectangular case) or a “pie slice” (in the cylindrical configuration) description of the geometry. Consequently, DEGAS 2 results are resolved in that direction. A second difference is that the zone type (solid, plasma, vacuum, or exit) can vary in the y or ϕ direction. This capability allows some 3-D objects, such as a poloidal limiter in a tokamak, to be simulated.

4 How To Use This Code

Virtually all of the information in the `definegeometry2d` input file is “high level”, e.g., point coordinates rarely appear. Instead, most of the detailed information regarding points and their connectivity is contained in the other files that can, and should, be generated by other packages. The names of these files are specified in the first part of the input file. The formats that are currently in use and the corresponding keywords in the input file are (all of these are text files):

dg_file An output (with extension `.dgo`) file from the DG code.

wallfile A simple text file containing a specified number of simply connected “walls”, with each wall consisting of a sequence of (X, Z) values.

sonnet_mesh A 2-D mesh specified in the Sonnet format; e.g., as generated by the **Sonnet** or **Carre** codes.

uedge_mesh A specific format used to transfer data between the **UEDGE** and **DEGAS 2** codes.

Both *dg_file* and *wallfile* result in the definition of “walls” that can be used to construct the polygons in the second part of the `definegeometry2d` input file using the *wall* keyword. The polygons can also be constructed directly in DG using its graphical interface. These polygons are invoked in the second part of the *definegeometry2d* input file with the *dg_polygon* keyword. A portion of the boundary of a plasma mesh imported via *sonnet_mesh* or *uedge_mesh* can be incorporated into a polygon via the *edge* command. A simpler approach to accomplishing the same result using DG is to associate a “mesh connection” with the polygon; an *iedge* (short for “intelligent edge”) keyword in the *definegeometry2d* input file provides the code with additional information on how to connect the mesh edge to the rest of the polygon. The corners of the universal cell (set with the *bounds* keyword) can be added to a polygon with the *outer* keyword. If points having the exact same coordinates are defined in DG, they can be used there in a *dg_polygon* and will be recognized as being on the boundary of the universal cell.

The two most frequently used approaches to setting up a tokamak based geometry begin with files such as these generated by the **UEDGE** or **DG** codes. Typical methodologies for these two cases are:

UEDGE A simple geometry can be set up by using the *uedge_mesh* keyword to establish all of the plasma zones. The *edge* and *outer* commands can be used to define solid zones that enclose this volume. Alternatively, a slightly more complicated approach involves specifying in a *wallfile* a more realistic vacuum vessel shape. The (vacuum) volume between that surface and the **UEDGE** mesh would need to be described by polygons set up using the *wall* and *edge* keywords.

DG The 2-D outlines of the vacuum vessel (or other hardware or surfaces) are first defined as “elements” in the DG code. These can subsequently be connected into and used to define DG’s “polygons”. Note that the Carre or Sonnet grid generation codes can be used to create Sonnet format plasma meshes that can be viewed and manipulated (in limited ways) in DG. The edges of the mesh can be incorporated into polygons with “mesh connections”. The DG elements, polygons and other information are contained in DG’s output file. Upon being read into *definegeometry2d*, these objects are reinterpreted in terms of its internal objects. The user can then utilize them in creating the polygons that will divide up the problem space.

Alternatively, there are two possible approaches to generating a new geometry from scratch:

1. If the problem possesses a basic symmetry, e.g., “plasma parameters are constant on a flux surface”, you can specify only those bounding surfaces and let *definegeometry2d* subdivide them (semi-arbitrarily) into smaller zones. First, construct a wall file containing the coordinates of each of the bounding surfaces, with each as a separate wall (see below under the *wallfile* keyword). Then, in the *definegeometry2d* input file, create a polygon out of each pair of adjacent walls, i.e., surfaces. Using the *triangulate_to_zones* keyword will then break these polygons up into smaller zones. The sizes of the triangles will be almost entirely controlled by the number of points used in the walls (see, however, the *triangle_area* keyword).
2. If a mesh composed of triangles or quadrilaterals already exists, the easier approach may be to write that information in the Sonnet format. Each mesh zone in the Sonnet file format is defined by five points. These coordinates are given on three consecutive lines. The “corners” appear on the first and third lines; the second line contains the “center” point. Inside *definegeometry2d*, the four corners must trace out the shape in a clockwise fashion. While there is some freedom in the way these points are labeled, for definiteness, we will say that the first line contains the second and third points (in that order) of a quadrilateral; the third line has the first and fourth points. For a triangle, just make the two points on the third line the same (the code will also agree to call it a triangle if the third and fourth points are the same). If the center point is set exactly to zero, built in routines will be used to compute an appropriate center. As in the above example, additional data about the hardware can be specified and translated into “walls”. The mesh edges can be used together with the walls to specify the rest of the polygons required to fill the universal cell.

The DEGAS 2 User’s Manual contains two example applications of **definegeometry2d**, one demonstrating the use of DG and the other illustrating the use of an external code to create walls and polygons out of flux surfaces.

5 Input File

Note that the name of the input file for **definegeometry2d** is not listed in **degas2.in**. Although this might be viewed as an omission, it is consistent in that **definegeometry2d** is, in principle, one of many different means of generating **DEGAS 2** geometries and, thus, is not as fundamental as the other entries in **degas2.in**.

Instead, the command line for the execution of **definegeometry2d** specifies the name of the main **definegeometry2d** input file:

```
definegeometry2d an_input_file
```

The input file, called here **an_input_file**, will contain pointers to other files that provide externally generated, detailed information about the geometry. Like other text input files used in **DEGAS 2**, blank lines, spaces, and lines beginning with a comment character **#** are ignored (comments can also appear at the end of a line).

The input file consists of two sections:

1. A preparatory stage in which global parameters are set and file names are specified,
2. A “construction” section containing the specification of polygons that will fill the universal cell.

Let us introduce the components of the input file with a nontrivial example:

```
# There are two parts to the input file. The first provides global
# information.
#
# It's a good idea to set the symmetry of the problem first.

symmetry cylindrical

# The bulk of the data describing the "hardware" in this problem are
# in a file generated by the DG code.

dg_file Run35/bypass_12.dgo

# The bounds keyword specifies the size of the universal cell.

bounds 0.3 1.1 -0.8 0.7

# Another big chunk of data required in setting up the problem is the
# mesh which will store the plasma-related data. In this case, the
# mesh is provided in the Sonnet format.

sonnet_mesh Run10/cmod.carre.008 61 26

# The end_prep command terminates the first part of the input file. The
# primary task undertaken with the execution of this command is the
# generation of plasma zones from the quadrilaterals described by the
# plasma mesh.

end_prep

# In the second part of the input file, the user must define zones that
# will completely fill (without overlap) the universal cell. These
# zones are specified as the union of one or more polygons. The
# new_zone command begins the definition of the next zone; in this
# case, a solid zone. The zone definition is terminated by the next
# "new_zone" command (or an "end" or "quit"). The code will assume that
# the polygon will be closed by connecting the last point specified
# to the first point. All polygons should be convex.

new_zone solid

# The following five polygons all belong to that single zone.
# The polygons are enumerated and labeled to facilitate external reference.

# The definition of a polygon is initiated by a "new_polygon" command
# and finished by "breakup_polygon", "triangulate_polygon", or
# "triangulate_to_zones". Each of these breaks the polygon down into
# DEGAS 2's internal description consisting of cells and surfaces.
# In the last case, each of the resulting triangles is assigned a new
# zone number.
#
```

```
# Keywords like "outer", "wall", and "edge" specify single points or a
# range of points used to construct the polygon. Other keywords like
# "stratum" or "material" specify properties associated with the polygon.
# Note, however, that these properties will apply to all subsequent
# polygons until overridden by new property settings. Finally, the
# "print_polygon" command (commented out since this input file was well
# past the testing stage) writes out the points in the polygon to a
# text file so they can be analyzed or plotted externally. This is
# useful for debugging problem polygons.

# Polygon 1: inner limiter
new_polygon
  stratum 1
  material mo          # For all solid zones (default temp. and recycling)
  outer 0 1
  wall 6 0 1
  wall 3 3 3
  edge 0 0 * reverse
#   print_polygon poly_1
breakup_polygon

# Polygon 2: upper divertor
new_polygon
  stratum 2
  outer 1 2
  wall 5 11 0
#   print_polygon poly_2
breakup_polygon

# Polygon 3: outer limiter
new_polygon
  stratum 3
  outer 2 3
  wall 5 49 11
#   print_polygon poly_3
triangulate_polygon

# Polygon 4: gas box, right side
new_polygon
  stratum 4
  outer 3 0
  wall 5 68 49
#   print_polygon poly_4
breakup_polygon

# Polygon 5: gas box, left side
new_polygon
  stratum 5
  outer 0
  edge 0 0 0 0
  wall 2 0 2
  wall 4 1 15
```

```
        wall 5 71 68
#       print_polygon poly_5
        breakup_polygon

new_zone solid

# Polygon 6: outer target
new_polygon
    stratum 6
    wall 7 0 15
    edge 61 61 *
    wall 7 24 26
#       print_polygon poly_6
        breakup_polygon

new_zone vacuum

# Polygon 7: private flux region
new_polygon
    stratum 7
    edge 0 10 0 0
    edge 52 61 0 0
    wall 4 6 0
    wall 2 1 0
#       print_polygon poly_7
        triangulate_to_zones

new_zone vacuum

# Polygon 8: gas box entrance
new_polygon
    stratum 8
    edge 61 61 0 0
#
# The following define two auxiliary sectors belonging to stratum 14
#
    aux_stratum 14
    wall 7 15 15
    end_aux_stratum
    wall 4 7 7
    aux_stratum 14
    wall 4 6 6
    end_aux_stratum
#       print_polygon poly_8
        breakup_polygon

new_zone vacuum

# Polygon 9: gas box
new_polygon
    stratum 9
    wall 7 15 3
```

```

        wall 5 51 71
        wall 4 15 7
#       print_polygon poly_9
        triangulate_to_zones

new_zone vacuum

# Polygon 10: gas box exit
        new_polygon
            stratum 10
            wall 7 3 3
#
# Here we add two more sectors to auxiliary stratum 14
#
        aux_stratum 14
        wall 7 2 2
        end_aux_stratum
        wall 5 50 50
        aux_stratum 14
        wall 5 51 51
        end_aux_stratum
#       print_polygon poly_10
        breakup_polygon

new_zone vacuum

# Polygon 11: lower main chamber
        new_polygon
            stratum 11
            edge 61 42 26 26
            wall 5 38 50
            wall 7 2 0
            wall 7 26 24
#       print_polygon poly_11
        triangulate_to_zones

new_zone vacuum

# Polygon 12: upper main chamber
        new_polygon
            stratum 12
            edge 42 0 26 26
            wall 3 3 3
            wall 6 1 1
            wall 5 0 38
#       print_polygon poly_12
        triangulate_to_zones

new_zone exit

# Polygon 13: core plasma
        new_polygon

```

```
        stratum 13
        edge 11 51 0 0
#       print_polygon poly_13
        breakup_polygon

# Now define a diagnostic based on the auxiliary stratum defined above
# in polygons 8 and 10:

new_diagnostic Throat sectors
    stratum 14
end_diagnostic

# The description of this geometry in terms of polygons is written out
# to a netCDF file with this name so that it may be used again later.

polygon_nc_file Run35/bypass_12_poly_a.nc

# The "end" command instructs the code to finish setting up the
# internal geometry arrays, to check the geometry for consistency,
# and to write its netCDF file. Since the process of generating
# a geometry as complex as this one is understandably iterative, the
# code also has a "quit" command (that can be used anywhere except
# within a polygon definition) that quietly terminates the code without
# performing any of these finalization steps.

end
```

6 Keyword Reference

Each line in the input file is of the form: *keyword arguments* with some keywords having multiple arguments, others having none at all. Lower case single letters (perhaps with subscripts) are used to represent integer arguments. Upper case single letters correspond to real (i.e., floating point) arguments. All other arguments are strings. All keywords and strings should be lower case. This first group of keywords is intended only for use in the preparatory stage of the code; invoking them after the *end_prep* command will likely result in the code crashing or behaving strangely.

Preparatory Section

symmetry **sym** describes the symmetry of the geometry with **sym** = *plane*, *cylindrical* (“toroidal symmetry”), *oned* (for a planar system with two symmetry directions), *plane.hw* (nearly symmetric planar system with resolution in the *y* direction), *cylindrical.hw* (nearly cylindrically symmetric system with resolution in the toroidal direction), *cylindrical.section* (same as *cylindrical.hw*, but over a limited range of toroidal angles). The *symmetry* keyword must appear prior to the *end.prep* keyword.

dg.file **filename** specifies the name of an output file from the DG code (with the extension **.dgo**). If **filename** does in fact contain **.dgo**, the dimensions therein are assumed to be in millimeters; filenames without this extension are assumed to have data in meters (data perhaps generated by some other means). The coordinates of the “nodes” (or points) specified in the file are read in. Their ordering in the file is that of the “elements” (or segments) to which the nodes belong. A list of elements in terms of the node numbers is compiled as the file is read in. Each of the nodes is then characterized as being of a particular type:

1. **node_no_elements** if the node has no corresponding elements,
2. **node_one_element** if the node is associated with only one element,
3. **node_regular** if the node is associated with two elements with normals of the same sense,
4. **node_mixed_normals** if the node is associated with two elements having normals in opposite sense (i.e., the node is either the “start” or the “end” of both elements),
5. **node_many_elements** if the node represents the intersection of more than two elements.

Once each of the nodes has been characterized, the resulting information permits the translation of the elements into “walls” that can be used in the subsequent specification of polygons. The code loops over all of the irregular nodes (that are connected to at least one element). Each of these serves as the start of a wall. The rest of the wall is mapped out from the adjacent elements using the well-defined connectivity associated with the regular nodes comprising those elements. The process is stopped, of course, when the trail ends at another irregular node. A subsequent search of the elements that have not been assigned to walls is performed to identify closed surfaces consisting of only regular nodes. The upshot of this procedure is to be cognizant of the location of irregular nodes when using DG. Handling them carefully there can result in a simpler set of walls and, hence, ease the task of designing the polygons.

Suitably updated versions of DG permit the polygons to be defined there first using DG’s graphical interface. DG counts these as they are defined; the resulting integer label is used with the *dg.polygon* keyword in the second part of the *definegeometry2d* file. In the DG output file, the polygons are delimited by a line of the form “polygon *i*”, where “*i*” is this integer label. Inside that polygon specification is the “wall” section, a list of the DG elements comprising that polygon. This is followed by a “polymat” parameter that is currently not being used; it may eventually be used to set the material associated with a polygon. Finally, the sections “meshcon1” and “meshcon2” describe connections between the elements in the polygon’s “wall” and the outer edge of the plasma mesh associated with the DG file. The code that processes the DG file assumes that these sections of the polygon specification appear in precisely this order.

sonnet_mesh filename n_x n_z i_{x,min} i_{x,max} i_{z,min} i_{z,max} specifies a file, **filename**, containing a plasma mesh in the Sonnet format. Files not generated by Sonnet (or, equivalently, by Carre) can be used, but the user should familiarize themselves with the details of the routine that reads the file, *read_sonnet_mesh*. The Sonnet format specifies the plasma mesh as a n_x by n_z rectangular grid of quadrilaterals. The designation of the x and z directions is somewhat arbitrary and does not necessarily have any correspondence with the (Cartesian) X and Z coordinates referred to elsewhere in this program. The usual tokamak configuration has the x coordinate following along flux surfaces and the z coordinate going across flux surfaces. Each quadrilateral is given as a set of four corners and a center so that the mesh can be treated as completely unstructured and no connectivity information is needed. On the other hand, corners of adjacent mesh cells that are intended to match should match *exactly*. In defining the surrounding polygons, the user will undoubtedly need to be familiar with the mesh's connectivity in defining the surrounding polygons. The ordering of the corners is important. See below and in subroutine *read_sonnet_mesh* for more details.

If the entire mesh is to be read in, only n_x and n_z parameters need to be specified. The other four parameters allow the user to describe and use a rectangular subset of the mesh. One might want to do this if the mesh is physically much larger than the volume of interest. By isolating a subset of the mesh with these four parameters, its boundaries can be used to construct polygons via the *edge* keyword in the second part of this input file. Once the mesh subset has been read in, the original mesh indexing is forgotten. The code will proceed as if an entire mesh of size $n_x = i_{x,max} - i_{x,min} + 1$ by $n_z = i_{z,max} - i_{z,min} + 1$ were read in. The user will need to be mindful of this index shifting when invoking the *edge* keyword.

Note that these parameters are used as if the mesh indices begin at 1. Since the Sonnet format typically starts the mesh indices at 0, the values of n_x and n_z should be 1 greater than the indices of the last cell in the Sonnet file. The same is true for the subset indices so that, for example, $i_{x,min} \geq 1$ and $i_{x,max} \leq n_x$.

uedge_mesh filename specifies the name of a file, **filename**, generated by the **UEDGE** code for the purpose of transferring data to **DEGAS 2**. The file contains both geometry and plasma data, although only the former is read in by the *read_uedge_mesh* routine. The first few lines of the file contain additional information describing the mesh. The first line is assumed to be a comment. The first number on the second line is assumed to be n_x . Likewise, the third line provides n_z . The next three lines are ignored. As with the Sonnet format, a center and four corners are given for each quadrilateral and no connectivity information is needed. See the routine *read_uedge_mesh* for more details on the order of the corners.

wallfile filename specifies the name of a file, **filename**, that contains a list of walls, each comprised by two or more points. Like other **DEGAS 2** text files, *wallfiles* can contain blank lines and comments (beginning with #). Spacing should not matter. The first line provides the number of walls in the file, *num_new_walls*. The following line or lines list the number of points comprising each wall. I.e., these lines must contain *num_new_walls* integers. The rest of the file contains the coordinates in ordered pairs, with one (X, Z) pair per line, starting with the first point of the first wall. The other points in that wall follow on consecutive lines. The points in the second wall (if there is one) come immediately after that with no delimiter (the code is using only the number of points provided in line two to determine which point goes in which wall), and so on. For simple one wall systems, the file can be easily created by hand. For more complicated systems involving several walls, the user may benefit from writing a code to generate the file.

print_min_max loops over all of the nodes (read in via the *dg_file* or *wallfile* keywords) and prints to standard out the maximum and minimum R and Z values (in meters). These can be useful in choosing suitable bounds for the universal cell.

bounds $X_{\min} X_{\max} Z_{\min} Z_{\max} Y_{\min} Y_{\max}$ specifies the corners of the universal cell in X ($X_{\min} \rightarrow X_{\max}$), Z ($Z_{\min} \rightarrow Z_{\max}$), and in nearly symmetric 3-D cases Y or ϕ ($Y_{\min} \rightarrow Y_{\max}$). The *outer* keyword can be used to incorporate these points into polygons.

The Y values are not needed for 2-D (*symmetry = plane* or *cylindrical*) or full torus 3-D (*symmetry = cylindrical.hw*) cases. For nearly symmetric 3-D plane cases (*symmetry = plane.hw*), the Y values are in meters. For 3-D cylindrical section cases (*symmetry = cylindrical.section*), the Y values describe the range of toroidal angles in degrees that are to be used in the problem (note, however, that these are converted to radians during processing). Negative values are permitted, e.g., so that the user can put $\phi = 0$ in a convenient location. The only constraint is that $0 < Y_{\max} - Y_{\min} < 360^\circ$. When the universal cell is defined with the invocation of the *end_prep* command, it will have surfaces at $\phi = Y_{\min}$ and $\phi = Y_{\max}$. for the full torus 3-D (*cylindrical.hw*) case, $Y_{\min}$ and $Y_{\max}$ may be specified; if they are not, they are assumed to be $Y_{\min} = 0$ and $Y_{\max} = 360^\circ$.

print_walls **format filename** instructs the code to write out the current set of walls. If **filename** is not provided, the data are written to standard out. There are two possible **formats**. The first, **tabular**, has the walls laid out in successive columns, suitable for plotting with an external program. The second, **linear**) is consistent with the format used by the *wallfile* keyword.

uniform_ys n_y sets up a uniform grid in the y or ϕ direction of nearly symmetric 3-D problems. This keyword can only be used with *symmetry* values *plane.hw*, *cylindrical.hw*, and *cylindrical.section*. In all three cases, n_y uniformly spaced $y = \text{constant}$ or $\phi = \text{constant}$ surfaces will be defined, breaking the problem up into equal size $n_y - 1$ zones in the third dimension.

In the *plane.hw* and *cylindrical.section* cases, the first and last of these surfaces are offset from the universal cell surfaces at $Y_{\min}$ and $Y_{\max}$ by a small amount, $0.01(Y_{\max} - Y_{\min})$, so that border zones can be defined there.

y_values **filename OR** $y_1 y_2 y_3 \dots$ This keyword allows the user to directly specify the grid in the y or ϕ direction for nearly symmetric 3-D problems. This keyword can only be used with *symmetry* values *plane.hw*, *cylindrical.hw*, and *cylindrical.section*. In planar cases, the y_i values are in meters; in cylindrical cases, they are in degrees (but converted to radians internally). In all instances, the user must have $Y_{\min} \leq y_i \leq Y_{\max}$ with the y_i monotonically increasing with $0 \leq i \leq n_y - 1$ (there will be n_y surfaces defined at the specified y values; hence, there are $n_y - 1$ segments in the third dimension). In the bounded cases (i.e., not *cylindrical.hw*), a small gap is needed between the first and last surfaces and the bounding universal cell surfaces, $y_0 > Y_{\min}$ and $y_{n_y-1} < Y_{\max}$. In full toroidal cases (*cylindrical.hw*), the code expects $y_0 = Y_{\min}$ and $y_{n_y-1} = Y_{\max}$. If the list of y values is too long to fit on a single line, they can instead be put into a file **filename**, one per line. This also facilitates commenting as in other DEGAS 2 text input files.

end_prep terminates the preparatory stage of the code. Dynamically allocated arrays associated with nodes, elements, and walls are trimmed to their final sizes, and the universal cell is established. If a plasma mesh has been specified via the *sonnet.mesh* or *uedge.mesh* keywords, the constituent quadrilaterals are translated into polygons and then decomposed into the internal elements of the DEGAS 2 geometry with each quadrilateral corresponding to a plasma zone. One advantage of making these the first zones defined in the problem is that transfer of zone-based output information to other codes (such as a fluid plasma code) is simplified.

Construction Section The rest of the keywords are associated with the “construction” section of the input file. While they may (even legally) be used prior to the *end_prep* command, such usage is not encouraged in the interest of maintaining the simplicity of the input file. One exception is the *quit* keyword that can be used anywhere in the input file (except inside a polygon definition). There is a further subdivision in that all keywords regarding a specific polygon must appear between its *new_polygon* command and the command that denotes its completion (*clear_polygon*, *breakup_polygon*, *triangulate_polygon*, or *triangulate_to_zones*). Likewise, all of the keywords pertaining to a diagnostic must appear between its *new_diagnostic* and *end_diagnostic* commands.

new_zone type alt_type i₁ i₂ i₃ ... starts definition of a new zone. All polygons specified after this command and prior to the next *new_zone* command will be added to this zone. For 2-D problems, just the first argument, **type**, is allowed. The possible values of **type** are **vacuum**, **plasma**, **solid**, and **exit**.

In nearly symmetric 3-D cases (*symmetry* must be *plane_hw*, *cylindrical_hw* or *cylindrical_section*), the zone type can be varied in the third dimension. If no variation is desired, again specify just the **type** with the *new_zone* command. To use mixed zone types, use the first argument **type** to specify the dominant zone type. The second zone type and its locations are determined by the remaining arguments. The **alt_type** provides the zone type and again must be **vacuum**, **plasma**, or **solid**. The following indices correspond to the grid in the *y* or ϕ locations specified by the *uniform_ys* or *y_values* keyword. The first segment of the grid corresponds to $i = 0$; the last to $i = n_y - 2$ (since there are $n_y - 1$ segments in the third dimension).

For convenience, a range of *i* values can be specified as *i - j* (with a space on either side of “-”). That is, the range “3 4 5 6” could be abbreviated as “3 - 6”.

Zones of mixed type must be decomposed with the *triangulate_to_zones* command. Note that currently this option cannot be used with exit zones. Nearly the same functionality can be obtained by specifying a solid zone with a recycling coefficient of zero.

new_polygon begins the definition of a new polygon. The following keywords (down to and including *triangulate_to_zones*) can be used in describing this polygon.

outer i j k adds to the current polygon the *i*th, *j*th, and *k*th points of the bounding rectangle of the problem (defined with the *bounds* keyword, i.e., the minimum and maximum *X* and *Z* used to define DEGAS 2’s “universal cell”). The numbering of the corners is clockwise:

- 0 (or 4) ($X_{\min}, Z_{\min}$)
- 1 ($X_{\min}, Z_{\max}$)
- 2 ($X_{\max}, Z_{\max}$)
- 3 ($X_{\max}, Z_{\min}$)

There can be any number of arguments (e.g., see the above example input file).

wall i_{wall} j_{start} j_{stop} reverse takes points from wall number *i_{wall}* that was specified with either the *wallfile* or *dg_file* command. The parameters *j_{start}* and *j_{stop}* prescribe which points from that wall to use. Note that the first point of the wall is numbered 0 so that the only valid values of *j_{start}* and *j_{stop}* are between 0 and the number of elements in the wall. If the wall needs to be traversed in the reverse direction, *j_{stop}* can be set less than *j_{start}*. If *j_{start}* = *j_{stop}*, a single point is added to the polygon. If both *j_{start}* and *j_{stop}* are replaced by *, all of the points in the wall are used. The wildcard character can also be used as: *j_{start} ** to indicate that all of the points from *j_{start}* to the end of the wall should be used. The additional argument **reverse** can be added at the end of the line to reverse the ordering of the points indicated by *j_{start}*, *j_{stop}*, and / or *. The primary usage of **reverse**, however, will be in conjunction with *.

edge $i_{x,start}$ $i_{x,stop}$ $i_{z,start}$ $i_{z,stop}$ **xcut** **reverse** takes an “edge” from a plasma mesh defined with either the *sonnet_mesh* or *uedge_mesh* keywords and adds it to the current polygon. To properly refer to an edge, the arguments must be such that either $i_{x,start} = i_{x,stop}$ or $i_{z,start} = i_{z,stop}$. Valid values of $i_{x,start}$ and $i_{x,stop}$ will be between 0 and n_x , inclusive. Likewise, $i_{z,start}$ and $i_{z,stop}$ will be between 0 and n_z . Either pair can be replaced with $*$ to indicate that the entire edge should be added to the polygon. The code used to process the arguments “knows” about the four corners comprising each mesh zone so that **edge** $* 0 0$ actually refers $n_x + 1$ points (see also the additional detail below on corners). If a subset of a Sonnet mesh is being used, these values are relative to it, not to the original, full mesh.

The one instance where this is not quite true is at a discontinuity (a “cut”) in the x direction of the computational mesh. On the “right” side of the cut (“right” in the topological sense; i.e., the mesh cells i_x and $i_x - 1$ do *not* share a common edge), adding the optional *xcut* argument to the *edge* command with $i_{x,start} = i_{x,stop} = i_x$ will allow the edge of the i_x cell along this cut to be added to the polygon. Without the *xcut* argument, this command would instead yield segments following the boundary between the i_x and $i_x + 1$ cells. On the “left” side of the cut (the mesh cells i_x and $i_x + 1$ do not share a common edge), the usual *edge* command with $i_{x,start} = i_{x,stop} = i_x$ will work as is, and the *xcut* command is not needed. The *xcut* argument may also be needed in conjunction with an $i_{y,start} = i_{y,stop}$ edge to specify a single point on the “right” side of a cut. Doing so requires a *separate edge* command for that point (i.e., it would have both $i_{x,start} = i_{x,stop}$ and $i_{y,start} = i_{y,stop}$, as well as the *xcut* argument). The specific implementation of the *xcut* option is described below under “Other Considerations”.

As with the *wall* keyword, the *reverse* argument can be used to indicate that the points should be added to the polygon in the order opposite to that indicated by the other arguments. One, none, or both of the *xcut* and *reverse* options may appear. If both are present, they do not need to be in a particular order.

dg-polygon i_{polygon} adds the segments associated with the i_{polygon} th polygon defined in DG to the current polygon. This functionality allows the user to set up polygons in DG rather than going through the more tedious and error prone process of defining them in this input file with the *wall* and *outer* keywords. Different types of DG polygons are possible:

1. If the DG polygon defines a closed figure, no other points need be specified for this *new-polygon* group, although the user should still assign *stratum*, *material*, *temperature*, and *recyc-coef* values as needed. The resulting polygon will have a clockwise orientation.
2. If the polygon is not closed, it can be closed by various means,
 - (a) The DG polygon’s “mesh connection” data identify a single point on the edge of the plasma mesh. This point will be used to close the polygon without any additional input from the user.
 - (b) The DG polygon’s “mesh connection” data identify a pair of points on edge of the plasma mesh, with the implication that the set of mesh edge elements “between” them will be used to close the polygon. However, this set is uncertain since it could consist of either the clockwise or counter-clockwise path between the specified points; the user must invoke an *iedge* command to determine which should be used.
 - (c) No “mesh connection” data are specified. This would be the case if the user wishes to complete the polygon with another command (e.g., *edge* or *wall*); *dg-polygon* must be the first command to add points to the polygon in this case. This approach is not recommended since the orientation of the DG polygon may not be as expected. If that is the case, the user can invoke the *reverse* command immediately after *dg-polygon*.

iedge **edge**₁ . . . specifies one or more edges of plasma mesh defined with either the *sonnet_mesh* or *uedge_mesh* keywords. The edges are identified by the strings *S*, *E*, *N*, and *W* (referring to south, east, north, and west), corresponding to the $i_z = 0$, $i_x = n_x$, $i_z = n_z$, $i_x = 0$ boundaries, respectively.

reverse instructs the code to reverse the order of all points that have been added to the current polygon; the polygon need not be complete.

stratum i_{stratum} labels the polygon as belonging to stratum number i_{stratum} . It will be applied to subsequent polygons until used again.

material **mat** sets the material for the polygon to the material having the symbol **mat**. A material with this symbol must appear in the *materials_infile* (named in the *degas2.in* file) for this run. It will be applied to all subsequent polygons until used again. Note that this setting only affects the calculation when one or more edges of this polygon ends up being a target or wall sector.

temperature T associates a temperature of T Kelvin (a real number) to the polygon. Its value currently is used only to determine the energy of desorbed species, although other applications may be incorporated. It will be applied to all subsequent polygons until used again. Note that this setting only affects the calculation when one or more edges of this polygon ends up being a target or wall sector.

recyc_coef R associates a recycling coefficient of R (R a real number, $0 \leq R \leq 1$) to the polygon. It will be applied to all subsequent polygons until used again. Note that this setting only affects the calculation when one or more edges of this polygon ends up being a target or wall sector. Setting $R = 0$ allows a purely absorbing boundary to be modeled.

triangle_area A specifies a minimum area in square meters for the triangles to be created by **Triangle** (with A a real number) when the *triangulate_to_zones* command is executed. Practically, it seems that this ends up as little more than a suggestion. The reason is that we cannot allow **Triangle** to add points along the boundary of the polygon. Although this would not compromise the definition of DEGAS 2 surfaces from the triangulation of the current polygon, the surface(s) of the adjacent polygon(s) would not be subdivided in the same way. Hence, the exact correspondence of adjacent surfaces that the code relies upon for establishing connectivity would be destroyed. **Triangle** can, however, add points to the interior of a polygon; this parameter A can be used to control that process.

triangle_hole n X Z specifies an integer number of holes, n . This is currently constrained to be a single hole, $n = 1$. The coordinates (real numbers) of the hole are given as X and Z . The hole location is used by **Triangle** when either the *triangulate_polygon* or *triangulate_to_zones* command is executed on a polygon that encircles a region that is *not* to be broken up into triangles. The hole location can be anywhere in this interior region.

aux_stratum i_{stratum} initiates the definition of an auxiliary stratum (consisting of one or more auxiliary sectors) labeled as stratum number i_{stratum} . The definition is terminated by an *end_aux_stratum* command. All of the polygon segments specified between the two commands will be used to define auxiliary sectors in this stratum. More precisely, sectors will be set up starting at each *point* between the two commands; the end point of each sector being the next point in the polygon. Thus, the end point of the last auxiliary sector is the first point specified *after* the *end_aux_stratum* command. For this to work correctly, *the polygon must have a clockwise orientation*. The *aux_stratum* and *end_aux_stratum* commands may be invoked multiple times across various polygons, with the same or different values of i_{stratum} . In this way, segments of multiple polygons can be incorporated into a single stratum.

end_aux_stratum terminates the definition of an auxiliary stratum begun with an *aux_stratum* command.

print_polygon **filename** prints the points of the current polygon to a file, **filename**. If **filename** is not present, the coordinates are written to standard out.

clear_polygon will cause the code to completely forget about this polygon in subsequent operations. This is useful for checking the integrity of several polygons with a single run of the code.

breakup_polygon instructs the code to use DEGAS 2's original *decompose_polygon* routine to break the polygon up into the internal geometry elements. This routine expects the polygon to be traced out in a clockwise direction. Note also that *decompose_polygon* is far from foolproof and will occasionally fail with a "unable to decompose polygon" error. This problem can usually be remedied by choosing a different point of the polygon as the first point. On the other hand, the functionality of the vastly more robust *triangulate_polygon* routine is equivalent. So, try using it instead when this error arises. The two approaches may result in different run efficiencies, but no such variability has ever been established.

triangulate_polygon instructs the code to use the **Triangle** package to break the polygon up into DEGAS 2's internal geometry elements. The **Triangle** routine is extremely tolerant and robust; the input polygon can be traced out in either direction. However, a clockwise orientation is recommended since this will preserve (as *strata_segment* labels) the relationship between the resulting bounding segments and the original polygon points. This command does only a basic triangulation and is only recommended for use with polygons in solid zones. In nearly symmetric 3-D cases, these zones *will not* be resolved in the i_y direction.

triangulate_to_zones instructs the code to use the **Triangle** package to break the polygon up into DEGAS 2's internal geometry elements. The **Triangle** routine is extremely tolerant and robust; the input polygon can be traced out in either direction. Each resulting triangle is added to the geometry as a separate zone (of the current type). A second refining call to **Triangle** beyond the one analogous to that done with *triangulate_polygon* is performed that permits the application of additional constraints such as a minimum area. A clockwise orientation is recommended since this will preserve (as *strata_segment* labels) the relationship between the resulting bounding segments and the original polygon points. This option also ensures that zones are resolved in the i_y direction in nearly symmetric 3-D problems. **Note: the next polygon to appear in the file must be preceded by a *new_zone* command.** (This is the last of the polygon-specific keywords.)

new_diagnostic **diagnostic name** initiates the definition of a diagnostic group (of sectors) associated with the name **diagnostic name** (the name of the diagnostic may contain spaces). **The following keywords can be used to describe this diagnostic.** An *end_diagnostic* command terminates its definition. The only other command that must be used is *stratum*; the others listed below are optional.

stratum i_{stratum} **type** designates a previously defined stratum that will determine the sectors to be used for this diagnostic. If i_{stratum} is associated with a default sector (i.e., corresponds to a stratum number specified with the *stratum* keyword inside a polygon or diagnostic definition), the optional argument **type** determines which of the two types of default sectors will be used. If **type** is set to **solid**, the diagnostic will be comprised of the wall, target, or exit sectors created from that polygon with that stratum label. If the **type** argument is omitted, its value is assumed to be **solid**. Alternatively, if **type** is set to **nonsolid**, the adjoining plasma or vacuum sectors are used. When an auxiliary stratum is used, all sectors of that stratum will be incorporated.

This capability can be used to duplicate and / or extend (e.g., to increase resolution in energy or angle of incidence) the default diagnostics and associated tallies without having to modify the code.

variable **var** requests that an independent variable mesh for the physical quantity **var** be set up for the diagnostic being defined by a preceding *new_diagnostic* command. The default is no variable; in that case, the diagnostic lumps together all flights passing through its sectors. By using **energy** or **angle** for **var**, the user can resolve these passages according to the flights energy or angle (angle of incidence relative to the normal to the sector), respectively. The user can control the units used for the energy mesh by instead using for **var** **energy J** or **energy eV** (the default is eV). Likewise, the angle can be in degrees, **angle degrees** (default) or radians, **angle radians**.

number n_{var} specifies the number of values in the independent variable mesh for the diagnostic being defined in conjunction with a preceding *new_diagnostic* command.

minimum $V_{\min}$ specifies the minimum value of an independent variable mesh being defined in conjunction with a preceding *new_diagnostic* command.

maximum $V_{\max}$ specifies the maximum value of an independent variable mesh being defined in conjunction with a preceding *new_diagnostic* command.

multiplier F specifies a multiplier to be applied to the *maximum* and *minimum* of a diagnostic independent variable mesh. This command allows the user to utilize units *other than* those described above under *variable*. The multiplier must result in MKS units for the mesh (i.e., J for energy and radians for angle).

spacing **spac** specifies the spacing of an independent variable mesh being defined by a preceding *new_diagnostic* command. The only valid values for **spac** are **linear** and **log**. No default is set, so one of these must appear explicitly.

end_diagnostic terminates the definition of the diagnostic begun with a preceding *new_diagnostic* command.

y_min_zone **mat** i_{stratum} $i_{p,\text{stratum}}$ specifies that the $Y_{\min}$ boundary of bounded, nearly symmetric 3-D cases (*symmetry* = *plane.hw* or *cylindrical.section*) is to be assigned the material with symbol **mat**. A material with this symbol must appear in the *materials_infile* (named in the *degas2.in* file). Sectors based on this zone will be labeled with stratum number i_{stratum} . If the optional second stratum label $i_{p,\text{stratum}}$ is present, sectors based on the adjacent (corresponding to $i_y = 0$) zone are also defined and assigned stratum label $i_{p,\text{stratum}}$.

y_max_zone **mat** i_{stratum} $i_{p,\text{stratum}}$ specifies that the $Y_{\max}$ boundary of bounded, nearly symmetric 3-D cases (*symmetry* = *plane.hw* or *cylindrical.section*) is to be assigned the material with symbol **mat**. A material with this symbol must appear in the *materials_infile* (named in the *degas2.in* file). Sectors based on this zone will be labeled with stratum number i_{stratum} . If the optional second stratum label $i_{p,\text{stratum}}$ is present, sectors based on the adjacent (corresponding to $i_y = n_y - 2$) zone are also defined and assigned stratum label $i_{p,\text{stratum}}$.

polygon_nc_file **filename** will cause the netCDF file containing the polygon description of the geometry (based on the **g2** class, as described in the file *geometry2d.hweb*) to be named **filename**. The default file name is **polygon.nc**.

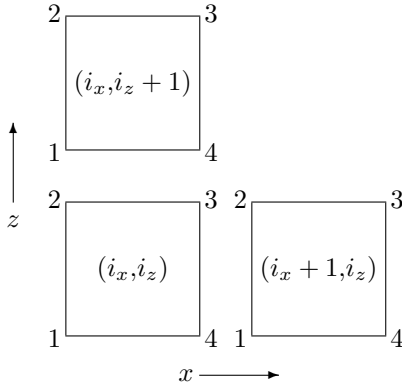
quit causes the code to terminate immediately. This is useful for preliminary runs in which the geometry is known to be inconsistent or incomplete, but the code needs to be run so that walls or polygons can be printed out for external examination.

end denotes completion of the geometry specification. A variety of derived relationships are set up by subroutine calls. The geometry is checked (thoroughly) and written to its netCDF file.

7 Other Considerations

Regarding corners with the edge keyword: Because the **UEDGE** and **Sonnet** mesh formats both provide four corners for each mesh zone, the number of points along each “edge” of the mesh is one greater than the number of zones along that edge. This fact must be taken into account in defining polygons along these edges.

For clarity, here is the convention used to number the corners in *definegeometry2d*:



In the **Sonnet** mesh format, the first line contains corners 2 and 3. The second line specifies the center. Corners 1 and 4 are on the third line. The **UEDGE** mesh format specifies the corners for each point in the order 0 (center), 1, 4, 2, and 3.

For a particular (i_x, i_z) specified via arguments to the *edge* keyword, the logic that determines the corner to be used is:

1. If $i_x = 0$ and $i_z = 0$, corner 1 of the $(1, 1)$ zone (recall that there is no $(0, 0)$ zone).
2. If $i_x = 0$, but $i_z \neq 0$, corner 2 of the $(1, i_z)$ zone.
3. If $i_x \neq 0$, but $i_z = 0$, corner 4 of the $(i_x, 1)$ zone.
4. Otherwise, use corner 3 of the (i_x, i_z) zone.

If the *xcut* argument for the *edge* keyword has been invoked, a separate set of logic is required:

1. $i_x \neq 0$ is asserted. The above logic already handles this case.
2. If $i_z = 0$, corner 1 of the $(i_x, 1)$ zone.
3. Otherwise, use corner 2 of the (i_x, i_z) zone.

In this way, the “left” edge of the i_x zone can be specified; the usual logic results in the “right” edge (except at $i_x = 0$).

Regarding the definition of detectors: Ideally the user would also be able to specify detectors via this input file. Instead, a placeholder subroutine called *detector_setup* has been defined in the file *def2ddetector.web*. The user can write a replacement routine that will be loaded in automatically if it is in a file named *usr2ddetector.web*. An example is contained in the file *btopdetector.web*. In practice, one would use such a file by executing commands analogous to:

```
cd $HOME/degas2/src
cp btopdetector.web usr2ddetector.web
touch usr2ddetector.web
cd ../SUN
gmake definegeometry2d
```

Likewise, to revert to the default (null) subroutine,

```
cd $HOME/degas2/src
rm usr2ddetector.web
touch def2ddetector.web
cd ../SUN
gmake definegeometry2d
```

Note the use of the **touch** command to make sure **gmake** recognizes the change.

Regarding the orientation of sectors: Sectors are defined using a zone and a surface bounding that zone. A flight leaving that zone through that surface is considered to be traveling “out” of that sector, establishing the applicability of the direction-sensitive variables available in **tallysetup**. For example, the flight would make a contribution to a sector-based tally for which the dependent variable is **mass_out**, but not for one in which the variable is **mass_in**.

The default diagnostics (see the “Diagnostic Sectors” section of the User’s Manual) are comprised of default wall, target, and exit sectors. So, flights traveling out of adjacent plasma or vacuum zones “into” a sector incorporated into one of these diagnostic groups will make contributions to corresponding tallies that have a dependent variable of **mass_in**, **momentum_in_vector**, or **energy_in**; these tallies represent sinks for the test species.

If the plasma-material interaction processes that result from the flight striking this surface yield one or more products to be tracked back into the plasma or vacuum, contributions will be made to tallies based on the same diagnostic group but having an outwardly directed dependent variable (e.g., **mass_out**).

The “auxiliary sectors” capability provided by this code allows the definition of:

1. Wall, target, or exit sectors at locations of specific interest. Such sectors will work just as the corresponding default sectors do.
2. Vacuum or plasma sectors at locations of specific interest. In nearly symmetric 3-D cases, these might be used to determine the variation of fluxes in the third dimension. These can also be used to track fluxes between adjoining plasma or vacuum zones, where there are no default sectors at all.

The former are set up using the *aux-stratum* and *end-aux-stratum* commands inside a solid polygon definition clause. The latter would have these keywords inside a vacuum or plasma polygon definition. The direction-sensitive variables for the former behave as above; the latter work in the opposing manner. I.e., a **mass_in** based tally represents a source for a plasma or vacuum sector. Since nothing should be lost across the interface, the “**mass_in**” contribution to a solid sector should equal the **mass_out** of an adjoining plasma or vacuum sector (of the same extent in the third dimension), and vice versa.

Special note on comments and debuggers: Part of this file is written in C. The current Sun Workshop debugger (there may be others) utilizes the original FWEB file in stepping through the source code of the C routines. Getting the object file and the source code lines to match up requires exercising some care in writing comments. First, do not use the // comment style on a single line. Second, do not put blank lines before or after multiline (delimited by /* */ comments). There may be other requirements as well.

\$Id: definegeometry2d.web,v 1.7 2007/06/19 17:34:28 dstotler Exp \$

```
"definegeometry2d.f" 1 ≡
  @m FILE 'definegeometry2d.web'

  @m dg_loop #:0
  @m meshcon_loop #:0
  @m wall_loop #:0
  @m wall_loop2 #:0
  @m wall_break #:0
  @m sector_break #:0
  @m sector_break2 #:0
  @m next_surface #:0
  @m y_loop #:0
  @m y_loop2 #:0
  @m mixed_type_loop #:0
```

The unnamed module.

```
"definegeometry2d.f" 7.1 ≡
@Lc:   <Functions and subroutines 8>
       #define REAL double    // Need to be sure this does not overlap with local macros
       #include <stdio.h>
       #include "triangle.h"    // Dependency explicitly included in Makefile
       <C Functions 36>C
```

8 The Main Program

```

"definegeometry2d.f" 8 ≡
  @m element_start 0
  @m element_end 1

  @m node_undefined 0
  @m node_regular 1
  @m node_no_elements 2
  @m node_one_element 3
  @m node_many_elements 4
  @m node_mixed_normals 5

  @m sec_undef 0
  @m sec_p1 1
  @m sec_p2 2
  @m sec_miss 3
  @m sec_skip 4
  @m sec_dg_poly 5
  @m sec_dg_wall 6
  @m sec_dg_meshcon1 7
  @m sec_dg_meshcon2 8
  @m sec_done 40    // Try to match to maximum?

  @m meshcon_max 2    // Number of meshcon sections in DG file
  @m meshcon_elem_max 2    // Number of elements in a meshcon
  @m mesh_h 0    // Labels for elements
  @m mesh_v 1

  @m poly_stratum 1    // Indices for poly_int_props
  @m poly_material 2
  @m poly_num_holes 3
  @m poly_int_max 3

  @m poly_temperature 1    // Indices for poly_real_props
  @m poly_recyc_coef 2
  @m poly_min_area 3
  @m poly_hole_x 4
  @m poly_hole_z 5
  @m poly_real_max 5

  @m clear_polygon 0
  @m breakup_polygon 1
  @m triangulate_polygon 2
  @m triangulate_to_zones 3

  @m max_triangles 5500

  @m increment_num_elements num_elements++
      dim_elements = max(dim_elements, num_elements)
      var_realloca(dg_elements_list)
      var_realloca(element_missing)
      var_realloca(element_skipped)
      var_realloca(element_assigned)

  @m increment_num_nodes num_nodes++

```

```

    dim_nodes = max(dim_nodes, num_nodes)
    var_realloca(nodes)
    var_realloca(node_type)
    var_realloca(node_element_count)

@m increment_num_dg_poly num_dg_poly++
    dim_dg_poly = max(dim_dg_poly, num_dg_poly)
    var_realloca(dg_polygons)
    var_realloca(dg_poly_num_elements)
    var_realloca(dg_poly_num_meshcon)
    var_realloca(dg_poly_meshcon)
    var_realloca(dg_poly_meshcon_hv)

@m increment_num_walls num_walls++
    dim_walls = max(dim_walls, num_walls)
    var_realloca(wall_nodes)
    var_realloca(wall_elements)
    var_realloca(wall_segment_count)

@m increment_y_div y_div++
    dim_y = max(dim_y, y_div)
    dim_ym = max(dim_ym, y_div)
    var_realloca(y_values)
    var_realloca(facearray)
    var_realloca(zonearray)
    var_realloca(zone_type_array)

@m increment_g2_num_polygons g2_num_polygons++
    var_realloca(g2_polygon_xz)
    var_realloca(g2_polygon_segment)
    var_realloca(g2_polygon_points)
    var_realloca(g2_polygon_zone)
    var_realloca(g2_polygon_stratum)
    var_realloca(poly_int_props)
    var_realloca(poly_real_props)
    g2_polygon_stratum_g2_num_polygons = int_unused
    do i_inc_g2 = 0, g2_num_points - 1
        g2_polygon_xz_g2_num_polygons,i_inc_g2,g2_x = zero
        g2_polygon_xz_g2_num_polygons,i_inc_g2,g2_z = zero
        g2_polygon_segment_g2_num_polygons,i_inc_g2 = int_unused
    end do

@m increment_num_aux_sectors num_aux_sectors++
    dim_aux_sectors = max(dim_aux_sectors, num_aux_sectors)
    var_realloca(aux_stratum)
    var_realloca(aux_stratum_poly)
    var_realloca(aux_stratum_points)
    var_realloca(aux_stratum_segment)
    aux_stratum_num_aux_sectors = int_uninit
    aux_stratum_poly_num_aux_sectors = int_uninit
    aux_stratum_points_num_aux_sectors = int_uninit
    aux_stratum_segment_num_aux_sectors = int_uninit

@m increment_num_diags num_diags++
    dim_diags = max(dim_diags, num_diags)
    var_realloca(diag_name)

```

```

var_realloca(diag_stratum)
var_realloca(diag_solid)
var_realloca(diag_variable)
var_realloca(diag_tab_index)
var_realloca(diag_var_min)
var_realloca(diag_var_max)
var_realloca(diag_mult)
var_realloca(diag_spacing)
diag_name_num_diags = char_uninit
diag_stratum_num_diags = int_uninit
diag_solid_num_diags = int_uninit
diag_variable_num_diags = int_uninit
diag_tab_index_num_diags = int_uninit
diag_var_min_num_diags = real_uninit
diag_var_max_num_diags = real_uninit
diag_mult_num_diags = real_uninit
diag_spacing_num_diags = int_uninit

@m open_file(aunit, aname)
  open(unit = aunit, file = aname, status = 'old', form = 'formatted', iostat = open_stat)
  if (open_stat ≠ 0) then
    write(stderr, *) 'Cannot open file', line(b:e), ', error number', open_stat
    return
  end if

@m mesh_xzm(jxz, i, ix, iz) mesh_xz(iz-1)*nxd+ix, i, jxz     // “m” for macro.
  /* The values of these macros embody the connectivity of the mesh edges. This fact is exploited in
  the corresponding code. Hence, these should not be changed without considerable thought. */
@m mesh_south 1
@m mesh_east 2
@m mesh_north 3
@m mesh_west 4
@m mesh_num_edges 4     // I.e., should equal last of the above!

⟨ Functions and subroutines 8 ⟩ ≡
program definegeometry2d
  implicit_none_f77
  implicit_none_f90
  character*FILELEN filename

  call readfilenames

  call command_arg(1, filename)
  call nc_read_materials
  call def_geom_2d_main(filename)

  stop
end

```

See also sections 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, and 35.

This code is used in section 7.1.

9 Main Subroutine

⟨ Functions and subroutines 8 ⟩ +≡

```
subroutine def_geom_2d_main(filename)

  define_dimen(node_ind, dim_nodes)
  define_dimen(element_ind, dim_elements)
  define_dimen(element_ends_ind, element_start, element_end)
  define_dimen(dg_poly_ind, dim_dg_poly)
  define_dimen(dg_poly_meshcon_ind, meshcon_max)
  define_dimen(dg_poly_meshcon_elem_ind, meshcon_elem_max)
  /* Add "2d" to distinguish from wall_ind in sector.hweb. */
  define_dimen(wall2d_ind, dim_walls)
  define_dimen(aux_sector_ind, dim_aux_sectors)
  define_dimen(diags_ind, dim_diags)
  define_dimen(mesh_corner_ind, 0, 4)
  define_dimen(mesh_xz_ind, nxd * nzd)
  define_dimen(mesh_nodes_ind, 4 * (nxd + nzd))
  define_dimen(mesh_elements_ind, 2 * (nxd + nzd))
  define_dimen(mesh_edge_elements_ind, nx_nz_max)
  define_dimen(mesh_senv_ind, mesh_num_edges)
  define_dimen(poly_int_ind, poly_int_max)
  define_dimen(poly_real_ind, poly_real_max)
  define_dimen(y_ind, 0, dim_ym)
  define_dimen(y_indm, 0, dim_y - 1)
  define_dimen(y_sect_ind, dim_y)

  define_varp(nodes, FLOAT, g2_xz_ind, node_ind)
  define_varp(node_type, INT, node_ind)
  define_varp(node_element_count, INT, node_ind)

  define_varp(dg_elements_list, INT, element_ends_ind, element_ind)
  define_varp(element_missing, INT, element_ind)
  define_varp(element_skipped, INT, element_ind)
  define_varp(element_assigned, INT, element_ind)

  define_varp(dg_polygons, INT, g2_points_ind, dg_poly_ind)
  define_varp(dg_poly_num_elements, INT, dg_poly_ind)
  define_varp(dg_poly_num_meshcon, INT, dg_poly_ind)
  define_varp(dg_poly_meshcon, INT, dg_poly_meshcon_elem_ind, dg_poly_meshcon_ind, dg_poly_ind)
  define_varp(dg_poly_meshcon_hv, INT, dg_poly_meshcon_elem_ind, dg_poly_meshcon_ind, dg_poly_ind)

  define_varp(wall_nodes, INT, g2_points_ind0, wall2d_ind)
  define_varp(wall_elements, INT, g2_points_ind, wall2d_ind)
  define_varp(wall_segment_count, INT, wall2d_ind)

  define_varp(y_values, FLOAT, y_ind)
  define_varp(facearray, INT, y_ind)
  define_varp(zonearray, INT, y_indm)
  define_varp(zone_type_array, CHAR, string, y_indm)
  define_varp(sect_zone1, INT, g2_poly_ind)
  define_varp(sect_zone2, INT, g2_poly_ind)

  define_varp(aux_stratum, INT, aux_sector_ind)
  define_varp(aux_stratum_poly, INT, aux_sector_ind)
```

```

define_varp(aux_stratum_points, INT, aux_sector_ind)
define_varp(aux_stratum_segment, INT, aux_sector_ind)

define_varp(diag_name, CHAR, sc_diag_name_string, diags_ind)
define_varp(diag_stratum, INT, diags_ind)
define_varp(diag_solid, INT, diags_ind)
define_varp(diag_variable, INT, diags_ind)
define_varp(diag_tab_index, INT, diags_ind)
define_varp(diag_var_min, FLOAT, diags_ind)
define_varp(diag_var_max, FLOAT, diags_ind)
define_varp(diag_mult, FLOAT, diags_ind)
define_varp(diag_spacing, INT, diags_ind)
    /* Since we can have only one adjustable dimension with pointers, combine the two into a single one
       here. We can use the desired 3-D representation in subroutines accessing these arrays. */
define_varp(mesh_xz, FLOAT, g2_xz_ind, mesh_corner_ind, mesh_xz_ind)
define_varp(mesh_nodes, FLOAT, g2_xz_ind, mesh_nodes_ind)
define_varp(mesh_elements, INT, element_ends_ind, mesh_elements_ind)
define_varp(mesh_edge_elements, INT, mesh_senw_ind, mesh_edge_elements_ind)
define_varp(mesh_edge_dg_label, INT, mesh_senw_ind, mesh_edge_elements_ind)
define_varp(mesh_edge_hv, INT, mesh_senw_ind)
define_varp(mesh_curve_num, INT, mesh_elements_ind)
define_varp(mesh_scratch, INT, mesh_elements_ind)

define_varp(poly_int_props, INT, poly_int_ind, g2_poly_ind)
define_varp(poly_real_props, FLOAT, poly_real_ind, g2_poly_ind)

implicit none f77
gi_common
zn_common
g2_common
sc_common
ma_common
implicit none f90

character*FILELEN filename    // Input /* Local variables, by section:
integer diskin2, length, p, b, e, open_stat, zone, fileid,    /* Main section */
    prep_done, i_inc_g2, i_prop, nxpt
integer current_int_props_1:poly_int_max
real rtest
real current_real_props_1:poly_real_max
character*LINELEN line, keyword
character*FILELEN tmpfilename

integer ielement, section, i, inode,    /* Process DG File */
    miss_elem, start, end , iwall, iseg, ielement2, skip_elem, poly_elem, num_h_elems,
    num_mesh_elems, mesh_elem, exp_inc
real temp_x, temp_z, mult

integer mesh_sense, ix, iz, n    // Mesh Polygons

vc_decl(yhat)
vc_decl(test_vec_1)
vc_decl(test_vec_2)
vc_decl(test_vec_3)
vc_decl(center)

real x_min, x_max, z_min, z_max, xb_min, xb_max,    /* Min., Max. */

```

```

    yb_min, yb_max, zb_min, zb_max, vol
vc_decl(min_corner)
vc_decl(max_corner)

integer nxd_0, nzd_0, ix_min, ix_max, iz_min, iz_max,      /* Sonnet mesh */
    nx_nz_max, mesh_tot_elements, iedge, itip, new_node, mesh_tot_nodes
integer mesh_edge_num_elements mesh_num_edges, mesh_ix_start mesh_num_edges,
    mesh_ix_end mesh_num_edges, mesh_iz_start mesh_num_edges,
    mesh_iz_end mesh_num_edges, mesh_ix_step mesh_num_edges, mesh_iz_step mesh_num_edges,
    mesh_corner mesh_num_edges, element_start:element_end
real test_node g2_x:g2_z

integer nunit      // Print Walls
character*LINLEN file_format
character*FILELEN walloutfile

integer num_new_walls, this_node      // Wall File
real x_wall, z_wall
character*FILELEN wallinfile

integer symmetry      // Symmetry

integer solid_faces0:1, solid_zone0:0,      /* Y resolution */
    end_faces0:1
character*LINLEN one_zone_type0:0
real coeff_NCOEFFS, poly4 0:4, g2_x:g2_z
real y_border, cos_y, sin_y
real solid_ys0:1

vc_decl(a_y)
vc_decl(b_y)

integer process_polygon, ntriangles, refine, j, k,      /* Zone, Polygon */
    temp_num_stratum_pts, temp_aux_stratum, temp_aux_segment, i_aux, nt, i_2, i_2_range, i_2_loop
integer temp_int_props 1:poly_int_max, temp_segment 0:max_triangles-1, 0:3,
    temp_stratum_pts 0:g2_num_points-1
real temp_polygon 0:g2_num_points-1, g2_x:g2_z, temp_triangles 0:max_triangles-1, 0:3, g2_x:g2_z,
    temp_real_props 1:poly_real_max, temp_holes 0:0, g2_x:g2_z
character*LINLEN new_zone_type, new_zone_type_2

integer write_poly_nc      // polygon.nc file
character*FILELEN polygon_nc_file

integer face1, num_zone1, num_zone2, k_zone1,      /* Mixed zone sectors */
    k_zone2, solid_zone_p, solid_face, other_zone, other_face, this_poly, i_poly, this_seg, i_sect,
    solid_sector, other_sector, other_type, y_max, this_stratum, this_mat, new_solid_sector,
    new_other_sector, other_stratum, other_seg
real this_temp, this_rc

integer i_diag, i_grp, is_aux_sector      // Diagnostic init
integer grp_sectors g2_num_points

integer y_mat0:1, y_stratum0:1, y_p_stratum0:1, end_zones0:1

integer num_nodes, num_elements, num_walls,      /* Local pointers */
    dim_nodes, dim_elements, dim_walls, nxd, nzd, num_y, y_div, dim_y, dim_ym, num_aux_sectors,
    dim_aux_sectors, num_diags, dim_diags, num_dg_poly, dim_dg_poly

```

```

declare_varp(nodes)
declare_varp(node_type)
declare_varp(node_element_count)

declare_varp(dg_elements_list)
declare_varp(element_missing)
declare_varp(element_skipped)
declare_varp(element_assigned)

declare_varp(dg_polygons)
declare_varp(dg_poly_num_elements)
declare_varp(dg_poly_num_meshcon)
declare_varp(dg_poly_meshcon)
declare_varp(dg_poly_meshcon_hv)

declare_varp(wall_nodes)
declare_varp(wall_elements)
declare_varp(wall_segment_count)

declare_varp(y_values)
declare_varp(facearray)
declare_varp(zonearray)
declare_varp(zone_type_array)
declare_varp(sect_zone1)
declare_varp(sect_zone2)

declare_varp(aux_stratum)
declare_varp(aux_stratum_poly)
declare_varp(aux_stratum_points)
declare_varp(aux_stratum_segment)

declare_varp(diag_name)
declare_varp(diag_stratum)
declare_varp(diag_solid)
declare_varp(diag_variable)
declare_varp(diag_tab_index)
declare_varp(diag_var_min)
declare_varp(diag_var_max)
declare_varp(diag_mult)
declare_varp(diag_spacing)

declare_varp(mesh_xz)
declare_varp(mesh_nodes)
declare_varp(mesh_elements)
declare_varp(mesh_edge_elements)
declare_varp(mesh_edge_dg_label)
declare_varp(mesh_edge_hv)
declare_varp(mesh_curve_num)
declare_varp(mesh_scratch)

declare_varp(poly_int_props)
declare_varp(poly_real_props)

⟨ Memory allocation interface 0 ⟩
st_decls
vc_decls
nc_decls

```



```

g2.ncdecl
gi_ext    /* PREPARATION STAGE */
open(unit = diskin, file = filename, status = 'old', form = 'formatted')
diskin2 = diskin + 1

⟨ Allocations Initializations 9.1 ⟩

loop1: continue

  assert(read_string(diskin, line, length))
  assert(length ≤ len(line))
  length = parse_string(line( : length))
  p = 0
  assert(next_token(line, b, e, p))
  keyword = line(b : e)

  if (keyword ≡ 'dg_file') then
    ⟨ Process DG File 10 ⟩

  else if (keyword ≡ 'sonnet_mesh' ∨ keyword ≡ 'uedge_mesh') then
    assert(next_token(line, b, e, p))
    tmpfilename = line(b : e)    // Sun F90 chokes if don't do this
    open_file(diskin2, tmpfilename)    /* E.g., sonnet_mesh filename 120 24 */
    if (keyword ≡ 'sonnet_mesh') then
      assert(next_token(line, b, e, p))
      nxd_0 = read_integer(line(b : e))
      assert(next_token(line, b, e, p))
      nzd_0 = read_integer(line(b : e))
      if (next_token(line, b, e, p)) then
        ix_min = read_integer(line(b : e))
        assert(next_token(line, b, e, p))
        ix_max = read_integer(line(b : e))
        assert(next_token(line, b, e, p))
        iz_min = read_integer(line(b : e))
        assert(next_token(line, b, e, p))
        iz_max = read_integer(line(b : e))
        nxd = ix_max - ix_min + 1
        nzd = iz_max - iz_min + 1
      else
        nxd = nxd_0
        nzd = nzd_0
        ix_min = 1
        ix_max = nxd
        iz_min = 1
        iz_max = nzd
      end if
    else if (keyword ≡ 'uedge_mesh') then
      read (diskin2, *)    // First line is a comment
      read (diskin2, *) nxd, nzd, nxpt
    end if
    var_alloc(mesh_xz)
    var_alloc(mesh_nodes)
    var_alloc(mesh_elements)
    nx_nz_max = max(nxd, nzd)
    var_alloc(mesh_edge_elements)

```

```

var_alloc(mesh_edge_dg_label)
var_alloc(mesh_edge_hv)
var_alloc(mesh_curve_num)
var_alloc(mesh_scratch)

if (keyword ≡ 'sonnet_mesh') then
  call read_sonnet_mesh(diskin2, nxd_0, nzd_0, ix_min, ix_max, iz_min, iz_max, nxd, nzd,
    mesh_xz)
else if (keyword ≡ 'uedge_mesh') then
  call read_uedge_mesh(diskin2, nxd, nzd, nxpt, mesh_xz)
  nxd_0 = nxd
  nzd_0 = nzd
  ix_min = 1
  ix_max = nxd
  iz_min = 1
  iz_max = nzd
end if

else if (keyword ≡ 'wallfile') then
  ⟨ Read Wall File 11 ⟩

else if (keyword ≡ 'print_min_max') then
  call find_min_max(num_nodes, nodes, node_type, x_min, x_max, z_min, z_max)
  write(stdout, *) 'X_range=', x_min, ' -> ', x_max
  write(stdout, *) 'Z_range=', z_min, ' -> ', z_max

else if (keyword ≡ 'bounds') then
  ⟨ Set Bounds 9.2 ⟩

else if (keyword ≡ 'symmetry') then
  ⟨ Set Symmetry 9.3 ⟩

else if (keyword ≡ 'print_walls') then
  assert(next_token(line, b, e, p))
  file_format = line(b : e)
  if (next_token(line, b, e, p)) then
    walloutfile = line(b : e)
    nunit = diskout
  else
    walloutfile = char_undef
    nunit = stdout
  end if
  call print_walls(nunit, file_format, walloutfile, num_walls, wall_segment_count, wall_nodes, nodes)

else if (keyword ≡ 'uniform_ys') then
  if ((symmetry ≡ geometry_symmetry_plane_hw) ∨ (symmetry ≡ geometry_symmetry_cyl_section))
    then
      assert(yb_max - yb_min > zero)
      y_border = const(1., -2) * (yb_max - yb_min)
    else if (symmetry ≡ geometry_symmetry_cyl_hw) then
      y_border = zero
      assert(yb_max - yb_min ≡ two * PI)
    else if ((symmetry ≡ geometry_symmetry_oned) ∨ (symmetry ≡
      geometry_symmetry_plane) ∨ (symmetry ≡ geometry_symmetry_cylindrical))
      then
        assert('Cannot specify y values with this symmetry' ≡ '')

```

```

else if (symmetry  $\equiv$  geometry_symmetry_none) then
    assert('Need to specify symmetry before y values'  $\equiv$  '')
end if
assert(next_token(line, b, e, p))
num_y = read_integer(line(b:e))
assert(num_y > 0)
y_div = num_y - 1
dim_ym = max(dim_ym, y_div)
var_realloca(y_values)
do i = 0, y_div
    /* This is intended to break up the existing range in y into the desired number of segments.
       For cases in which we will define solid end pieces, have set y_border to a non-zero value. */
    y_values_i = (yb_min + y_border) + ((yb_max - y_border) - (yb_min + y_border)) * areal(i) / areal(y_div)
end do

else if (keyword  $\equiv$  'y_values') then
    mult = PI / const(1.8, 2)
    assert(next_token(line, b, e, p))
    rtest = read_real_soft_fail(line(b:e))
    if (rtest  $\equiv$  real_undef) then /* If the list of y values needs to be on separate lines, the argument
        should be a file name. That file will have one value per line. Assume these are input in
        degrees and converted here to radians. */
        tmpfilename = line(b:e)
        open_file(diskin2, tmpfilename)
        y_div = -1
    y_loop: continue
    if (read_string(diskin2, line, length)) then
        assert(length  $\leq$  len(line))
        length = parse_string(line(: length))
        p = 0
        assert(next_token(line, b, e, p))
        increment_y_div
        y_values_y_div = read_real(line(b:e)) * mult
        if (y_div > 0) then
            assert(y_values_y_div > y_values_y_div-1)
        end if
        assert(y_values_y_div  $\geq$  yb_min)
        assert(y_values_y_div  $\leq$  yb_max)
        goto y_loop
    end if
else /* Else, all y values have to be on a single line in the main input file. */
    y_div = 0
    y_values_0 = read_real(line(b:e)) * mult
y_loop2: continue
    if (next_token(line, b, e, p)) then
        increment_y_div
        y_values_y_div = read_real(line(b:e)) * mult
        if (y_div > 0) then
            assert(y_values_y_div > y_values_y_div-1)
        end if
        goto y_loop2
    end if
end if
end if

```

```

else if (keyword ≡ 'end_prep') then
  ⟨ End Prep 9.4 ⟩
else if (keyword ≡ 'new_zone') then
  zone++
  assert(next_token(line, b, e, p))
  new_zone_type = line(b : e)    /* Allow for mixed zone types when the geometry is resolved in the
    third dimension. The format is to read in a second zone type, new_zone_type_2, then a list of
    y values. The first zone type is understood to be the default. The list of y values (actually,
    integers between 0 and y_div-1) indicate the divisions having the second zone type. */
  if (next_token(line, b, e, p)) then
    new_zone_type_2 = line(b : e)
    assert((symmetry ≡ geometry_symmetry_plane_hw) ∨ (symmetry ≡
      geometry_symmetry_cyl_hw) ∨ (symmetry ≡ geometry_symmetry_cyl_section))
    do i = 0, y_div - 1
      zone_type_array_i = new_zone_type
    end do
    new_zone_type = "mixed"
mixed_type_loop: continue
  if (next_token(line, b, e, p)) then    /* Will this work? May want to try using elsewhere. */
    if (line(b : e) ≠ '-') then
      i_2 = read_integer(line(b : e))
      assert((i_2 ≥ 0) ∧ (i_2 ≤ y_div - 1))
      zone_type_array_i_2 = new_zone_type_2
    else
      assert(next_token(line, b, e, p))
      i_2_range = read_integer(line(b : e))
      assert((i_2_range ≥ 0) ∧ (i_2_range ≤ y_div - 1))
      assert(i_2_range > i_2)
      do i_2_loop = i_2 + 1, i_2_range
        zone_type_array_i_2_loop = new_zone_type_2
      end do
    end if
    goto mixed_type_loop
  end if
end if

else if (keyword ≡ 'new_polygon') then
  ⟨ New Polygon 14 ⟩
else if (keyword ≡ 'new_diagnostic') then
  assert(next_token(line, b, e, p))
  increment_num_diags
  diag_name_num_diags = line(b :)
  call specify_diagnostic(diskin, diag_stratum_num_diags, diag_solid_num_diags, diag_variable_num_diags,
    diag_tab_index_num_diags, diag_var_min_num_diags, diag_var_max_num_diags, diag_mult_num_diags,
    diag_spacing_num_diags)
else if (keyword ≡ 'y_min_zone') then
  assert((symmetry ≡ geometry_symmetry_plane_hw) ∨ (symmetry ≡ geometry_symmetry_cyl_section))
  assert(next_token(line, b, e, p))
  y_mat_0 = ma_lookup(line(b : e))
  assert(ma_check(y_mat_0))
  assert(next_token(line, b, e, p))
  y_stratum_0 = read_integer(line(b : e))

```

```

    if (next_token(line, b, e, p)) then
        y-p-stratum0 = read_integer(line(b : e))
    end if

    else if (keyword ≡ 'y_max_zone') then
        assert((symmetry ≡ geometry_symmetry_plane_hw) ∨ (symmetry ≡ geometry_symmetry_cyl_section))
        assert(next_token(line, b, e, p))
        y-mat1 = ma_lookup(line(b : e))
        assert(ma_check(y-mat1))
        assert(next_token(line, b, e, p))
        y-stratum1 = read_integer(line(b : e))
        if (next_token(line, b, e, p)) then
            y-p-stratum1 = read_integer(line(b : e))
        end if

    else if (keyword ≡ 'polygon_nc_file') then
        assert(next_token(line, b, e, p))
        if (line(b : e) ≡ 'none' ∨ line(b : e) ≡ 'NONE') then
            write_poly_nc = FALSE
        else
            polygon_nc_file = line(b : e)
        end if

    else if (keyword ≡ 'quit' ∨ keyword ≡ 'end') then
        go to eof
    end if
    go to loop1

eof: continue
close(unit = diskin)

var_reallocb(g2_polygon_xz)
var_reallocb(g2_polygon_segment)
var_reallocb(g2_polygon_points)
var_reallocb(g2_polygon_zone)
var_reallocb(g2_polygon_stratum)
var_reallocb(poly_int_props)
var_reallocb(poly_real_props)
var_reallocb(aux_stratum)
var_reallocb(aux_stratum_poly)
var_reallocb(aux_stratum_points)
var_reallocb(aux_stratum_segment)
var_reallocb(diag_name)
var_reallocb(diag_stratum)
var_reallocb(diag_solid)
var_reallocb(diag_variable)
var_reallocb(diag_tab_index)
var_reallocb(diag_var_min)
var_reallocb(diag_var_max)
var_reallocb(diag_mult)
var_reallocb(diag_spacing)

var_alloc(sect_zone1)
var_alloc(sect_zone2)

if (write_poly_nc ≡ TRUE) then

```

```

    fileid = nccreate(trim(polygon_nc_file), NC_CLOBBER, nc_stat)
    g2_ncdef(fileid)
    call ncendef(fileid, nc_stat)
    g2_ncwrite(fileid)
    call ncclose(fileid, nc_stat)
end if

if (keyword == 'end') then
    < Setup End Zones 15 >

    call boundaries_neighbors

    call setup_sectors(g2_num_polygons, g2_polygon_points, g2_polygon_xz, g2_polygon_segment,
        g2_polygon_zone, poly_int_props, poly_real_props, num_aux_sectors, aux_stratum,
        aux_stratum_poly, aux_stratum_points, aux_stratum_segment, y_div, sect_zone1, sect_zone2)

    < Setup Toroidally Facing Sectors 16 >

    call default_diag_setup

    < Setup Sector Diagnostics 17 >

    call end_sectors

    call detector_setup

    call end_detectors

    call check_geometry

    call pixel_map_test

    call write_geometry

    call erase_geometry
end if

var_free(nodes)
var_free(node_type)
var_free(node_element_count)
var_free(dg_elements_list)
var_free(element_missing)
var_free(element_skipped)
var_free(element_assigned)
var_free(wall_nodes)
var_free(wall_elements)
var_free(wall_segment_count)
var_free(dg_polygons)
var_free(dg_poly_num_elements)
var_free(dg_poly_num_meshcon)
var_free(dg_poly_meshcon)
var_free(dg_poly_meshcon_hv)
var_free(y_values)
var_free(facearray)
var_free(zonearray)
var_free(zone_type_array)
var_free(sect_zone1)
var_free(sect_zone2)
var_free(aux_stratum)
var_free(aux_stratum_poly)

```

```
var_free(aux_stratum_points)
var_free(aux_stratum_segment)
var_free(diag_name)
var_free(diag_stratum)
var_free(diag_solid)
var_free(diag_variable)
var_free(diag_tab_index)
var_free(diag_var_min)
var_free(diag_var_max)
var_free(diag_mult)
var_free(diag_spacing)
var_free(mesh_xz)
var_free(mesh_nodes)
var_free(mesh_elements)
var_free(mesh_edge_elements)
var_free(mesh_edge_dg_label)
var_free(mesh_edge_hv)
var_free(mesh_curve_num)
var_free(mesh_scratch)
var_free(poly_int_props)
var_free(poly_real_props)

return end
```

Allocations and initializations. Apparently, this code is getting too large for FWEAVE. Splitting these lines off into a separate subsection allowed it to run successfully.

⟨ Allocations Initializations 9.1 ⟩ ≡

```

zone = 0
g2_num_polygons = 0
symmetry = geometry_symmetry_none
xb_min = zero
xb_max = zero
yb_min = zero
yb_max = zero
zb_min = zero
zb_max = zero
nxd = 0
nzd = 0
num_walls = 0
num_nodes = 0
num_elements = 0
num_dg_poly = 0
num_aux_sectors = 0
num_diags = 0    /* To permit these to be reallocated by any of the three possible macros, helps to
    make the initial allocation of size mem_inc. */
dim_walls = mem_inc
dim_nodes = mem_inc
dim_elements = mem_inc
dim_aux_sectors = mem_inc
dim_diags = mem_inc
dim_dg_poly = mem_inc

var_alloc(nodes)
var_alloc(node_type)
var_alloc(node_element_count)

var_alloc(dg_elements_list)
var_alloc(element_missing)
var_alloc(element_skipped)
var_alloc(element_assigned)

var_alloc(dg_polygons)
var_alloc(dg_poly_num_elements)
var_alloc(dg_poly_num_meshcon)
var_alloc(dg_poly_meshcon)
var_alloc(dg_poly_meshcon_hv)

var_alloc(wall_nodes)
var_alloc(wall_elements)
var_alloc(wall_segment_count)    /* Default in purely 2-D case. */
y_div = 1
@#if 0
    dim_y = 2 * mem_inc
@#else
    dim_y = mem_inc
    dim_ym = mem_inc - 1
@#endif
    var_alloc(y_values)

```



```

var_alloc(facearray)
var_alloc(zonearray)
var_alloc(zone_type_array)
y_values_0 = real_unused
y_values_1 = real_unused

y_mat_0 = int_unused
y_mat_1 = int_unused
y_stratum_0 = int_unused
y_stratum_1 = int_unused
y_p_stratum_0 = int_unused
y_p_stratum_1 = int_unused
end_zones_0 = int_unused
end_zones_1 = int_unused

var_alloc(aux_stratum)
var_alloc(aux_stratum_poly)
var_alloc(aux_stratum_points)
var_alloc(aux_stratum_segment)

var_alloc(diag_name)
var_alloc(diag_stratum)
var_alloc(diag_solid)
var_alloc(diag_variable)
var_alloc(diag_tab_index)
var_alloc(diag_var_min)
var_alloc(diag_var_max)
var_alloc(diag_mult)
var_alloc(diag_spacing)

prep_done = FALSE
current_int_props_poly_stratum = int_undef
current_int_props_poly_material = 0
current_int_props_poly_num_holes = 0
current_real_props_poly_temperature = const(3., 2)
current_real_props_poly_recyc_coef = one
current_real_props_poly_min_area = const(1., -3)
current_real_props_poly_hole_x = real_undef
current_real_props_poly_hole_z = real_undef
write_poly_nc = TRUE
polygon_nc_file = 'polygon.nc'

```

This code is used in section 9.

Set dimensions of universal cell. For cylindrical cases, the *y**b* values are input in degrees and converted to radians at the end of the preparation section. Also, assumes that symmetry has already been set. DOCUMENT!!!

```

⟨Set Bounds 9.2⟩ ≡
  assert(next_token(line, b, e, p))
  xb_min = read_real(line(b : e))
  assert(next_token(line, b, e, p))
  xb_max = read_real(line(b : e))
  assert(next_token(line, b, e, p))
  zb_min = read_real(line(b : e))
  assert(next_token(line, b, e, p))
  zb_max = read_real(line(b : e))
  if (next_token(line, b, e, p)) then
    yb_min = read_real(line(b : e))
    assert(next_token(line, b, e, p))
    yb_max = read_real(line(b : e))
  else
    yb_min = zero
    yb_max = one
    if ((symmetry ≡ geometry_symmetry_cylindrical) ∨ (symmetry ≡ geometry_symmetry_cyl_hw))
      yb_max = const(3.6, 2)
    end if
    if ((symmetry ≡ geometry_symmetry_cylindrical) ∨ (symmetry ≡
      geometry_symmetry_cyl_hw) ∨ (symmetry ≡ geometry_symmetry_cyl_section))
      then
        yb_min *= PI / const(1.8, 2)
        yb_max *= PI / const(1.8, 2)
      else
        assert((symmetry ≡ geometry_symmetry_plane) ∨ (symmetry ≡
          geometry_symmetry_plane_hw) ∨ (symmetry ≡ geometry_symmetry_oned))
      end if
    end if
  end if

```

This code is used in section 9.

Set symmetry parameter.

```

⟨ Set Symmetry 9.3 ⟩ ≡
  assert(next_token(line, b, e, p))
  if (line(b : e) ≡ 'cylindrical') then
    symmetry = geometry_symmetry_cylindrical
  else if (line(b : e) ≡ 'plane') then
    symmetry = geometry_symmetry_plane
  else if (line(b : e) ≡ 'oned') then
    symmetry = geometry_symmetry_oned
  else if (line(b : e) ≡ 'plane_hw') then
    symmetry = geometry_symmetry_plane_hw
  else if (line(b : e) ≡ 'cylindrical_hw') then
    symmetry = geometry_symmetry_cyl_hw
  else if (line(b : e) ≡ 'cylindrical_section') then
    symmetry = geometry_symmetry_cyl_section
  else
    assert('Unexpected_symmetry_specification' ≡ '␣')
  end if

```

This code is used in section 9.

End preparation stage.

⟨ End Prep 9.4 ⟩ =

```

dim_elements = num_elements
var_reallocb(dg_elements_list)
var_reallocb(element_missing)
var_reallocb(element_skipped)
var_reallocb(element_assigned)

dim_nodes = num_nodes
var_reallocb(nodes)
var_reallocb(node_type)
var_reallocb(node_element_count)  /* Allow the "outer" points to be used in the DG polygons. To
    ensure consistency of the surfaces generated from them with the universal cell surfaces, search the
    node list for points close to the "outer" ones, resetting them as needed. */
do i = 1, num_nodes
  if (abs(nodesi,g2_x - xb_min) < geom_epsilon) then
    if (abs(nodesi,g2_z - zb_min) < geom_epsilon) then
      nodesi,g2_x = xb_min
      nodesi,g2_z = zb_min
    else if (abs(nodesi,g2_z - zb_max) < geom_epsilon) then
      nodesi,g2_x = xb_min
      nodesi,g2_z = zb_max
    end if
  else if (abs(nodesi,g2_x - xb_max) < geom_epsilon) then
    if (abs(nodesi,g2_z - zb_min) < geom_epsilon) then
      nodesi,g2_x = xb_max
      nodesi,g2_z = zb_min
    else if (abs(nodesi,g2_z - zb_max) < geom_epsilon) then
      nodesi,g2_x = xb_max
      nodesi,g2_z = zb_max
    end if
  end if
end do

dim_walls = num_walls
var_reallocb(wall_nodes)
var_reallocb(wall_elements)
var_reallocb(wall_segment_count)

dim_dg_poly = num_dg_poly
var_reallocb(dg_polygons)
var_reallocb(dg_poly_num_elements)
var_reallocb(dg_poly_num_meshcon)
var_reallocb(dg_poly_meshcon)
var_reallocb(dg_poly_meshcon_hv)

dim_y = y_div
dim_ym = y_div
var_reallocb(y_values)
var_reallocb(facearray)
var_reallocb(zonearray)
var_reallocb(zone_type_array)

call init_geometry  /* yb_min and yb_max are now set with the bounds keyword. */

```

```

if (symmetry  $\equiv$  geometry-symmetry-cyl-section) then
    assert(yb_max - yb_min < two * PI)
end if
vc_set(min_corner, xb_min, yb_min, zb_min)
vc_set(max_corner, xb_max, yb_max, zb_max)
    /* Have generalized this for the various 3-D cases, following the planar case in boxgen. */
call universal_cell_3d(symmetry, min_corner, max_corner, vol)

solid_ys0 = y_values0
solid_ys1 = y_valuesy_div
solid_faces0 = int_unused
solid_faces1 = int_unused
if ((symmetry  $\equiv$  geometry-symmetry-cylindrical)  $\vee$  (symmetry  $\equiv$  geometry-symmetry-plane)) then
    facearray0 = int_unused    // Default case
    facearray1 = int_unused
else
    do i = 0, y_div
        if (symmetry  $\equiv$  geometry-symmetry-plane_hw) then
            vc_set(a_y, zero, one, zero)
            vc_set(b_y, xb_max, y_valuesi, zb_max)
        else
            assert((symmetry  $\equiv$  geometry-symmetry-cyl_hw)  $\vee$  (symmetry  $\equiv$  geometry-symmetry-cyl-section))
            /* Ensure beginning and end faces match. */
            if ((symmetry  $\equiv$  geometry-symmetry-cyl_hw)  $\wedge$  (i  $\equiv$  y_div)) then
                cos_y = cos(y_values0)
                sin_y = sin(y_values0)
            else
                cos_y = cos(y_valuesi)    // General case
                sin_y = sin(y_valuesi)
            end if
            vc_set(a_y, -sin_y, cos_y, zero)
            vc_set(b_y, xb_max * cos_y, xb_max * sin_y, zb_max)
        end if
        call plane(b_y, a_y, coeff)
        facearrayi = define_surface(coeff, T)
    end do    /* Don't think this will be enough. */
    if ((symmetry  $\equiv$  geometry-symmetry-plane_hw)  $\vee$  (symmetry  $\equiv$  geometry-symmetry-cyl-section))
        then
            solid_faces0 = facearray0
            solid_faces1 = facearrayy_div
        end if
    end if

if (nxd * nzd > 0) then
    <Set Mesh Polygons 12>
    <Set Mesh Elements 13>
else    /* Allocate something since it appears in argument lists later. */
    nxd = 1
    nzd = 1
    var_alloc(mesh_xz)
    var_alloc(mesh_nodes)
    var_alloc(mesh_elements)
    nx_nz_max = 1

```

```
    var_alloc(mesh_edge_elements)  
    var_alloc(mesh_edge_dg_label)  
    var_alloc(mesh_edge_hv)  
    var_alloc(mesh_curve_num)  
    var_alloc(mesh_scratch)  
end if  
prep_done = TRUE
```

This code is used in section 9.

10 Read and process DG file

```

⟨Process DG File 10⟩ ≡
  assert(next_token(line, b, e, p))
  tmpfilename = line(b : e)
  open_file(diskin2, tmpfilename)    /* Read DG's .dgo file */
  ielement = 0
  section = sec_undef    /* Assume that if the file has a ".dgo" that it came from DG and needs to be
                           converted from mm to meters. Assume that it is otherwise in meters. Developed read_real_scaled
                           expressly for this task since direct multiplication introduces round-off error that can cause these
                           coordinates to differ from the same ones read in from other files. */
  if (index(line(b : e), '.dgo') > 0) then
    exp_inc = -3
  else
    exp_inc = 0
  end if
dg_loop: continue
  if (read_string(diskin2, line, length)) then
    assert(length ≤ len(line))
    /* We do not want to have to manually change the format of the files DG writes. The node
       coordinates are currently comma-delimited (with possible additional spaces). Replace the
       commas in the current line with spaces so we can use our usual string utilities. */
    do i = 1, length
      if (line(i : i) ≡ ',')
        line(i : i) = ' '
      end do
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))
    if (line(b : e) ≡ 'p1') then
      section = sec_p1
    else if (line(b : e) ≡ 'p2') then
      section = sec_p2
    else if (line(b : e) ≡ 'misselem') then
      section = sec_miss
    else if (line(b : e) ≡ 'skipelem') then
      section = sec_skip
    else if (line(b : e) ≡ 'polygon') then
      assert(next_token(line, b, e, p))
      increment_num_dg_poly
      assert(num_dg_poly ≡ read_integer(line(b : e)))
      dg_poly_num_elements_num_dg_poly = 0
      dg_poly_num_meshcon_num_dg_poly = 0
      do i = 1, meshcon_max
        do j = 1, meshcon_elem_max
          dg_poly_meshcon_num_dg_poly,i,j = int_unused
          dg_poly_meshcon_hv_num_dg_poly,i,j = int_unused
        end do
      end do
      section = sec_dg_poly
    else if (line(b : e) ≡ 'wall') then

```

```

    assert(section ≡ sec_dg_poly)
    section = sec_dg_wall
    /* Note that we are assuming we know the order of the polygon variables. Since we have
       explicitly defined them in DG, this should be OK. The “polymat” entry is a single line, so
       does not need a separate section and would have to process here. Not using it yet, though. */
else if (line(b : e) ≡ 'polymat') then
    assert(section ≡ sec_dg_wall)
else if (line(b : e) ≡ 'meshcon1') then
    assert(section ≡ sec_dg_wall)
    section = sec_dg_meshcon1
else if (line(b : e) ≡ 'meshcon2') then
    assert(section ≡ sec_dg_meshcon1)
    section = sec_dg_meshcon2
    /* ADD SECTION TO READ jedgi1, jedgo2, jedgo1, jedgi2 SECTIONS AND PROCESS */
else if (line(b : e) ≡ 'finish') then
    section = sec_done
else
    if (section ≡ sec_p1 ∨ section ≡ sec_p2) then
        temp_x = read_real_scaled(line(b : e), exp_inc)
        if (temp_x ≠ real_undef) then // Failure
            assert(next_token(line, b, e, p))
            temp_z = read_real_scaled(line(b : e), exp_inc)
        end if
        if (temp_x ≡ real_undef ∨ temp_z ≡ real_undef) then
            section = sec_undef
            go to dg-loop // Can trash this line since it did not match anything we've planned for
        end if
    @#if 0
        // Replaced this with the use of read_real_scaled .
        temp_x *= mult
        temp_z *= mult
    @#endif
    inode = 0
    if (num_nodes > 0) then
        do i = 1, num_nodes
            if (nodesi,g2_x ≡ temp_x) then
                if (nodesi,g2_z ≡ temp_z) then
                    assert(inode ≡ 0)
                    inode = i // is a duplicate
                end if
            end if
        end do
    end if
    if (inode ≡ 0) then
        increment_num_nodes // is a new node
        inode = num_nodes
        nodesnum_nodes,g2_x = temp_x
        nodesnum_nodes,g2_z = temp_z
    end if
    if (section ≡ sec_p1) then
        increment_num_elements
        dg_elements_listnum_elements,element_start = inode
    end if

```



```

    element_missingnum_elements = FALSE    // will be set later
    element_skippednum_elements = FALSE    // ditto
    element_assignednum_elements = FALSE
else if (section  $\equiv$  sec_p2) then
    ielement++
    dg_elements_listielement,element_end = inode
end if    /* Note that “missing” elements are those that should be completely ignored, e.g.,
           elements that were deleted, but still appear in the DG file. “Skipped” elements, on
           the other hand, are intended to be used in setting up DG-defined polygons, but not in
           characterizing nodes or setting up walls. */
else if (section  $\equiv$  sec_miss) then
    assert(ielement  $\equiv$  num_elements)    // Compare p1 and p2 reads
    miss_elem = read_int_soft_fail(line(b : e))
    if (miss_elem > 0  $\wedge$  miss_elem  $\leq$  num_elements) then
        element_missingmiss_elem = TRUE
    else
        assert(miss_elem  $\equiv$  int_undef)    // Assume we’ve stumbled into another section
        section = sec_undef
        go to dg_loop
    end if
else if (section  $\equiv$  sec_skip) then
    assert(ielement  $\equiv$  num_elements)    // Compare p1 and p2 reads
    skip_elem = read_int_soft_fail(line(b : e))
    if (skip_elem > 0  $\wedge$  skip_elem  $\leq$  num_elements) then
        element_skippedskip_elem = TRUE
    else
        assert(skip_elem  $\equiv$  int_undef)    // Assume we’ve stumbled into another section
        section = sec_undef
        go to dg_loop
    end if
else if (section  $\equiv$  sec_dg_wall) then
    poly_elem = read_int_soft_fail(line(b : e))
    assert((poly_elem > 0)  $\wedge$  (poly_elem  $\leq$  num_elements))
    assert(num_dg_poly > 0)
    dg_poly_num_elementsnum_dg_poly++
    dg_polygonsnum_dg_poly,dg_poly_num_elementsnum_dg_poly = poly_elem
else if ((section  $\equiv$  sec_dg_meshcon1)  $\vee$  (section  $\equiv$  sec_dg_meshcon2)) then
    num_h_elems = read_int_soft_fail(line(b : e))
    assert((num_h_elems  $\geq$  0)  $\wedge$  (num_h_elems  $\leq$  2))
    num_mesh_elems = 0    /* Processing of these sections differs from the others in that we have a
                           count (first line) and then a list of elements, which is at least “count” long. Because of this,
                           we need to process the list here. Have set this up so that either one of the two “meshcon”
                           variables can be used to specify a single point. */
meshcon_loop: continue
    assert(read_string(diskin2, line, length))
    assert(length  $\leq$  len(line))
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))
    mesh_elem = read_int_soft_fail(line(b : e))
    if (mesh_elem  $\neq$  int_undef) then
        if (num_mesh_elems  $\equiv$  0) then

```

```

        dg_poly_num_meshconnum_dg_poly++
        assert(dg_poly_num_meshconnum_dg_poly ≤ meshcon_max)
    end if
    num_mesh_elems++
    assert(num_mesh_elems ≤ meshcon_elem_max)
    dg_poly_meshconnum_dg_poly, dg_poly_num_meshconnum_dg_poly, num_mesh_elems = mesh_elem
    if (num_mesh_elems ≤ num_h_elems) then
        dg_poly_meshcon_hvnum_dg_poly, dg_poly_num_meshconnum_dg_poly, num_mesh_elems = mesh_h
    else
        dg_poly_meshcon_hvnum_dg_poly, dg_poly_num_meshconnum_dg_poly, num_mesh_elems = mesh_v
    end if
    go to meshcon_loop
else /* The list of elements is finished. Use the backspace command here to allow the main
    loop to read this line again and handle accordingly. */
    backspace(diskin2)
    assert(num_mesh_elems ≥ num_h_elems)
    assert((num_mesh_elems ≡ 0) ∨ (num_mesh_elems ≡ meshcon_elem_max))
    /* This is the last piece of the polygon section. Reset section since we don't know what
       might be next. */
    if (section ≡ sec_dg_meshcon2)
        section = sec_undef
    go to dg_loop
end if
end if
end if
if (section ≠ sec_done)
    go to dg_loop
end if
close(unit = diskin2) /* Now have elements and nodes. Characterize all of the nodes. */
do inode = 1, num_nodes
    node_element_countinode = 0
    start = FALSE end = FALSE
    do ielement = 1, num_elements
        if ((element_missingielement ≡ FALSE) ∧ (element_skippedielement ≡ FALSE)) then
            if (dg_elements_listielement, element_start ≡ inode) then
                node_element_countinode++
                start = TRUE
            else if (dg_elements_listielement, element_end ≡ inode) then
                node_element_countinode++ end = TRUE
            end if
        end if
    end do
end do
if (node_element_countinode ≡ 0) then
    node_typeinode = node_no_elements
else if (node_element_countinode ≡ 1) then assert ( start ≡ TRUE ∨ end ≡ TRUE )
    node_typeinode = node_one_element
else if (node_element_countinode ≡ 2) then if ( start ≡ TRUE ∧ end ≡ TRUE ) then
    node_typeinode = node_regular
else
    node_typeinode = node_mixed_normals
end if
end if

```

```

    else if (node_element_countinode > 2) then
        node_typeinode = node_many_elements
    end if
end do    /* Set up the walls. First, start walls at all irregular nodes that start an element. */
do inode = 1, num_nodes
    if (node_typeinode ≠ node_regular ∧ node_typeinode ≠ node_no_elements) then
        do ielement = 1, num_elements
            if (dg_elements_listielement,element_start ≡ inode) then
                increment_num_walls
                wall_elementsnum_walls,1 = ielement
                wall_nodesnum_walls,0 = inode
                element_assignedielement = TRUE
            end if
        end do
    end if
end do    /* Next, follow each of those walls until they hit another irregular node. */
do iwall = 1, num_walls
    iseg = 1
wall_loop: continue
    inode = dg_elements_listwall_elementsiwall,iseg,element_end
    assert(inode > 0 ∧ inode ≤ num_nodes)
    wall_nodesiwall,iseg = inode
    if (node_typeinode ≡ node_regular) then
        iseg++
        assert(iseg ≤ g2_num_points)
        wall_elementsiwall,iseg = 0
        do ielement = 1, num_elements
            if ((dg_elements_listielement,element_start ≡ inode) ∧ (element_skippedielement ≡ FALSE)) then
                assert(wall_elementsiwall,iseg ≡ 0)
                wall_elementsiwall,iseg = ielement
                element_assignedielement = TRUE
                go to wall_loop
            end if
        end do
    else    // End the wall at the next irregular node.
        wall_segment_countiwall = iseg
    end if
end do    /* Finally, follow all other elements connected only by regular nodes (these should be closed
surfaces). */
do ielement = 1, num_elements
    if ((element_missingielement ≡ FALSE) ∧ (element_skippedielement ≡ FALSE) ∧ (element_assignedielement ≡
FALSE)) then
        increment_num_walls    // An unassigned element...
        wall_elementsnum_walls,1 = ielement
        wall_nodesnum_walls,0 = dg_elements_listielement,element_start
        element_assignedielement = TRUE
        iseg = 1
wall_loop2: continue
        inode = dg_elements_listwall_elementsnum_walls,iseg,element_end
        assert(inode > 0 ∧ inode ≤ num_nodes)
        wall_nodesnum_walls,iseg = inode

```

```

    assert(node_typeinode  $\equiv$  node_regular)
    /* Search for adjacent element. Can start at ielement+1 because all previous elements have been
       assigned. If it has already been assigned, we're done with this wall. */
    do ielement2 = ielement + 1, num_elements
        if ((dg_elements_listielement2,element_start  $\equiv$  inode)  $\wedge$  (element_assignedielement2  $\equiv$  FALSE)) then
            iseg++
            assert(iseg  $\leq$  g2_num_points)
            wall_elementsnum_walls,iseg = ielement2
            element_assignedielement2 = TRUE
            go to wall_loop2
        end if
    end do
    wall_segment_countnum_walls = iseg     // End of this wall

    end if
end do

@if 0
    // Move to after end of prep. stage. OK?
    var_reallocb(wall_nodes)
    var_reallocb(wall_elements)
    var_reallocb(wall_segment_count)
@endif

```

This code is used in section 9.

11 Read wall file

This is intended to duplicate the wallfile functionality provided in *readgeometry*. At the same time, this format should allow a file generated with the “linear” format of this code’s *print_walls* keyword to be read back in. Hence, the numbers on the second line of the file represent the **number of points, not the number of segments**, as was the case in *readgeometry*. The points listed are checked against the existing list of nodes. If new, they are added. Either way, the new walls are stored as a list of nodes, just as are those generated from a DG file. Note that we presently make no attempt to update the elements. Only the node numbers and coordinates are used later in the code.

⟨ Read Wall File 11 ⟩ ≡

```

    assert(next_token(line, b, e, p))
    wallinfile = line(b : e)
    open(unit = diskin2, file = wallinfile, status = 'old', form = 'formatted')

    assert(read_string(diskin2, line, length))
    assert(length ≤ len(line))
    length = parse_string(line(: length))
    p = 0

    assert(next_token(line, b, e, p))
    num_new_walls = read_int_soft_fail(line(b : e))
    var_realloc(wall_nodes, dim_walls, num_walls + num_new_walls)
    var_realloc(wall_elements, dim_walls, num_walls + num_new_walls) // not used
    var_realloc(wall_segment_count, dim_walls, num_walls + num_new_walls)

    assert(read_string(diskin2, line, length))
    assert(length ≤ len(line))
    length = parse_string(line(: length))
    p = 0
    do iwall = 1, num_new_walls
        if (¬next_token(line, b, e, p)) then
            assert(read_string(diskin2, line, length))
            assert(length ≤ len(line))
            length = parse_string(line(: length))
            p = 0
            assert(next_token(line, b, e, p))
        end if
        wall_segment_count_{num_walls+iwall} = read_int_soft_fail(line(b : e))
    end do

    do iwall = num_walls + 1, num_walls + num_new_walls
        wall_segment_count_{iwall} = 0 // iseg starts at 0
        do iseg = 0, wall_segment_count_{iwall}
            assert(read_string(diskin2, line, length))
            assert(length ≤ len(line))
            length = parse_string(line(: length))
            p = 0
            assert(next_token(line, b, e, p))
            x_wall = read_real_soft_fail(line(b : e))
            assert(next_token(line, b, e, p))
            z_wall = read_real_soft_fail(line(b : e))

            if (num_nodes > 0) then

```

```

    do inode = 1, num_nodes
      if ((abs(x_wall - nodes_{inode,g2_x}) < geom_epsilon) ∧ (abs(z_wall - nodes_{inode,g2_z}) <
        geom_epsilon)) then
        this_node = inode
        go to wall_break
      end if
    end do
  end if
  increment_num_nodes
  nodes_{num_nodes,g2_x} = x_wall
  nodes_{num_nodes,g2_z} = z_wall
  this_node = num_nodes
wall_break: continue
  wall_nodes_{iwall,iseq} = this_node
end do
end do
num_walls += num_new_walls
dim_walls = max(dim_walls, num_walls)

close(unit = disk_in2)

```

This code is used in section 9.

12 Convert mesh data into polygons

⟨Set Mesh Polygons 12⟩ ≡

```

/* A separate question is the direction around the physical rectangle. The polygon sent to
   decompose_polygon must be traversed in a clockwise direction in physical space. */
vc_set(yhat, zero, one, zero)
vc_set(test_vec_1, mesh_xzm(g2_x, 2, 1, 1) - mesh_xzm(g2_x, 1, 1, 1), zero, mesh_xzm(g2_z, 2, 1,
    1) - mesh_xzm(g2_z, 1, 1, 1))
vc_set(test_vec_2, mesh_xzm(g2_x, 3, 1, 1) - mesh_xzm(g2_x, 2, 1, 1), zero, mesh_xzm(g2_z, 3, 1,
    1) - mesh_xzm(g2_z, 2, 1, 1))
vc_cross(test_vec_1, test_vec_2, test_vec_3)
if (vc_product(test_vec_3, yhat) > zero) then
    mesh_sense = 1
else if (vc_product(test_vec_3, yhat) < zero) then
    mesh_sense = 2
else
    assert('First_cell_of_mesh_degenerate' ≡ '␣')
end if
do ix = 1, nxd
    do iz = 1, nzd
        zone++
        n = 0
        increment_g2_num_polygons

        g2_polygon_xz g2_num_polygons,n,g2_x = mesh_xzm(g2_x, 1, ix, iz)
        g2_polygon_xz g2_num_polygons,n,g2_z = mesh_xzm(g2_z, 1, ix, iz);
        n++

        if (mesh_sense ≡ 1) then
            g2_polygon_xz g2_num_polygons,n,g2_x = mesh_xzm(g2_x, 2, ix, iz)
            g2_polygon_xz g2_num_polygons,n,g2_z = mesh_xzm(g2_z, 2, ix, iz);
            n++
            g2_polygon_xz g2_num_polygons,n,g2_x = mesh_xzm(g2_x, 3, ix, iz)
            g2_polygon_xz g2_num_polygons,n,g2_z = mesh_xzm(g2_z, 3, ix, iz);
            n++
            g2_polygon_xz g2_num_polygons,n,g2_x = mesh_xzm(g2_x, 4, ix, iz)
            g2_polygon_xz g2_num_polygons,n,g2_z = mesh_xzm(g2_z, 4, ix, iz);
            n++

        else if (mesh_sense ≡ 2) then
            g2_polygon_xz g2_num_polygons,n,g2_x = mesh_xzm(g2_x, 4, ix, iz)
            g2_polygon_xz g2_num_polygons,n,g2_z = mesh_xzm(g2_z, 4, ix, iz);
            n++
            g2_polygon_xz g2_num_polygons,n,g2_x = mesh_xzm(g2_x, 3, ix, iz)
            g2_polygon_xz g2_num_polygons,n,g2_z = mesh_xzm(g2_z, 3, ix, iz);
            n++
            g2_polygon_xz g2_num_polygons,n,g2_x = mesh_xzm(g2_x, 2, ix, iz)
            g2_polygon_xz g2_num_polygons,n,g2_z = mesh_xzm(g2_z, 2, ix, iz);
            n++

        else
            assert('mesh_sense_improperly_set' ≡ '␣')
        end if
        g2_polygon_xz g2_num_polygons,n,g2_x = g2_polygon_xz g2_num_polygons,0,g2_x
    
```

```

g2_polygon_xz g2_num_polygons, n, g2_z = g2_polygon_xz g2_num_polygons, 0, g2_z
do i = 0, n - 1
  g2_polygon_segment g2_num_polygons, i = i
end do
g2_polygon_segment g2_num_polygons, n = 0    /* Now also have g2_polygon_stratum to set, but not
clear if there is any intelligent use for it since we already have the zone number. */
g2_polygon_points g2_num_polygons = n
g2_polygon_zone g2_num_polygons = zone
/* These are really not used for solid zones and will probably contain the default values. */
do i_prop = 1, poly_int_max
  poly_int_props g2_num_polygons, i_prop = current_int_props i_prop
end do
do i_prop = 1, poly_real_max
  poly_real_props g2_num_polygons, i_prop = current_real_props i_prop
end do
@if 0
  zonearray_0 = zone
  call decompose_polygon(n, g2_polygon_xz g2_num_polygons, 0, g2_x, zonearray, 1, facearray)
#else
  do i = 0, y_div - 1
    if (i > 0)
      zone = zone + 1
      zonearray_i = zone
      zone_type_array_i = "plasma"
    end do
    call decompose_polygon(n, g2_polygon_xz g2_num_polygons, 0, g2_x, zonearray, y_div, facearray)
  @endif
  /* Check to see if the mesh center is zero. If so, set using existing routines (note: triangles can
now be specified in a Sonnet mesh format, but only if corner 4 matches corner 1 or 3). */
  if ((mesh_xzm(g2_x, 0, ix, iz) == zero) ^ (mesh_xzm(g2_z, 0, ix, iz) == zero)) then
    if (((mesh_xzm(g2_x, 4, ix, iz) == mesh_xzm(g2_x, 1, ix, iz)) ^ (mesh_xzm(g2_z, 4, ix,
iz) == mesh_xzm(g2_z, 1, ix, iz))) ^ ((mesh_xzm(g2_x, 4, ix, iz) == mesh_xzm(g2_x, 3,
ix, iz)) ^ (mesh_xzm(g2_z, 4, ix, iz) == mesh_xzm(g2_z, 3, ix, iz)))) then
      call triangle_centroid(mesh_xzm(g2_x, 1, ix, iz), center)
    else
      call quad_center(mesh_xzm(g2_x, 1, ix, iz), center)
    end if
  else
    vc_set(center, mesh_xzm(g2_x, 0, ix, iz), zero, mesh_xzm(g2_z, 0, ix, iz))
  end if
  call update_zone_info(zonearray, ix, iz, zone_type_array, n, g2_polygon_xz g2_num_polygons, 0, g2_x,
center, y_div, y_values)
enddo
enddo

```

This code is used in section 9.4.

13 Set up mesh elements

Convert the edges of the Sonnet or UEDGE mesh into nodes and elements analogous to those read in from the DG file. This step facilitate their use in polygons.

⟨Set Mesh Elements 13⟩ ≡

```

/* These are the basic parameters describing the mesh. The directional names for the edges comes
   from an old B2 convention. The basic idea is that looping over these edges corresponds to a
   counterclockwise trip around the computational mesh. The corresponding values for the corner
   indices can be verified using the diagram provided in the introductory documentation of this code.
   */
mesh_ix_start_mesh_south = 1
mesh_ix_end_mesh_south = nxd
mesh_iz_start_mesh_south = 1
mesh_iz_end_mesh_south = 1
mesh_edge_num_elements_mesh_south = nxd
mesh_ix_step_mesh_south = 1
mesh_iz_step_mesh_south = 0
mesh_corner_mesh_south.element_start = 1
mesh_corner_mesh_south.element_end = 4
mesh_edge_hv_mesh_south = mesh_h

mesh_ix_start_mesh_east = nxd
mesh_ix_end_mesh_east = nxd
mesh_iz_start_mesh_east = 1
mesh_iz_end_mesh_east = nxd
mesh_edge_num_elements_mesh_east = nxd
mesh_ix_step_mesh_east = 0
mesh_iz_step_mesh_east = 1
mesh_corner_mesh_east.element_start = 4
mesh_corner_mesh_east.element_end = 3
mesh_edge_hv_mesh_east = mesh_v

mesh_ix_start_mesh_north = nxd
mesh_ix_end_mesh_north = 1
mesh_iz_start_mesh_north = nxd
mesh_iz_end_mesh_north = nxd
mesh_edge_num_elements_mesh_north = nxd
mesh_ix_step_mesh_north = -1
mesh_iz_step_mesh_north = 0
mesh_corner_mesh_north.element_start = 3
mesh_corner_mesh_north.element_end = 2
mesh_edge_hv_mesh_north = mesh_h

mesh_ix_start_mesh_west = 1
mesh_ix_end_mesh_west = 1
mesh_iz_start_mesh_west = nxd
mesh_iz_end_mesh_west = 1
mesh_edge_num_elements_mesh_west = nxd
mesh_ix_step_mesh_west = 0
mesh_iz_step_mesh_west = -1
mesh_corner_mesh_west.element_start = 2
mesh_corner_mesh_west.element_end = 1

```

```

mesh_edge_hv mesh_west = mesh_v
/* Set up the nodes and elements contained in the four (mesh_num_edges) mesh edges. */
mesh_tot_elements = 0
mesh_tot_nodes = 0
do iedge = 1, mesh_num_edges
  ix = mesh_ix_start_iedge
  iz = mesh_iz_start_iedge
  do iseg = 1, mesh_edge_num_elements_iedge /* Each segment of a mesh edge yields a new
    (presumably!) element, with a start and an end. */
    mesh_tot_elements++
    mesh_edge_elements_iseg,iedge = mesh_tot_elements
    mesh_edge_dg_label_iseg,iedge = (iz + (iz_min - 1) - 1) * nxd_0 + (ix + (ix_min - 1) - 1)
    if (iedge == mesh_east) then
      if (ix_max < nxd_0) then
        mesh_edge_dg_label_iseg,iedge ++
      else
        mesh_edge_dg_label_iseg,iedge = -(mesh_edge_dg_label_iseg,iedge + 1)
      end if
    else if (iedge == mesh_north) then
      if (iz_max < nzd_0) then
        mesh_edge_dg_label_iseg,iedge += nxd_0
      else
        mesh_edge_dg_label_iseg,iedge = -(mesh_edge_dg_label_iseg,iedge + 1)
      end if
    end if
    do itip = element_start, element_end /* The points at each end of the segment is compared
      with the list of existing nodes and added to that list if new. */
      test_node_g2_x = mesh_xzm(g2_x, mesh_corner_iedge,itip, ix, iz)
      test_node_g2_z = mesh_xzm(g2_z, mesh_corner_iedge,itip, ix, iz)
      new_node = TRUE
      if (mesh_tot_nodes > 0) then
        do inode = 1, mesh_tot_nodes
          if ((mesh_nodes_inode,g2_x == test_node_g2_x) ^ (mesh_nodes_inode,g2_z == test_node_g2_z)) then
            assert(new_node == TRUE) // Right?
            new_node = FALSE
            this_node = inode
          end if
        end do
      end if
      if (new_node == TRUE) then
        mesh_tot_nodes++
        mesh_nodes_mesh_tot_nodes,g2_x = test_node_g2_x
        mesh_nodes_mesh_tot_nodes,g2_z = test_node_g2_z
        this_node = mesh_tot_nodes
      end if
      mesh_elements_mesh_tot_elements,itip = this_node
    end do // itip
    ix += mesh_ix_step_iedge
    iz += mesh_iz_step_iedge
  end do // iseg
  assert(ix == mesh_ix_end_iedge + mesh_ix_step_iedge)
  assert(iz == mesh_iz_end_iedge + mesh_iz_step_iedge)

```

```
end do     // iedge /* Note that mesh “cuts” are manifested as adjacent corners of adjacent elements
          not being identical. Since this situation is the exception rather than the rule, should really have
          mesh_tot_nodes  $\ll 4(nxd + nzd)$ .
assert(mesh_tot_nodes < 4 * (nxd + nzd))
assert(mesh_tot_elements  $\equiv 2 * (nxd + nzd)$ )
```

This code is used in section 9.4.

14 Specify and break up a new polygon

```

⟨ New Polygon 14 ⟩ ≡
  assert(prep_done ≡ TRUE)
  call specify_polygon(diskin, num_walls, wall_segment_count, wall_nodes, nodes, dg_elements_list,
    num_dg_poly, dg_poly_num_elements, dg_polygons, dg_poly_num_meshcon, dg_poly_meshcon,
    dg_poly_meshcon_hv, xb_min, xb_max, zb_min, zb_max, nxd, nzd, mesh_xz, mesh_nodes,
    mesh_elements, mesh_edge_num_elements, mesh_edge_elements, mesh_edge_dg_label,
    mesh_edge_hv, mesh_curve_num, mesh_scratch, temp_polygon0,g2_x, n, temp_int_props,
    temp_real_props, temp_aux_stratum, temp_stratum_pts, temp_num_stratum_pts, process_polygon)
  if (process_polygon ≠ clear_polygon) then
    ix = 0
    iz = 0
    do i_prop = 1, poly_int_max
      if (temp_int_propsi_prop ≠ int_uninit)
        current_int_propsi_prop = temp_int_propsi_prop
      end do
    do i_prop = 1, poly_real_max
      if (temp_real_propsi_prop ≠ real_uninit)
        current_real_propsi_prop = temp_real_propsi_prop
      end do
    if (process_polygon ≡ breakup_polygon) then
      increment g2_num_polygons
      do i = 0, n
        g2_polygon_xzg2_num_polygons,i,g2_x = temp_polygoni,g2_x
        g2_polygon_xzg2_num_polygons,i,g2_z = temp_polygoni,g2_z
        if (i < n) then
          g2_polygon_segmentg2_num_polygons,i = i
        else
          g2_polygon_segmentg2_num_polygons,i = 0
        end if
      end do
      g2_polygon_pointsg2_num_polygons = n
      g2_polygon_zoneg2_num_polygons = zone
      g2_polygon_stratumg2_num_polygons = current_int_props poly_stratum
      do i_prop = 1, poly_int_max
        poly_int_propsg2_num_polygons,i_prop = current_int_propsi_prop
      end do
      do i_prop = 1, poly_real_max
        poly_real_propsg2_num_polygons,i_prop = current_real_propsi_prop
      end do
      /* Had originally thought of dividing plasma and vacuum zones up in y, but assume
         user would be using triangulate_to_zones in that case. */
      solid_zone0 = zone
      call decompose_polygon(n, g2_polygon_xzg2_num_polygons,0,g2_x, solid_zone, 1, solid_faces)
      /* Try to handle carefully those cases where we can. For n > 4, just use the first point. If this
         zone really needs a center, can break up into triangles first. */
      if (n ≡ 3) then
        call triangle_centroid(temp_polygon0,g2_x, center)
      else if (n ≡ 4) then
        call quad_center(temp_polygon0,g2_x, center)
      end if
    end if
  end if

```

```

else
    vc_set(center, temp_polygon0,g2_x, zero, temp_polygon0,g2_z)
end if
one_zone_type0 = new_zone_type

call update_zone_info(solid_zone, ix, iz, one_zone_type, n, g2_polygon_xzg2_num_polygons,0,g2_x,
    center, 1, solid_ys)
/* Transfer data on auxiliary strata to main arrays. In this case, the original polygon point
   number can be used directly to identify the chosen segment in setup_sectors. */
if (temp_num_stratum_pts > 0) then
    do i = 1, temp_num_stratum_pts
        increment_num_aux_sectors
        aux_stratumnum_aux_sectors = temp_aux_stratum
        aux_stratum_polynum_aux_sectors = g2_num_polygons
        aux_stratum_pointsnum_aux_sectors = temp_stratum_ptsi
        /* Assign segment number for a stratum just by counting the number that have been
           specified. Easiest to just start at the end and find the last one. */
        temp_aux_segment = 1
        if (num_aux_sectors > 1) then
            do i_aux = num_aux_sectors - 1, 1, -1
                if (aux_stratumi_aux ≡ temp_aux_stratum) then
                    temp_aux_segment = aux_stratum_segmenti_aux + 1
                    go to sector_break
                end if
            end do
        end if
        sector_break: continue
    end if
    aux_stratum_segmentnum_aux_sectors = temp_aux_segment
end do
end if

else if (process_polygon ≡ triangulate_polygon ∨ process_polygon ≡ triangulate_to_zones) then
    if (process_polygon ≡ triangulate_to_zones) then
        refine = TRUE    // we care
    else
        refine = FALSE    // we're just breaking 'em up.
    end if
    if (current_int_propspoly_num_holes > 0) then    /* Not sure of the need for multiple holes.
        Easiest to implement just a single one. Note that the segment labels for the initial polygon
        are assigned in poly2triangles and transferred to the resulting triangles in temp_segment. */
        assert(current_int_propspoly_num_holes ≡ 1)
        temp_holes0,g2_x = current_real_propspoly_hole_x
        temp_holes0,g2_z = current_real_propspoly_hole_z
    end if
    call poly2triangles(n, temp_polygon0,g2_x, current_real_propspoly_min_area,
        current_int_propspoly_num_holes, temp_holes0,g2_x, refine, ntriangles,
        temp_triangles0,0,g2_x, temp_segment0,0)
    assert(ntriangles < max_triangles)
    nt = 3    // Number of points in a triangle !
    do j = 0, ntriangles - 1
        increment_g2_num_polygons
        do i = 0, nt

```

```

g2_polygon_xz g2_num_polygons,i,g2_x = temp_triangles j,i,g2_x
g2_polygon_xz g2_num_polygons,i,g2_z = temp_triangles j,i,g2_z
g2_polygon_segment g2_num_polygons,i = temp_segment j,i /* Transfer data on auxiliary strata
to main arrays. This is more complicated than breakup_polygon case above since we need
to search the triangles to identify which one contains the desired segment. */
if ((temp_num_stratum_pts > 0) ∧ (i ≠ 3)) then
  do k = 1, temp_num_stratum_pts
    /* The following is a little subtle. The user needs to specify polygons in the clockwise
    direction in order to preserve segment number. First, assume that the user does
    likewise in specifying auxiliary strata. Then, the selected polygon segment consists
    of the point indicated by temp_stratum_ptsk and that point number plus one. The
    corresponding triangle side must be labeled with both of these point numbers.
    Again, because we know the triangles are all clockwise, we can assume that the i + 1
    segment would match the larger point number. Furthermore, since the i = 0 and
    i = 3 points are the same, we don't have to worry that i = 3 will be hit first. One
    thing missed on first implementation was the need for the mod function here when
    the desired segment is the last one in the original polygon so that the second point in
    the segment has been assigned 0 instead of n. Also, initial code just had the assert
    below on i, allowing temp_segmentj,4 to be evaluated, accessing undefined memory.
    Instead, we now exclude i = 3 from the above if-then clause. */
    if ((temp_segment j,i ≡ temp_stratum_ptsk) ∧ (temp_segment j,i+1 ≡
      mod(temp_stratum_ptsk + 1, n))) then
      assert(i ≠ 3)
      increment_num_aux_sectors
      aux_stratum_num_aux_sectors = temp_aux_stratum
      aux_stratum_poly_num_aux_sectors = g2_num_polygons
      aux_stratum_points_num_aux_sectors = i /* As above, the segment number for a given
      stratum is just an enumeration of the segments comprising the stratum. */
      temp_aux_segment = 1
      if (num_aux_sectors > 1) then
        do i_aux = num_aux_sectors - 1, 1, -1
          if (aux_stratum i_aux ≡ temp_aux_stratum) then
            temp_aux_segment = aux_stratum_segment i_aux + 1
            go to sector_break2
          end if
        end do
      sector_break2: continue
      end if
      aux_stratum_segment_num_aux_sectors = temp_aux_segment
    end if
  end do
end if

end do
g2_polygon_points g2_num_polygons = nt
if (process_polygon ≡ triangulate_to_zones ∧ j > 0)
  zone = zone + 1 /* Write out stratum numbers with each of the newly created zones.
  Should probably incorporate this information into geometry2d. */
@#if 0
  if (process_polygon ≡ triangulate_to_zones ∨ j ≡ 0) then
    write(35, *) zone, current_int_props poly_stratum
  end if
@#endif

```

```

g2_polygon_zone g2_num_polygons = zone
g2_polygon_stratum g2_num_polygons = current_int_props poly_stratum
do i_prop = 1, poly_int_max
  poly_int_props g2_num_polygons, i_prop = current_int_props i_prop
end do
do i_prop = 1, poly_real_max
  poly_real_props g2_num_polygons, i_prop = current_real_props i_prop
end do
@if 0
  if ((new_zone_type == "solid") ∨ (new_zone_type == "exit") ∨ (process_polygon ==
    triangulate_polygon)) then
    @#else
      /* Have removed consideration of zone type in deciding whether or not to resolve zones in
        the y direction. This is now controlled entirely by process_polygon. */
      if (process_polygon == triangulate_polygon) then
        @#endif
          solid_zone_0 = zone
          one_zone_type_0 = new_zone_type
          call decompose_polygon(nt, g2_polygon_xz g2_num_polygons, 0, g2_x, solid_zone, 1, solid_faces)
        else
          @#if 0
            assert((new_zone_type == "plasma") ∨ (new_zone_type == "vacuum") ∨ (new_zone_type ==
              "mixed"))
          @#endif
            assert(process_polygon == triangulate_to_zones)
            do i = 0, y_div - 1
              if (i > 0)
                zone = zone + 1
                zonearray_i = zone
                if (new_zone_type ≠ "mixed")
                  zone_type_array_i = new_zone_type
                end do
                call decompose_polygon(nt, g2_polygon_xz g2_num_polygons, 0, g2_x, zonearray, y_div, facearray)
              end if
              if (j == 0 ∨ process_polygon == triangulate_to_zones)
                call triangle_centroid(temp_triangles_j, 0, g2_x, center)
              @#if 0
                if ((new_zone_type == "solid") ∨ (new_zone_type == "exit") ∨ (process_polygon ==
                  triangulate_polygon)) then
                  @#else
                    if (process_polygon == triangulate_polygon) then
                      @#endif
                        call update_zone_info(solid_zone, ix, iz, one_zone_type, nt,
                          g2_polygon_xz g2_num_polygons, 0, g2_x, center, 1, solid_ys)
                      else
                        call update_zone_info(zonearray, ix, iz, zone_type_array, nt,
                          g2_polygon_xz g2_num_polygons, 0, g2_x, center, y_div, y_values)
                      end if
                    end do
                  end if
                end if
              end if
            end if
          end if
        end if
      end if
    end if
  end if

```

§14 [#13] `definegeometry2d`

Specify and break up a new polygon 62

This code is used in section 9.

15 Set up trivial end zones

These are needed for “hardware” (quasi-3D) symmetry cases. The material is specified above and set below with the corresponding sectors. Do this at the end just in case having all of the plasma and vacuum zones defined first turns out to be useful.

```

⟨ Setup End Zones 15 ⟩ ≡
  if (symmetry ≡ geometry_symmetry_plane_hw) then
    vc_set(a_y, zero, one, zero)
    call plane(min_corner, a_y, coeff)
    ix = 0
    iz = 0
    end_faces_0 = define_surface(coeff, T)
    end_faces_1 = solid_faces_0
    zone++
    n = 0
    poly4_n,g2_x = min_corner_1
    poly4_n,g2_z = min_corner_3
    n++
    poly4_n,g2_x = min_corner_1
    poly4_n,g2_z = max_corner_3
    n++
    poly4_n,g2_x = max_corner_1
    poly4_n,g2_z = max_corner_3
    n++
    poly4_n,g2_x = max_corner_1
    poly4_n,g2_z = min_corner_3
    n++
    poly4_n,g2_x = poly4_0,g2_x
    poly4_n,g2_z = poly4_0,g2_z
    solid_zone_0 = zone
    end_zones_0 = zone
    call decompose_polygon(n, poly4_0,g2_x, solid_zone, 1, end_faces)
    call quad_center(poly4_0,g2_x, center)
    one_zone_type_0 = "solid"
    solid_ys_0 = min_corner_2
    solid_ys_1 = y_values_0
    call update_zone_info(solid_zone, ix, iz, one_zone_type, n, poly4_0,g2_x, center, 1, solid_ys)
    /* Back side */
    end_faces_0 = solid_faces_1
    call plane(max_corner, a_y, coeff)
    end_faces_1 = define_surface(coeff, T)
    zone++
    solid_zone_0 = zone
    end_zones_1 = zone
    call decompose_polygon(n, poly4_0,g2_x, solid_zone, 1, end_faces)
    one_zone_type_0 = "solid"
    solid_ys_0 = y_values_y_div
    solid_ys_1 = max_corner_2
    call update_zone_info(solid_zone, ix, iz, one_zone_type, n, poly4_0,g2_x, center, 1, solid_ys)
    /* Cylindrical section, front side */
  else if (symmetry ≡ geometry_symmetry_cyl_section) then

```

```

cos_y = cos(min_corner_2)
sin_y = sin(min_corner_2)
vc_set(a_y, -sin_y, cos_y, zero)
vc_set(b_y, max_corner_1 * cos_y, max_corner_1 * sin_y, max_corner_3)
call plane(b_y, a_y, coeff)
ix = 0
iz = 0
end_faces_0 = define_surface(coeff, T)
end_faces_1 = solid_faces_0
zone++
n = 0
poly4_{n,g2_x} = min_corner_1
poly4_{n,g2_z} = min_corner_3
n++
poly4_{n,g2_x} = min_corner_1
poly4_{n,g2_z} = max_corner_3
n++
poly4_{n,g2_x} = max_corner_1
poly4_{n,g2_z} = max_corner_3
n++
poly4_{n,g2_x} = max_corner_1
poly4_{n,g2_z} = min_corner_3
n++
poly4_{n,g2_x} = poly4_{0,g2_x}
poly4_{n,g2_z} = poly4_{0,g2_z}
solid_zone_0 = zone
end_zones_0 = zone
call decompose_polygon(n, poly4_{0,g2_x}, solid_zone, 1, end_faces)

call quad_center(poly4_{0,g2_x}, center)
one_zone_type_0 = "solid"
solid_ys_0 = min_corner_2
solid_ys_1 = y_values_0
call update_zone_info(solid_zone, ix, iz, one_zone_type, n, poly4_{0,g2_x}, center, 1, solid_ys)
/* Back side. As was done in setting up the universal cell surfaces, need to look out for the
special case of an exactly 180 degree section. */
if (abs(max_corner_2 - min_corner_2 - PI) > epsilon_angle) then
    cos_y = cos(max_corner_2)
    sin_y = sin(max_corner_2)
    vc_set(a_y, -sin_y, cos_y, zero)
    vc_set(b_y, max_corner_1 * cos_y, max_corner_1 * sin_y, max_corner_3)
    call plane(b_y, a_y, coeff)
    end_faces_1 = define_surface(coeff, T)
else /* The bounding faces are identical; need the minus sign to counter the one appearing in
process_polygon_cylindrical. */
    end_faces_1 = -end_faces_0
end if
end_faces_0 = solid_faces_1
zone++
solid_zone_0 = zone
end_zones_1 = zone
call decompose_polygon(n, poly4_{0,g2_x}, solid_zone, 1, end_faces)
one_zone_type_0 = "solid"

```

```

    solid_ys0 = y_valuesy_div
    solid_ys1 = max_corner2
    call update_zone_info(solid_zone, ix, iz, one_zone_type, n, poly40,g2-x, center, 1, solid_ys)
end if

```

This code is used in section 9.

16 Set up toroidally facing sectors

These are needed for toroidally mixed zone types in 3-D cases.

⟨ Setup Toroidally Facing Sectors 16 ⟩ ≡

```

if ((symmetry ≡ geometry-symmetry-cyl.hw) ∨ (symmetry ≡
    geometry-symmetry-cyl.section) ∨ (symmetry ≡ geometry-symmetry-plane.hw))
  then
    assert(y_div > 0)
    if (symmetry ≡ geometry-symmetry-cyl.hw) then
      y_max = y_div - 1     // First and last faces identical
    else
      y_max = y_div
    end if
    solid_sector = int_undef
    other_sector = int_undef
    do i = 0, y_max     /* Don't need to repeat for other side since it checks both sides. Should get
      num_zone1 > 1 and num_zone2 > 1 in the typical case. The principal exception should be
      the end zones in a box or toroidal section case. */
      face1 = abs(facearrayi)
      call find_poly_zone(face1, zn_undefined, zn_undefined, g2_num_polygons, sect_zone1,
        num_zone1, sect_zone2, num_zone2)
      if ((num_zone1 > 0) ∧ (num_zone2 > 0)) then
        other_seg = 0
        do k_zone1 = 1, num_zone1
          do k_zone2 = 1, num_zone2
            assert((num_zone1 ≡ num_zone2) ∨ (num_zone1 ≡ 1) ∨ (num_zone2 ≡ 1))
            if (((num_zone1 ≡ 1) ∨ (num_zone2 ≡ 1) ∨ (zn_index(sect_zone2k_zone2,
              zi_ptr) ≡ zn_index(sect_zone1k_zone1,
              zi_ptr))) ∧ (zn_type(sect_zone1k_zone1) ≠ zn_type(sect_zone2k_zone2)) ∧
              (zn_type(sect_zone1k_zone1) ≠ zn_exit) ∧ (zn_type(sect_zone2k_zone2) ≠ zn_exit))
                then
                  if (zn_type(sect_zone1k_zone1) ≡ zn_solid) then
                    solid_zone_p = sect_zone1k_zone1     // solid_zone used as array elsewhere
                    solid_face = face1
                    other_zone = sect_zone2k_zone2
                    other_face = -face1
                  else if (zn_type(sect_zone2k_zone2) ≡ zn_solid) then
                    solid_zone_p = sect_zone2k_zone2
                    solid_face = -face1
                    other_zone = sect_zone1k_zone1
                    other_face = face1     /* No other else since we can certainly have matching zone types
                      adjacent to one another. */
                  end if     /* For the end zones, we need to find their stratum numbers, etc. from the
                      corresponding arrays. For everything else, can search for the matching polygon.
                      If y_p_stratum is set in the input file, sectors are defined on the adjacent plasma
                      or vacuum zones. Note that the value of other_stratum is tested below against
                      int_unused to enforce this. The segment number is incremented for these sectors to
                      provide a distinguishing label in case they are used to define a diagnostic. */
                if (((symmetry ≡ geometry-symmetry-plane.hw) ∨ (symmetry ≡
                  geometry-symmetry-cyl.section)) ∧ ((solid_zone_p ≡ end_zones0) ∨ (solid_zone_p ≡
                  end_zones1))) then

```

```

if (solid_zone_p  $\equiv$  end_zones0) then
    this_stratum = y_stratum0
    this_mat = y_mat0
    other_stratum = y_p_stratum0
else
    this_stratum = y_stratum1
    this_mat = y_mat1
    other_stratum = y_p_stratum1
end if
this_seg = 1
other_seg++
this_temp = const(3., 2) * boltzmanns_const
this_rc = one
else /* Search the polygon information for the stratum number to be used. In this
    case, the “other” stratum and segment numbers are just set equal to the values
    found here for simplicity. */
    this_poly = int_undef
    do i_poly = 1, g2_num_polygons
        if (g2_polygon_zonei_poly  $\equiv$  zn_index(solid_zone_p, zi_ptr)) then
            assert(this_poly  $\equiv$  int_undef)
            this_poly = i_poly
        end if
    end do
    assert(this_poly  $\neq$  int_undef)
    this_stratum = poly_int_propsthis_poly, poly_stratum
    other_stratum = this_stratum
    this_mat = poly_int_propsthis_poly, poly_material
    this_temp = poly_real_propsthis_poly, poly_temperature * boltzmanns_const
    this_rc = poly_real_propsthis_poly, poly_recyc_coef /* Set the segment number to one
    larger than the largest one in use for this stratum. Note that this requires that
    setup_sectors be called before this since it uses the segment labels 1 through the
    number of polygon sides. */
    this_seg = int_undef
    do i_sect = 1, nsectors
        if (stratai_sect  $\equiv$  poly_int_propsthis_poly, poly_stratum) then
            if (this_seg  $\equiv$  int_undef) then
                this_seg = sector_strata_segmenti_sect
            else
                this_seg = max(this_seg, sector_strata_segmenti_sect)
            end if
        end if
    end do
    assert((this_seg  $\geq$  0)  $\wedge$  (this_seg  $\neq$  int_undef)) // Temporary diagnostic
    this_seg++
    other_seg = this_seg
end if
    /* To prevent redundant sector definitions (e.g., for num_zone1 = 1, num_zone2  $\geq$  1),
    check to see if the sector parameters compiled on this pass through the loop match
    the ones last used to define the sector of this type. */
    new_solid_sector = TRUE
    if (solid_sector  $\neq$  int_undef) then

```

```

        if (sc_compare(solid_sector, solid_zone_p, solid_face, this_stratum, this_seg))
            new_solid_sector = FALSE
    end if
    new_other_sector = TRUE
    if (other_sector  $\neq$  int_undef) then
        if (sc_compare(other_sector, other_zone, other_face, other_stratum, other_seg))
            new_other_sector = FALSE
        end if
        /* Define new sectors as needed. */
        if (new_solid_sector  $\equiv$  TRUE) then
            solid_sector = define_sector(this_stratum, this_seg, solid_face, solid_zone_p, other_zone)
            other_type = zn_type(other_zone)
            if (other_type  $\equiv$  zn_plasma) then
                define_sector_target(solid_sector, this_mat, this_temp, this_rc)
            else if (other_type  $\equiv$  zn_vacuum) then
                define_sector_wall(solid_sector, this_mat, this_temp, this_rc)
            else
                assert('Unexpected_zone_type'  $\equiv$  '_')
            end if
        end if
        if ((new_other_sector  $\equiv$  TRUE)  $\wedge$  (other_stratum  $\neq$  int_unused)) then
            other_sector = define_sector(other_stratum, other_seg, other_face, other_zone,
                solid_zone_p)
            other_type = zn_type(other_zone)
            if (other_type  $\equiv$  zn_plasma) then
                define_sector_plasma(other_sector)
            else if (other_type  $\equiv$  zn_vacuum) then
                define_sector_vacuum(other_sector)
            else
                assert('Unexpected_zone_type'  $\equiv$  '_')
            end if
        end if
    end if
    end if
    end if // adjacent zone types differ
end do // k_zone2
end do // k_zone1
end if // Nonzero num_zone1 and num_zone2
end do // Loop over y index
end if // Toroidally segmented symmetry

```

This code is used in section 9.

17 Set up sector based diagnostics, including those composed of auxiliary strata

⟨Setup Sector Diagnostics 17⟩ ≡

```

if (num_diags > 0) then
  do i_diag = 1, num_diags
    i_grp = 0
    do i_sect = 1, nsectors
      if (stratai_sect ≡ diag-stratumi_diag) then /* The auxiliary sectors were set up in such a way
        that the above test identifies them adequately. So, if this sector is an auxiliary sector, we
        can add it to the list for this diagnostic without any further checking. */
        is_aux_sector = FALSE
      do i_aux = 1, num_aux_sectors
        if ((stratai_sect ≡ aux-stratumi_aux) ∧ (sector-strata-segmenti_sect ≡
          aux-stratum-segmenti_aux)) then
          is_aux_sector = TRUE
        end if
      end do
      if (is_aux_sector ≡ TRUE) then
        i_grp++
        assert(i_grp ≤ g2-num-points) // Dim. of grp_sectors
        grp_sectorsi_grp = i_sect
      else /* However, if the diag-stratum in question is associated with a default sector, we
        need to dig further since the sectors on both side of a surface are assigned the same
        stratum number. We then use the polygon information to select only the sectors on
        one side of (both sides use the same stratum number). If we allow sectors on both
        sides in a diagnostics, the directional independent variables (mass-in, etc.) will not
        work properly. */
        if (((diag-solidi_diag ≡ TRUE) ∧ (sc-target-check(sector-type-pointeri_sect,sc-target) ∨
          sc-wall-check(sector-type-pointeri_sect,sc-wall) ∨
          sc-exit-check(sector-type-pointeri_sect,sc-exit))) ∨ ((diag-solidi_diag ≡
          FALSE) ∧ (sc-plasma-check(sector-type-pointeri_sect,sc-plasma) ∨
          sc-vacuum-check(sector-type-pointeri_sect,sc-vacuum)))) then
          i_grp++
          assert(i_grp ≤ g2-num-points) // Dim. of grp_sectors
          grp_sectorsi_grp = i_sect
        end if
      end if
    end do
  end do
  @#if 0
    do i_poly = 1, g2-num-polygons
      if ((g2-polygon-stratumi_poly ≡ diag-stratumi_diag) ∧ (zn-index(g2-polygon-zonei_poly,
        zi_ptr) ≡ zn-index(sector-zonei_sect, zi_ptr))) then
        i_grp++
        assert(i_grp ≤ g2-num-points) // Dim. of grp_sectors
        grp_sectorsi_grp = i_sect
      end if
    end do
  end if
end if

```

```

    end do
@#endif
    assert(i_grp > 0)
    assert(diag_namei_diag ≠ char_uninit)
    call diag_grp_init(diag_namei_diag, i_grp, diag_variablei_diag, diag_tab_indexi_diag, diag_var_mini_diag,
        diag_var_maxi_diag, diag_multi_diag, diag_spacingi_diag, grp_sectors)
    end do
end if

```

This code is used in section 9.

18 Read mesh file from UEDGE

```

"definegeometry2d.f" 18 ≡
  @m max_xpts 2

⟨Functions and subroutines 8⟩ +≡
  subroutine read_uedge_mesh(nunit, nxd, nzd, nxpt, mesh_xz)
    implicit_none_f77
    implicit_none_f90

    integer nunit, nxd, nzd, nxpt    // Input
    real mesh_xz nzd,nxd,0:4,g2_x:g2_z    // Output
    integer ix, iz, i    // Local
    integer corner_ptr0:4, iysptrx1 max_xpts, iysptrx2 max_xpts, ixlb max_xpts, ixpt1 max_xpts,
      ixmdp max_xpts, ixpt2 max_xpts, ixrb max_xpts
    real dummy    /* This file format now deals with both single and double nulls. Although we don't
      really need to know which for present purposes (since we have corner data for all zones), we
      do need to use the number of X-points, nxpt, to count off the initial lines. All of the other
      indices are read in for completeness and possible future use. */

    assert((nxpt ≥ 1) ∧ (nxpt ≤ max_xpts))
    do i = 1, nxpt
      read(nunit, *) iysptrx1_i, iysptrx2_i
      read(nunit, *) ixlb_i, ixpt1_i, ixmdp_i, ixpt2_i, ixrb_i
    end do    /* The specific ordering to be used in the rest of this code is that a polygon will be traced
      out in the clockwise direction by following the corners in order 1 → 4. The center is denoted as
      corner 0. The corner_ptr array is used to translate the known orientation of UEDGE mesh
      corners into the desired one. */
    corner_ptr_0 = 0
    corner_ptr_1 = 1
    corner_ptr_2 = 4
    corner_ptr_3 = 2
    corner_ptr_4 = 3

    read(nunit, *) SP(((mesh_xz iz,ix,corner_ptr_i,g2_x, ix = 1, nxd), iz = 1, nzd), i = 0, 4)
    read(nunit, *) SP(((mesh_xz iz,ix,corner_ptr_i,g2_z, ix = 1, nxd), iz = 1, nzd), i = 0, 4)

    dummy = zero
    open(unit = nunit + 1, file = 'uedge_sonnet', status = 'unknown', form = 'formatted')
    write(nunit + 1, *)
    write(nunit + 1, *) 'Element output:'
    write(nunit + 1, *)
    write(nunit + 1, '(a,f16.13)') 'R*Btor=0', dummy
    write(nunit + 1, *) 'ncut=0'
    write(nunit + 1, *)
    do iz = 1, nzd
      do ix = 1, nxd    /* The specific ordering to be used in the rest of this code is that a polygon will
        be traced out in the clockwise direction by following the corners in order 1 → 4. The corners
        used here with mesh_x and mesh_z are chosen so as to translate the known orientation of
        the Sonnet mesh corners into the desired one. The center is denoted as corner 0. */
        write(nunit + 1, '(a,i4,a,i3,a,i3,a,e17.10,a,e17.10,a,6x,a,e17.10,a,e17.10,a)')
          'Element', (iz-1)*nxd+ix-1, '(', ix-1, ', ', iz-1, ') : (', mesh_xz iz,ix,2,g2_x,
          ', ', mesh_xz iz,ix,2,g2_z, ')', '(', mesh_xz iz,ix,3,g2_x, ', ', mesh_xz iz,ix,3,g2_z, ')',

```

```

write(nunit + 1, '(a,e17.10,13x,a,e17.10,a,e17.10,a)') 'Field_ratio=', dummy,
  '(', mesh_xz_iz,ix,0,g2_x, ', ', mesh_xz_iz,ix,0,g2_z, ')',
write(nunit + 1, '(29x,a,e17.10,a,e17.10,a,6x,a,e17.10,a,e17.10,a)') '(',
  mesh_xz_iz,ix,1,g2_x, ', ', mesh_xz_iz,ix,1,g2_z, ')', '(', mesh_xz_iz,ix,4,g2_x, ', ',
  mesh_xz_iz,ix,4,g2_z, ')',
write(nunit + 1, *) '-----\
-----',
end do
end do
close(unit = nunit + 1)

return
end

```

19 Compute minimum and maximum coordinates

Search *nodes* array for maximum and minimum coordinate values that have been entered so far. These can then be used in setting the arguments of the *bounds* keyword.

(Functions and subroutines 8) +≡

subroutine *find_min_max*(*num_nodes*, *nodes*, *node_type*, *x_min*, *x_max*, *z_min*, *z_max*)

implicit none f77

implicit none f90

integer *num_nodes* // Input

integer *node_type**

real *nodes**,g2_x:g2_z

real *x_min*, *x_max*, *z_min*, *z_max* // Output

integer *inode* // Local

x_min = *const*(1., 5)

x_max = *zero*

z_min = *const*(1., 5)

z_max = *zero*

do *inode* = 1, *num_nodes*

if (*node_type*_{*inode*} ≠ *node_no_elements*) **then**

x_min = *min*(*x_min*, *nodes*_{*inode*,g2_x})

x_max = *max*(*x_max*, *nodes*_{*inode*,g2_x})

z_min = *min*(*z_min*, *nodes*_{*inode*,g2_z})

z_max = *max*(*z_max*, *nodes*_{*inode*,g2_z})

end if

end do

assert(*x_max* > *x_min*)

assert(*z_max* > *z_min*)

return

end

20 Print wall coordinates to a file for user examination

There are two possible formats. The first (“tabular”) has the walls laid out in successive columns, suitable for plotting with an external program. The second (“linear”) is consistent with the format used by the “wallfile” keyword. The user can provide the name of the file. If absent, the data are written to stdout.

```
"definegeometry2d.f" 20 ≡
  @m table_size 4    // Walls per table section

⟨Functions and subroutines 8⟩ +≡
  subroutine print_walls(nunit, file_format, walloutfile, num_walls, wall_segment_count, wall_nodes,
    nodes)

    implicit_none_f77
    implicit_none_f90

    integer nunit, num_walls    // Input
    integer wall_segment_count*, wall_nodes*, 0:g2_num_points-1
    real nodes*, g2_x:g2_z
    character*LINELEN file_format
    character*FILELEN walloutfile

    integer num_sections, isec, istart, iend, max_segs,    /* Local */
      iwall, iseg, inode, b, e
    character*2 csec
    character*130 wall_line    // Holds table_size (i.e., 4) pairs
    character*31 wall_chunk    // Space to write x,z pair

    st_decls

    if (file_format ≡ 'linear') then
      if (walloutfile ≠ char_undef) then
        open(unit = nunit, file = trim(walloutfile), status = 'unknown')
      end if
      write(nunit, *) num_walls
      write(nunit, *) SP(wall_segment_count_iwall + 1, iwall = 1, num_walls)
      do iwall = 1, num_walls
        do iseg = 0, wall_segment_count_iwall
          inode = wall_nodes_iwall,iseg
          write(nunit, '(1x,1pe18.10,2x,e18.10)') nodes_inode,g2_x, nodes_inode,g2_z
        end do
      end do
    else if (file_format ≡ 'tabular') then
      num_sections = (num_walls / table_size)
      if (num_sections * table_size < num_walls) then
        num_sections++
      end if
      if (num_sections * table_size > num_walls) then
        assert('Problem calculating num_sections' ≡ '␣')
      end if
      do isec = 1, num_sections
        if (walloutfile ≠ char_undef) then
          write(csec, '(i2.2)') isec
          open(unit = nunit, file = trim(walloutfile) || csec, status = 'unknown')
        end if
```

```

istart = table_size * (isec - 1) + 1
iend = min(table_size * isec, num_walls)
max_segs = 0
do iwall = istart, iend
    max_segs = max(max_segs, wall_segment_countiwall)
end do
do iseg = -1, max_segs
    wall_line = '␣␣'
    b = 1
    e = 2
    do iwall = istart, iend
        if (iseg ≡ -1) then
            write(wall_chunk, '(5x,a,i2.2,11x,a,i2.2,6x)') 'X_', iwall, 'Z_', iwall
        else if (iseg ≤ wall_segment_countiwall) then
            inode = wall_nodesiwall,iseg
            write(wall_chunk, '(1p14.6,1x,e14.6,2x)') nodesinode,g2_x, nodesinode,g2_z
        else /* KaleidaGraph likes using periods to denote empty data cells. It appears that
            Excel can live with them as well. */
            write(wall_chunk, '(7x,a,14x,a,7x)') '.', '.'
        end if
        wall_line = wall_line(b : e) || wall_chunk
        e += 30
    end do
    write(nunit, *) wall_line(b : e)
end do
if (walloutfile ≡ char_undef) then
    write(nunit, *)
    write(nunit, *) '-----',
    write(nunit, *)
else
    close(unit = diskout)
end if
end do
else
    write(stderr, *) '␣The␣format␣for␣the␣wall␣file␣must␣be␣either␣"linear"␣or␣"tabular"'
end if

return
end

```

21 Read input lines specifying a polygon

The structures in the argument list can get a little confusing. Let’s spell them out here at least.

nunit The logical unit number to which the input file is attached.

num_walls Number of walls created during the preparatory stage, either from a *wallfile* or a *dg_file* command.

wall_segment_count Number of elements (or, equivalently, segments) in each of these walls.

wall_nodes List of nodes comprising these walls. Because the wall nodes and elements are constructed together at the same time, they are trivially related and only the nodes are needed here. This array just contains pointers into the *nodes* array.

nodes Coordinates of all of the nodes specified in the preparatory stage via the *wallfile* and *dg_file* commands.

dg_elements_list List of the “start” and “end” nodes associated with each element described in the *dg_file*.

num_dg_poly Number of “polygons” (in the DG sense) contained in the *dg_file*. Note that these may be open or closed.

dg_poly_num_elements Number of elements in each of the DG polygons contained in the *dg_file*.

dg_polygons List of elements in each of the DG polygons contained in the *dg_file*.

dg_poly_num_meshcon Number of mesh connections (referred to as “meshcon”) associated with each DG polygon. Currently, the only valid values are 0, 1, or 2.

dg_poly_meshcon Integer label partially identifying the mesh connections associated with each DG polygon.

dg_poly_meshcon_hv Switch (= *mesh_h* ↔ “horizontal” or *mesh_v* ↔ “vertical”) completing the identification of the mesh connections associated with each DG polygon.

xb_min Minimum value of *x* as set with the *bounds* command.

xb_max Maximum value of *x* as set with the *bounds* command.

zb_min Minimum value of *z* as set with the *bounds* command.

zb_max Maximum value of *z* as set with the *bounds* command.

nxd Number of cells in the first dimension of the input Sonnet or UEDGE mesh.

nzd Number of cells in the second dimension of the input Sonnet or UEDGE mesh.

mesh_xz Coordinates of the input Sonnet or UEDGE mesh.

mesh_nodes A list of (unique) nodes created from the four edges of the input Sonnet or UEDGE mesh. This array contains the coordinates rather than pointers to another array. These nodes are independent of those mentioned above.

mesh_elements Full list of the “start” and “end” nodes associated with each element created from the four edges of the input Sonnet or UEDGE mesh. No assumptions are made regarding the order or connectivity of the entries in this list.

mesh_edge_num_elements Number of elements (or segments) on each of the four edges of the input Sonnet or UEDGE mesh.

mesh_edge_elements List of (with *mesh_edge_num_elements* telling how many) elements in each of the four edges of the input Sonnet or UEDGE mesh.

mesh_edge_dg_label Integer used by DG to partially label mesh elements.

mesh_edge_hv Switch (= *mesh_h* $\leftrightarrow$ “horizontal” or *mesh_v* $\leftrightarrow$ “vertical”) that completes DG’s label for the mesh elements.

mesh_curve_num Array dimensioned in the calling program for use below by the *iedge* command. It will contain labels identifying which of one or more simply connected “curves” (equivalent to “walls”) created from the mesh edges.

mesh_scratch Array dimensioned in the calling program to provide scratch space for the *iedge* command.

```

"definegeometry2d.f" 21 ≡
  @m dim_curves 5    // Used in determining topology of mesh edges
  @m curve_open 1
  @m curve_closed 2

  @m order_undefined 0    // Used to keep track of order of iedge edges
  @m order_decreasing -1
  @m order_increasing 1

  @m poly_loop #:0
  @m outer_loop #:0
  @m iedge_loop #:0
  @m forward_loop #:0
  @m backward_loop #:0

<Functions and subroutines 8> +≡
  subroutine specify_polygon(nunit, num_walls, wall_segment_count, wall_nodes, nodes,
    dg_elements_list, num_dg_poly, dg_poly_num_elements, dg_polygons, dg_poly_num_meshcon,
    dg_poly_meshcon, dg_poly_meshcon_hv, xb_min, xb_max, zb_min, zb_max, nxd, nzd, mesh_xz,
    mesh_nodes, mesh_elements, mesh_edge_num_elements, mesh_edge_elements,
    mesh_edge_dg_label, mesh_edge_hv, mesh_curve_num, mesh_scratch, polygon, n, int_props,
    real_props, aux_stratum, aux_stratum_pts, num_aux_stratum_pts, process_polygon)

  implicit none f77
  ma-common    // Common
  implicit none f90

  integer nunit, num_walls, num_dg_poly, nxd, nzd    // Input
  integer wall_segment_count*, wall_nodes*, 0:g2_num_points-1, dg_elements_list*, element_start:element_end,
    dg_poly_num_elements*, dg_polygons*, 1:g2_num_points, dg_poly_num_meshcon*,
    dg_poly_meshcon*, 1:meshcon_max, 1:meshcon_elem_max,
    dg_poly_meshcon_hv*, 1:meshcon_max, 1:meshcon_elem_max,
    mesh_elements*, element_start:element_end, mesh_edge_num_elements, mesh_num_edges,
    mesh_edge_elements*, mesh_num_edges, mesh_edge_dg_label*, mesh_num_edges,
    mesh_edge_hv, mesh_num_edges, mesh_curve_num*, mesh_scratch*
  real xb_min, xb_max, zb_min, zb_max
  real nodes*, g2_x:g2_z, mesh_xz, nzd, nxd, 0:4, g2_x:g2_z, mesh_nodes*, g2_x:g2_z

  integer n, process_polygon, aux_stratum,    /* Output */
    num_aux_stratum_pts
  integer int_props 1:poly_int_max, aux_stratum_pts 0:g2_num_points-1
  real polygon 0:g2_num_points-1, g2_x:g2_z, real_props 1:poly_real_max

  integer length, p, b, e, i, wall, start, stop, temp,    /* Local */
    ix_start, ix_stop, iz_start, iz_stop, ix_step, iz_step, corner, ixi, izi, num, ix, iz, nout, step, i_prop,
    aux_stratum_def, aux_stratum_start, poly, j, num_ends, end_point, end_tip, print_polygon,
    iedge_points, num_curves, num_iedges, i_tot, closed_curve, j_min, j_max, n_iedge, iedge_order,
    i_mc, swap_edges, tmp_edge, tmp_seg, j_init, j_fin, common_node, edge_reverse, xcut
  integer end_nodes 2*g2_num_points, end_elements 2*g2_num_points, end_tips 2*g2_num_points,
    curve_type dim_curves, curve_start dim_curves, curve_start_tip dim_curves,
    iedges, mesh_num_edges, mc_edge 1:meshcon_max, 1:meshcon_elem_max,
    mc_seg 1:meshcon_max, 1:meshcon_elem_max, mc_node 4
  real poly_area, min_dist
  real iedge_temp 0:g2_num_points-1, g2_x:g2_z, dist 4
  character* FILELEN polyoutfile

```

```

character*LINELEN line, keyword

vc_decl(poly_p2)
vc_decl(iedge_p2)
vc_decl(delta)

st_decls
vc_decls

external polygon_area
real polygon_area

process_polygon = int_undef
print_polygon = FALSE
n = 0
poly = int_undef
iedge_points = 0    // Need this still?
num_iedges = 0    /* Set these to peculiar values. If not explicitly set in this routine, they will be
                    overridden by the “current” values in the calling routine. */
do i_prop = 1, poly_int_max
    int_props_i_prop = int_uninit
end do
do i_prop = 1, poly_real_max
    real_props_i_prop = real_uninit
end do

aux_stratum = int_uninit
num_aux_stratum_pts = 0
aux_stratum_def = FALSE

poly-loop: continue
  assert(read_string(diskin, line, length))
  assert(length ≤ len(line))
  length = parse_string(line(: length))
  p = 0
  assert(next_token(line, b, e, p))
  keyword = line(b : e)

  if (keyword ≡ 'outer') then
    ⟨Outer Keyword 21.1⟩

  else if (keyword ≡ 'wall') then
    ⟨Wall Keyword 21.2⟩

  else if (keyword ≡ 'edge') then
    ⟨Edge Keyword 21.3⟩

  else if (keyword ≡ 'dg_polygon') then
    ⟨DG Polygon Keyword 21.4⟩

  else if (keyword ≡ 'iedge') then
    ⟨Intelligent Edge Specification 21.5⟩

  else if (keyword ≡ 'reverse') then
    assert(n > 0)
    /* Use n - 1 here since the last point doesn't get set until process_polygon is set. */
    call reverse_polygon(n - 1, polygon0,g2_x)

  else if (keyword ≡ 'stratum') then

```



```

    assert(next_token(line, b, e, p))
    int_props_poly_stratum = read_integer(line(b : e))

else if (keyword  $\equiv$  'material') then
    assert(next_token(line, b, e, p))
    int_props_poly_material = ma_lookup(line(b : e))
    assert(ma_check(int_props_poly_material))

else if (keyword  $\equiv$  'temperature') then
    assert(next_token(line, b, e, p))
    real_props_poly_temperature = read_real(line(b : e))
    assert(real_props_poly_temperature > zero)

else if (keyword  $\equiv$  'recyc_coef') then
    assert(next_token(line, b, e, p))
    real_props_poly_recyc_coef = read_real(line(b : e))
    assert(real_props_poly_recyc_coef  $\geq$  zero  $\wedge$  real_props_poly_recyc_coef  $\leq$  one)

else if (keyword  $\equiv$  'triangle_area') then
    assert(next_token(line, b, e, p))
    real_props_poly_min_area = read_real(line(b : e))
    assert(real_props_poly_min_area > zero)

else if (keyword  $\equiv$  'triangle_hole') then
    assert(next_token(line, b, e, p))
    int_props_poly_num_holes = read_integer(line(b : e))
    if (int_props_poly_num_holes > 0) then
        assert(int_props_poly_num_holes  $\equiv$  1)
        assert(next_token(line, b, e, p))
        real_props_poly_hole_x = read_real(line(b : e))
        assert(next_token(line, b, e, p))
        real_props_poly_hole_z = read_real(line(b : e))
    end if

else if (keyword  $\equiv$  'aux_stratum') then
    /* Set up currently to handle only one auxiliary stratum per polygon. */
    assert(next_token(line, b, e, p))
    if (aux_stratum  $\equiv$  int_uninit) then
        aux_stratum = read_integer(line(b : e))
        assert(aux_stratum  $\neq$  int_uninit)
    else
        assert(aux_stratum  $\equiv$  read_integer(line(b : e)))
    end if
    aux_stratum_def = TRUE
    aux_stratum_start = n

else if (keyword  $\equiv$  'end_aux_stratum') then
    assert(aux_stratum_def  $\equiv$  TRUE)
    do i = aux_stratum_start, n - 1
        num_aux_stratum_pts++
        aux_stratum_pts_num_aux_stratum_pts = i
    end do
    assert(num_aux_stratum_pts > 0)
    aux_stratum_def = FALSE

```

```

else if (keyword ≡ 'print_polygon') then
  print_polygon = TRUE
  if (next_token(line, b, e, p)) then
    polyoutfile = line(b : e)
    nout = diskout
  else
    polyoutfile = char_undef
    nout = stdout
  end if

else if (keyword ≡ 'breakup_polygon') then
  process_polygon = breakup_polygon

else if (keyword ≡ 'triangulate_polygon') then
  process_polygon = triangulate_polygon

else if (keyword ≡ 'triangulate_to_zones') then
  process_polygon = triangulate_to_zones

else if (keyword ≡ 'clear_polygon') then
  process_polygon = clear_polygon

else
  write(stderr, *) 'Unexpected_polygon_keyword'
  process_polygon = clear_polygon
end if

if (process_polygon ≡ int_undef)
  go to poly_loop /* Everything else is done, so we can process the iedge keyword and / or mesh
                  connection data. */
⟨ Intelligent Edge Processing 21.6 ⟩
  /* Polygon is complete. Close it up if it is to be further processed. */
  if (process_polygon ≠ clear_polygon) then
    polygonn,g2-x = polygon0,g2-x
    polygonn,g2-z = polygon0,g2-z
  end if /* Print it out, if requested. */
  if (print_polygon ≡ TRUE) then
    if (polyoutfile ≠ char_undef) then
      open(unit = nout, file = trim(polyoutfile), status = 'unknown')
    end if
    write(nout, '(8x,a,15x,a)') 'X', 'Z'
    do i = 0, n - 1
      write(nout, '(1x,1pe14.6,2x,e14.6)') polygoni,g2-x, polygoni,g2-z
    end do
    if (polyoutfile ≠ char_undef) then
      close(unit = nout)
    end if
  end if
return
end

```

Add points along universal cell to current polygon.

⟨ Outer Keyword 21.1 ⟩ ≡

```

outer_loop: continue
  if (next_token(line, b, e, p)) then
    i = read_integer(line(b : e))
    if ( $i \equiv 0 \vee i \equiv 4$ ) then
      polygonn,g2_x = xb_min
      polygonn,g2_z = zb_min
    else if ( $i \equiv 1$ ) then
      polygonn,g2_x = xb_min
      polygonn,g2_z = zb_max
    else if ( $i \equiv 2$ ) then
      polygonn,g2_x = xb_max
      polygonn,g2_z = zb_max
    else if ( $i \equiv 3$ ) then
      polygonn,g2_x = xb_max
      polygonn,g2_z = zb_min
    end if
    n++
    assert( $n \leq g2\_num\_points - 1$ )
    go to outer_loop
  end if

```

This code is used in section 21.

Take points from a wall and add to current polygon.

```

⟨ Wall Keyword 21.2 ⟩ ≡
  assert(next_token(line, b, e, p))
  wall = read_integer(line(b : e))
  assert(wall > 0 ∧ wall ≤ num_walls)
  assert(next_token(line, b, e, p))
  if (line(b : e) ≡ '*') then
    start = 0 stop = wall_segment_countwall
  else
    start = read_integer(line(b : e))
    assert(next_token(line, b, e, p))
    if (line(b : e) ≡ '*') then stop = wall_segment_countwall
    else stop = read_integer(line(b : e))
    end if
  end if
  if (next_token(line, b, e, p)) then
    assert(line(b : e) ≡ 'reverse')
    temp = start start = stop stop = temp
  end if if ( start ≤ stop ) then
    step = 1 else if ( start > stop ) then
      step = -1
    end if
  do i = start , stop , step
    assert(i ≥ 0 ∧ i ≤ wall_segment_countwall)
    polygonn,g2_x = nodeswall_nodeswall,i,g2_x
    polygonn,g2_z = nodeswall_nodeswall,i,g2_z
    n++
    assert(n ≤ g2_num_points - 1)
  end do

```

This code is used in section 21.

Take points along a mesh edge and add to current polygon.

```

⟨Edge Keyword 21.3⟩ ≡
  assert(next_token(line, b, e, p))
  if (line(b : e) ≡ '*' ) then
    ix_start = 0
    ix_stop = nxd
  else
    ix_start = read_integer(line(b : e))
    assert(next_token(line, b, e, p))
    ix_stop = read_integer(line(b : e))
  end if
  assert(next_token(line, b, e, p))
  if (line(b : e) ≡ '*' ) then
    iz_start = 0
    iz_stop = nzd
  else
    iz_start = read_integer(line(b : e))
    assert(next_token(line, b, e, p))
    iz_stop = read_integer(line(b : e))
  end if  /* The reverse and xcut options can both appear and may be in either order. Hence, the
           extended logic here. */
  edge_reverse = FALSE
  xcut = FALSE
  if (next_token(line, b, e, p)) then
    if (line(b : e) ≡ 'reverse' ) then
      edge_reverse = TRUE
      if (next_token(line, b, e, p)) then
        assert(line(b : e) ≡ 'xcut' )
        xcut = TRUE
      end if
    else
      assert(line(b : e) ≡ 'xcut' )
      xcut = TRUE
      if (next_token(line, b, e, p)) then
        assert(line(b : e) ≡ 'reverse' )
        edge_reverse = TRUE
      end if
    end if
  end if
  if (edge_reverse ≡ TRUE) then
    temp = ix_start
    ix_start = ix_stop
    ix_stop = temp
    temp = iz_start
    iz_start = iz_stop
    iz_stop = temp
  end if
  if (iz_stop ≡ iz_start) then
    iz_step = 0
    if (ix_stop ≥ ix_start) then
      ix_step = 1

```

```

        num = ix_stop - ix_start + 1
    else
        ix_step = -1
        num = ix_start - ix_stop + 1
    end if
else if (ix_stop ≡ ix_start) then
    ix_step = 0
    if (iz_stop ≥ iz_start) then
        iz_step = 1
        num = iz_stop - iz_start + 1
    else
        iz_step = -1
        num = iz_start - iz_stop + 1
    end if
else
    assert('Not following a mesh edge' ≡ ' ')
end if

ix = ix_start
iz = iz_start
do i = 1, num
    if (xcut ≡ FALSE) then
        if (ix ≡ 0 ∧ iz ≡ 0) then
            corner = 1
            ixi = 1
            izi = 1
        else if (ix ≡ 0) then
            corner = 2
            ixi = 1
            izi = iz
        else if (iz ≡ 0) then
            corner = 4
            ixi = ix
            izi = 1
        else
            corner = 3
            ixi = ix
            izi = iz
        end if
    else // xcut == TRUE
        assert(ix ≠ 0) // A cut at ix = 0 makes no sense
        if (iz ≡ 0) then
            corner = 1
            ixi = ix
            izi = 1
        else
            corner = 2
            ixi = ix
            izi = iz
        end if
    end if
end if

polygonn,g2_x = mesh_xz izi,ixi,corner,g2_x

```

```
polygonn, g2_z = mesh_xz izi,ixi,corner,g2_z  
ix += ix_step  
iz += iz_step  
n++  
assert(n ≤ g2_num_points − 1)  
end do
```

This code is used in section 21.

Process points of a polygon that was contained in the DG file. This assumes that we know nothing about the connectivity of these elements. Hence, the process is more involved than that used for creating “walls” out of DG’s elements and nodes. Note that the node characterization done at that stage is not used here. Both open and closed loops are treated.

```

⟨ DG Polygon Keyword 21.4 ⟩ ≡
  assert(next_token(line, b, e, p))
  poly = read_integer(line(b : e))
  assert(poly > 0 ∧ poly ≤ num_dg_poly)    /* The task here is to determine the desired ordering of the
    nodes in the DG polygon, i.e., their connectivity. In the case of an open loop, we do not know a
    priori which node is the first (or last) in the loop. This prevents the user from combining the
    dg_polygon command with any other construction keyword, except iedge. */
  assert(n ≡ 0)
  call find_endpoints(dg_elements_list, dg_poly_num_elements_poly, dg_polygons_poly,1, num_ends,
    end_nodes, end_elements, end_tips)
  if (num_ends ≡ 0) then
    end_point = 1
    end_tip = element_start
  else if (num_ends ≡ 2) then
    end_point = end_elements_1
    end_tip = end_tips_1
  else    /* If you see this assertion, you likely have a stray, unintended element in this dg_polygon. */
    assert('should_have_0_or_2_endpoints_here' ≡ ' ')
  end if
  call elements_to_polygon(nodes, dg_elements_list, dg_poly_num_elements_poly, dg_polygons_poly,1,
    end_point, end_tip, n, polygon_0,g2_x)
  if (num_ends ≡ 0) then    /* The polygon closes on itself. The assertion verifies this. We can then
    check the orientation of the polygon. If counter-clockwise, reverse the order of the points. */
    assert(n ≡ dg_poly_num_elements_poly)
    assert((polygon_n,g2_x ≡ polygon_0,g2_x) ∧ (polygon_n,g2_z ≡ polygon_0,g2_z))
    poly_area = polygon_area(n, polygon_0,g2_x)
    if (poly_area < zero)
      call reverse_polygon(n, polygon_0,g2_x)
    else
      assert(n ≡ dg_poly_num_elements_poly + 1)
      /* Pretty sure this test is supposed to be the “else” version. Because the nth entry is usually
      garbage, the first version has been succeeding regardless of the situation. */
    @#if 0
      assert((polygon_n,g2_x ≠ polygon_0,g2_x) ∨ (polygon_n,g2_z ≠ polygon_0,g2_z))
    @#else
      assert((polygon_n-1,g2_x ≠ polygon_0,g2_x) ∨ (polygon_n-1,g2_z ≠ polygon_0,g2_z))
    @#endif
  end if

```

This code is used in section 21.

Isolate a mesh edge for subsequent addition to current polygon. The “intelligent” part arises from the search procedure that attempts to determine the portion of that edge the user needs. This bit of code just reads in the specification. The actual processing takes place once the *process_polygon* command has been issued.

```

⟨ Intelligent Edge Specification 21.5 ⟩ ≡
    assert(num_iedges ≡ 0)    // Only allow one invocation
iedge_loop: continue
    if (next_token(line, b, e, p)) then
        num_iedges++
        assert(num_iedges ≤ mesh_num_edges)
        if (line(b : e) ≡ 'S') then
            iedges_num_iedges = mesh_south
        else if (line(b : e) ≡ 'E') then
            iedges_num_iedges = mesh_east
        else if (line(b : e) ≡ 'N') then
            iedges_num_iedges = mesh_north
        else if (line(b : e) ≡ 'W') then
            iedges_num_iedges = mesh_west
        else
            write(stderr, *) 'Unexpected_iedge_specification,', line(b : e)
            assert( $\mathcal{F}$ )
        end if
        go to iedge_loop
    end if
    assert(num_iedges > 0)    /* Check order and connectivity of iedges. This makes explicit use of the
        integer values of the macros associated with these labels. More specifically, the edges are assumed
        to trace out a counter-clockwise loop, S → E → N → W which corresponds to monotonically
        increasing values for the integer macro values (modulo mesh_num_edges = 4). Hence, the specified
        list must either be in this order or in the opposite of this order. In the latter case, the list is
        reversed. */
    if (num_iedges > 1) then
        iedge_order = order_undefined
        do i = 1, num_iedges - 1
            if (mod(iedgesi+1, mesh_num_edges) ≡ mod((iedgesi + 1), mesh_num_edges)) then
                if (iedge_order ≡ order_undefined) then
                    iedge_order = order_increasing
                else
                    assert(iedge_order ≡ order_increasing)
                end if
            else if (mod(iedgesi+1, mesh_num_edges) ≡ mod((iedgesi - 1), mesh_num_edges)) then
                if (iedge_order ≡ order_undefined) then
                    iedge_order = order_decreasing
                else
                    assert(iedge_order ≡ order_decreasing)
                end if
            else
                assert('Invalid_iedge_ordering!' ≡ '')
            end if
        end do
        if (iedge_order ≡ order_decreasing) then
            do i = 1, num_iedges / 2
                j = iedgesi

```

```

        iedgesi = iedgesnum_iedges−i+1
        iedgesnum_iedges−i+1 = j
    end do
else
    assert(iedge_order ≡ order_increasing)
end if
end if

```

This code is used in section 21.

Remainder of the code for isolating a mesh edge. This portion does the processing.

```

⟨ Intelligent Edge Processing 21.6 ⟩ ≡
    if ((n ≡ 0) ∧ (num_iedges > 0)) then
        ⟨ Process Closed iedge Polygon 21.7 ⟩
    else if (n > 0) then
        if ((num_iedges ≡ 0) ∧ (poly ≠ int_undef)) then
            if (dg_poly_num_meshconpoly > 0) then
                /* If you see this assertion, you may have mesh connections unintentionally set for this
                dg_polygon. Go back to DG, unmark everything and hit the “set” button for both of these
                mesh connections (see that they are listed as “Empty”). */
                assert(dg_poly_num_meshconpoly ≡ 1)
                ⟨ Process Single Mesh Point 21.8 ⟩
            else
                assert(dg_poly_num_meshconpoly ≡ 0)    /* Do nothing; don’t need iedge processing. */
            end if
        else if (num_iedges > 0) then
            assert(poly ≠ int_undef)
            assert(dg_poly_num_meshconpoly ≡ 2)
            ⟨ Process and Match Up Mesh Edge 21.9 ⟩
        else
            assert((num_iedges ≡ 0) ∧ (poly ≡ int_undef))
            /* Do nothing; didn’t invoke either dg_polygon or iedge. */
        end if
    else
        if ((process_polygon ≠ clear_polygon) ∧ (n ≡ 0) ∧ (num_iedges ≡ 0)) then
            assert('Null_⊔polygon!' ≡ '⊔')
        end if
    end if

```

This code is used in section 21.

Look for a closed curve along the specified mesh edge(s) and turn it into a polygon.

⟨Process Closed iedge Polygon 21.7⟩ ≡

```

    assert(poly ≡ int_undef)
    /* This is the only keyword specified for this polygon. We need to look for a closed curve. First,
       assemble the elements from the specified edge(s) into a single array. Note that the dimensions of
       mesh_edge_elements are ordered such that it cannot be used directly in an argument list. */
    i_tot = 0
    do i = 1, num_iedges
        do j = 1, mesh_edge_num_elements_iedges_i
            i_tot++
            mesh_scratch_i_tot = mesh_edge_elements_j,iedges_i
        end do
    end do
    /* Determine how many open and closed curves are described by this set of elements. */
    call count_curves(mesh_elements, i_tot, mesh_scratch, num_curves, curve_type, curve_start,
                     curve_start_tip, mesh_curve_num)
    assert(num_curves ≥ 1)
    /* Go through the resulting list to see if any of them are closed. We need one and only one. If we
       encounter situations in which more than one is legitimately possible, need to allow for it. */
    closed_curve = 0
    do i = 1, num_curves
        if (curve_type_i ≡ curve_closed) then
            assert(closed_curve ≡ 0)
            closed_curve = i
        end if
    end do
    assert(closed_curve ≠ 0)
    /* Finally, convert the list of elements corresponding to that curve into a polygon. */
    call elements_to_polygon(mesh_nodes, mesh_elements, i_tot, mesh_scratch, curve_start_closed_curve,
                           curve_start_tip_closed_curve, n, polygon_0,g2_x)
    /* Check to see if the polygon is clockwise (positive area); reverse it if not. */
    poly_area = polygon_area(n, polygon_0,g2_x)
    if (poly_area < zero)
        call reverse_polygon(n, polygon_0,g2_x)
    end if

```

This code is used in section 21.6.

Use the mesh connection information to identify a single point and add it to the existing polygon.

⟨ Process Single Mesh Point 21.8 ⟩ ≡

```

call get_mc_edge_seg(dg_poly_meshconpoly,1,1, dg_poly_meshcon_hvpoly,1,1, mesh_edge_num_elements,
    mesh_edge_dg_label, mesh_edge_hv, mc_edge1,1, mc_seg1,1) /* Collect the node numbers */
mc_node1 = mesh_elementsmesh_edge_elements mc_seg 1,1, mc_edge 1,1, element_start
mc_node2 = mesh_elementsmesh_edge_elements mc_seg 1,1, mc_edge 1,1, element_end
mc_node3 = mesh_elementsmesh_edge_elements mc_seg 1,2, mc_edge 1,2, element_start
mc_node4 = mesh_elementsmesh_edge_elements mc_seg 1,2, mc_edge 1,2, element_end

    /* Find the common one */
if ((mc_node1 ≡ mc_node3) ∨ (mc_node2 ≡ mc_node3)) then
    common_node = mc_node3
else if ((mc_node1 ≡ mc_node4) ∨ (mc_node2 ≡ mc_node4)) then
    common_node = mc_node4
else
    assert('Mesh_connection_elements_do_not_have_a_common_node!' ≡ '␣')
end if /* Add to the polygon */
polygonn,g2_x = mesh_nodescommon_node,g2_x
polygonn,g2_z = mesh_nodescommon_node,g2_z
n++ /* It should now be "closed". The orientation is still uncertain, so we need to check it and
    reverse if it's counter-clockwise. But, we need to explicitly fill in the last point to use these
    routines since the above operations would not have set it. */
polygonn,g2_x = polygon0,g2_x
polygonn,g2_z = polygon0,g2_z
poly_area = polygon_area(n, polygon0,g2_x)
if (poly_area < zero)
    call reverse_polygon(n, polygon0,g2_x)

```

This code is used in section 21.6.

Use the *iedge* and mesh connection information to identify a mesh edge and connect it to the existing polygon.

⟨Process and Match Up Mesh Edge 21.9⟩ ≡

```

do i_mc = 1, 2
  call get_mc_edge_seg(dg_poly_meshcon_poly,i_mc,1, dg_poly_meshcon_hv_poly,i_mc,1,
    mesh_edge_num_elements, mesh_edge_dg_label, mesh_edge_hv, mc_edge_i_mc,1, mc_seg_i_mc,1)
  /* Check order of the resulting edges and segments and swap if necessary so they conform to the
    convention for traversing the mesh edges. Also, verify that they are consecutive. */
  if (mod(mc_edge_i_mc,2, mesh_num_edges) ≡ mod((mc_edge_i_mc,1 + 1), mesh_num_edges)) then
    swap_edges = FALSE
    assert(mc_seg_i_mc,1 ≡ mesh_edge_num_elements_mc_edge_i_mc,1)
    assert(mc_seg_i_mc,2 ≡ 1)
  else if (mod(mc_edge_i_mc,2, mesh_num_edges) ≡ mod((mc_edge_i_mc,1 - 1), mesh_num_edges)) then
    swap_edges = TRUE
    assert(mc_seg_i_mc,1 ≡ 1)
    assert(mc_seg_i_mc,2 ≡ mesh_edge_num_elements_mc_edge_i_mc,2)
  else
    assert(mc_edge_i_mc,2 ≡ mc_edge_i_mc,1)
    if (mc_seg_i_mc,2 ≡ mc_seg_i_mc,1 + 1) then
      swap_edges = FALSE
    else if (mc_seg_i_mc,2 ≡ mc_seg_i_mc,1 - 1) then
      swap_edges = TRUE
    else
      assert('Bad_mesh_connection_data' ≡ '_')
    end if
  end if
  if (swap_edges ≡ TRUE) then
    tmp_edge = mc_edge_i_mc,1
    tmp_seg = mc_seg_i_mc,1
    mc_edge_i_mc,1 = mc_edge_i_mc,2
    mc_seg_i_mc,1 = mc_seg_i_mc,2
    mc_edge_i_mc,2 = tmp_edge
    mc_seg_i_mc,2 = tmp_seg
  end if
end do /* The result of this is two possible paths around the mesh edge(s): 1,2 → 2,1 or 2,2 → 1,1.
  We use the iedge specification to choose between them. */
if ((mc_edge_1,2 ≡ iedges_1) ∧ (mc_edge_2,1 ≡ iedges_num_iedges)) then
  j_init = mc_seg_1,2
  j_fin = mc_seg_2,1
else if ((mc_edge_2,2 ≡ iedges_1) ∧ (mc_edge_1,1 ≡ iedges_num_iedges)) then
  j_init = mc_seg_2,2
  j_fin = mc_seg_1,1
else
  assert('Mesh_connection_data_and_iedge_specification_inconsistent!' ≡ '_')
end if

i_tot = 0
do i = 1, num_iedges
  if (i ≡ 1) then
    j_min = j_init
  else
    j_min = 1
  end if
end do

```

```

    end if
    if (i ≡ num_iedges) then
        j_max = j_fin
    else
        j_max = mesh_edge_num_elements_iedges_i
    end if
    do j = j_min, j_max
        i_tot++
        mesh_scratch_i_tot = mesh_edge_elements_j_iedges_i
    end do
end do    /* Convert these into a polygon. Have to use a temporary array since we may still need to
invert the order to match up with the existing polygon. */
call elements_to_polygon(mesh_nodes, mesh_elements, i_tot, mesh_scratch, 1, element_start, n_iedge,
    iedge_temp_0,g2_x)    /* Now match up with the end points of the existing polygon. The default
ordering has the last point of polygon getting connected to the first point of iedge_temp (segment
“1” below) and, thus, the last point of iedge_temp connected to the first point of polygon (segment
“3”). The other possible ordering is represented by the other two segments and corresponds to
reversing iedge_temp. Use the shortest of the four segments to decide between the two orderings.
*/
vc_set(poly_p_1, polygon_0,g2_x, zero, polygon_0,g2_z)
vc_set(poly_p_2, polygon_{n-1,g2_x}, zero, polygon_{n-1,g2_z})
vc_set(iedge_p_1, iedge_temp_0,g2_x, zero, iedge_temp_0,g2_z)
vc_set(iedge_p_2, iedge_temp_{n_iedge-1,g2_x}, zero, iedge_temp_{n_iedge-1,g2_z})
vc_difference(poly_p_2, iedge_p_1, delta)
dist_1 = vc_abs(delta)
vc_difference(poly_p_1, iedge_p_1, delta)
dist_2 = vc_abs(delta)
vc_difference(poly_p_1, iedge_p_2, delta)
dist_3 = vc_abs(delta)
vc_difference(poly_p_2, iedge_p_2, delta)
dist_4 = vc_abs(delta)
min_dist = min(dist_1, dist_2, dist_3, dist_4)
if ((min_dist ≡ dist_2) ∨ (min_dist ≡ dist_4)) then
    call reverse_polygon(n_iedge - 1, iedge_temp_0,g2_x)
end if    /* Now add the mesh points to the polygon. */
do i = 0, n_iedge - 1
    i_tot = n + i
    assert(i_tot ≤ g2_num_points - 1)
    polygon_{i_tot,g2_x} = iedge_temp_{i,g2_x}
    polygon_{i_tot,g2_z} = iedge_temp_{i,g2_z}
end do
n = i_tot + 1    /* It should now be “closed”. The orientation is still uncertain, so we need to check it
and reverse if it’s counter-clockwise. But, we need to explicitly fill in the last point to use these
routines since the above operations would not have set it. */
polygon_{n,g2_x} = polygon_0,g2_x
polygon_{n,g2_z} = polygon_0,g2_z
poly_area = polygon_area(n, polygon_0,g2_x)
if (poly_area < zero)
    call reverse_polygon(n, polygon_0,g2_x)

```

This code is used in section 21.6.

22 Read data specifying a diagnostic

```

"definegeometry2d.f" 22 ≡
  @m diag_loop #:0

⟨ Functions and subroutines 8 ⟩ +=
  subroutine specify_diagnostic(nunit, stratum, solid, variable, tab_index, var_min, var_max, mult,
    spacing)

    implicit_none_f77
    implicit_none_f90

    integer nunit      // Input

    integer stratum, solid, variable, tab_index, spacing      // Output
    real var_min, var_max, mult

    integer length, p, b, e      // Local
    character*LINELEN line, keyword

    st_decls      /* Defaults */
    stratum = int_uninit
    solid = TRUE
    variable = sc_diag_unknown
    tab_index = 0
    var_min = zero
    var_max = zero
    mult = zero
    spacing = sc_diag_spacing_unknown

diag_loop: continue
  assert(read_string(nunit, line, length))
  assert(length ≤ len(line))
  length = parse_string(line(: length))
  p = 0
  assert(next_token(line, b, e, p))
  keyword = line(b:e)

  if (keyword ≡ 'stratum') then
    assert(next_token(line, b, e, p))
    stratum = read_integer(line(b:e))
    assert(stratum ≠ int_uninit)
    if (next_token(line, b, e, p)) then
      keyword = line(b:e)
      if (keyword ≡ 'nonsolid') then
        solid = FALSE
      else if (keyword ≡ 'solid') then
        solid = TRUE
      else
        write(stderr, *) 'Unexpected sector type, ', line(b:e)
        assert(F)
      end if
    end if
  end if

  else if (keyword ≡ 'variable') then

```

```

    ⟨ Variable Keyword 22.1 ⟩

else if (keyword  $\equiv$  'number') then
    assert(next_token(line, b, e, p))
    tab_index = read_integer(line(b:e))
    assert(tab_index > 0)

else if (keyword  $\equiv$  'minimum') then
    assert(next_token(line, b, e, p))
    var_min = read_real(line(b:e))

else if (keyword  $\equiv$  'maximum') then
    assert(next_token(line, b, e, p))
    var_max = read_real(line(b:e))

else if (keyword  $\equiv$  'multiplier') then
    assert(next_token(line, b, e, p))
    mult = read_real(line(b:e))
    assert(mult  $\neq$  zero)

else if (keyword  $\equiv$  'spacing') then
    assert(next_token(line, b, e, p))
    if (line(b:e)  $\equiv$  'linear') then
        spacing = sc_diag_spacing_linear
    else if (line(b:e)  $\equiv$  'log') then
        spacing = sc_diag_spacing_log
    else
        write (stderr, *) '␣Unexpected␣spacing,␣', line(b:e)
        assert( $\mathcal{F}$ )
    end if

else if (keyword  $\equiv$  'end_diagnostic') then
    assert(stratum  $\neq$  int_uninit)
    if (variable  $\neq$  sc_diag_unknown) then
        assert(tab_index > 0)
        assert(mult  $\neq$  zero)
        assert(var_min  $\neq$  var_max)
        assert(spacing  $\neq$  sc_diag_unknown)
        if (spacing  $\equiv$  sc_diag_spacing_log) then
            assert(var_min * mult > zero)
            assert(var_max * mult > zero)
        end if
    end if
    return

end if

go to diag_loop

end

```


Specify the diagnostic variable.

```

⟨ Variable Keyword 22.1 ⟩ ≡
  assert(next_token(line, b, e, p))
  if (line(b : e) ≡ 'energy') then
    variable = sc_diag_energy
    if (next_token(line, b, e, p)) then
      if (line(b : e) ≡ 'J') then
        mult = one
      else if (line(b : e) ≡ 'eV') then
        mult = electron_charge
      else
        write(stderr, *) 'Unexpected_energy_unit,', line(b : e)
        assert( $\mathcal{F}$ )
      end if
    else
      mult = electron_charge    // Default to eV
    end if
  else if (line(b : e) ≡ 'angle') then
    variable = sc_diag_angle
    if (next_token(line, b, e, p)) then
      if (line(b : e) ≡ 'radians') then
        mult = one
      else if (line(b : e) ≡ 'degrees') then
        mult = PI / const(1.8, 2)
      else
        write(stderr, *) 'Unexpected_angle_unit,', line(b : e)
        assert( $\mathcal{F}$ )
      end if
    else
      mult = PI / const(1.8, 2)    // Default to degrees
    end if
  else
    write(stderr, *) 'Unexpected_diagnostic_variable,', line(b : e)
    assert( $\mathcal{F}$ )
  end if

```

This code is used in section 22.

23 Fill in data for this zone

NOTE: the center point is set using the argument *center* from the first call to this routine for the current zone (i.e., the routine should be called for each polygon comprising a complex zone).

⟨ Functions and subroutines 8 ⟩ +≡

```
subroutine update_zone_info(zonearray, ind_x, ind_z, zone_type_array, n, polygon, center, y_div,
    y_values)
```

```
implicit none_f77
```

```
zn_common    // Common
```

```
gi_common
```

```
implicit none_f90
```

```
integer ind_x, ind_z, n, y_div    // Input
```

```
real polygon_0:g2_num_points-1,g2_x:g2_z, y_values_0:y_div
```

```
integer zonearray_0:y_div-1
```

```
character*(*) zone_type_array_0:y_div-1
```

```
vc_decl(center)
```

```
integer iy    // Local
```

```
real phi_mid, y_frac
```

```
vc_decls
```

```
gi_ext
```

```
do iy = 0, y_div - 1
```

```
  if (zn_volume(zonearray_iy)  $\equiv$  real_undef) then
```

```
    zn_volume(zonearray_iy) = zero
```

```
    zn_index(zonearray_iy, zi_ix) = ind_x
```

```
    zn_index(zonearray_iy, zi_iz) = ind_z
```

```
    zn_index(zonearray_iy, zi_iy) = iy
```

```
    zn_index(zonearray_iy, zi_ptr) = zonearray_0
```

```
  if (zone_type_array_iy  $\equiv$  "solid") then
```

```
    zn_type_set(zonearray_iy, zn_solid)
```

```
  else if (zone_type_array_iy  $\equiv$  "exit") then
```

```
    zn_type_set(zonearray_iy, zn_exit)
```

```
  else if (zone_type_array_iy  $\equiv$  "vacuum") then
```

```
    zn_type_set(zonearray_iy, zn_vacuum)
```

```
  else if (zone_type_array_iy  $\equiv$  "plasma") then
```

```
    zn_type_set(zonearray_iy, zn_plasma)
```

```
  else if (zone_type_array_iy  $\equiv$  "exit") then
```

```
    zn_type_set(zonearray_iy, zn_exit)
```

```
  else
```

```
    write (stderr, *) 'Unknown zone type: ', zone_type_array_iy
```

```
    assert( $\mathcal{F}$ )
```

```
  end if
```

```
  call set_zn_min_max(n, polygon_0:g2_x, zonearray_iy, T)    /* This isn't the same test that's
    used in process_polygon_plane to discern a 3-D case from a 2-D. Is it adequate? Explicitly
    reset only for non-symmetric cases to be consistent with previous usage. */
```

```
  if ((geometry_symmetry  $\equiv$  geometry_symmetry_plane_hw)  $\vee$  (geometry_symmetry  $\equiv$ 
    geometry_symmetry_cyl_hw)  $\vee$  (geometry_symmetry  $\equiv$  geometry_symmetry_cyl_section))
```

```
    then
```

```
      if (y_values_iy < y_values_{iy+1}) then
```

```

    zone_min zonearrayiy,2 = y_valuesiy
    zone_max zonearrayiy,2 = y_valuesiy+1
  else if (y_valuesiy > y_valuesiy+1) then
    zone_max zonearrayiy,2 = y_valuesiy
    zone_min zonearrayiy,2 = y_valuesiy+1
  end if
end if /* Think this will always produce something in the zone. Leave y value equal to zero
      in the symmetry cases so as to be consistent with previous usage. */
vc_copy(center, zone_center zonearrayiy)
if (geometry_symmetry ≡ geometry_symmetry_plane_hw) then
  zone_center zonearrayiy,2 = half * (y_valuesiy + y_valuesiy+1)
else if (geometry_symmetry ≡ geometry_symmetry_cyl_hw ∨ geometry_symmetry ≡
  geometry_symmetry_cyl_section) then
  assert(center2 ≡ zero)
  phi_mid = half * (y_valuesiy + y_valuesiy+1)
  zone_center zonearrayiy,1 = center1 * cos(phi_mid)
  zone_center zonearrayiy,2 = center1 * sin(phi_mid)
else
  assert((geometry_symmetry ≡ geometry_symmetry_oned) ∨ (geometry_symmetry ≡
    geometry_symmetry_plane) ∨ (geometry_symmetry ≡ geometry_symmetry_cylindrical))
end if
else
  /* This routine doesn't touch the y value on subsequent calls so that the above generalization
  to 3-D is sufficient. */
  call set_zn_min_max(n, polygon0,g2_x, zonearrayiy,  $\mathcal{F}$ )
end if
if (geometry_symmetry ≡ geometry_symmetry_plane_hw ∨ geometry_symmetry ≡
  geometry_symmetry_cyl_hw ∨ geometry_symmetry ≡ geometry_symmetry_cyl_section) then
  y_frac = abs(y_valuesiy+1 - y_valuesiy) / (universal_cell_max2 - universal_cell_min2)
else
  y_frac = one
end if
zn_volume(zonearrayiy) = zn_volume(zonearrayiy) + y_frac * polygon_volume(n, polygon0,g2_x)
end do
return
end

```

24 Define universal cell

Copied the version in *composite.web* and extended to handle nearly symmetric 3-D cases.

```

⟨ Functions and subroutines 8 ⟩ +=
  subroutine universal_cell_3d(symmetry, min_corner, max_corner, vol) implicit_none_f77
    gi_common
    implicit_none_f90
    integer symmetry    // Input
    real min_corner_3, max_corner_3
    real vol    // Output
    integer face, face1, face2, surf_type, i    // Local
    real cos_y, sin_y
    real x3_0_4, apex_3, a3, b3, x0_3, x1_3, x2_3, x3_3, coeff_NCOEFFS, tx_3_4

    gi_ext
    external vector_compare
    integer vector_compare

    geometry_symmetry = symmetry

    $DO (I, 1, 3)
    {
      assert(min_corner_I < max_corner_I);
    }
    vc_copy(min_corner, universal_cell_min)
    vc_copy(max_corner, universal_cell_max)

    i = 0
    x1_i = min_corner_1;
    x2_i = zero;
    x3_i = min_corner_3;
    i++
    x1_i = min_corner_1;
    x2_i = zero;
    x3_i = max_corner_3;
    i++
    x1_i = max_corner_1;
    x2_i = zero;
    x3_i = max_corner_3;
    i++
    x1_i = max_corner_1;
    x2_i = zero;
    x3_i = min_corner_3;
    i++
    vc_copy(x0, x_i)

    if ((symmetry ≡ geometry_symmetry_cylindrical) ∨ (symmetry ≡
      geometry_symmetry_cyl_hw) ∨ (symmetry ≡ geometry_symmetry_cyl_section))
      then

      do i = 0, 3
        call conea(xi,1, xi+1,1, coeff, surf_type, apex)
        face = define_surface(coeff, T)
        if (face > 0) then

```

```

        vc_copy(xi, surface_points0, face)
        vc_copy(xi+1, surface_points1, face)
    end if
    call add_surface(face, 0, T)
end do

if (symmetry ≡ geometry_symmetry_cyl_section) then
    cos_y = cos(min_corner2)
    sin_y = sin(min_corner2)
    vc_set(a, -sin_y, cos_y, zero)
    vc_set(b, max_corner1 * cos_y, max_corner1 * sin_y, max_corner3)
    call plane(b, a, coeff)
    face1 = define_surface(coeff, T)
    call add_surface(face1, 0, T)
    /* Check to see that the two ends are genuinely distinct surfaces. */
    if (abs(max_corner2 - min_corner2 - PI) > epsilon_angle) then
        cos_y = cos(max_corner2)
        sin_y = sin(max_corner2)
        vc_set(a, -sin_y, cos_y, zero)
        vc_set(b, max_corner1 * cos_y, max_corner1 * sin_y, max_corner3)
        call plane(b, a, coeff)
        face2 = define_surface(coeff, T)
        call add_surface(-face2, 0, T)
    end if
end if

vol = half * (max_corner2 - min_corner2) * (max_corner12 - min_corner12) * (max_corner3 - min_corner3)

else if ((symmetry ≡ geometry_symmetry_plane) ∨ (symmetry ≡ geometry_symmetry_oned) ∨ (symmetry ≡
    geometry_symmetry_plane_hw)) then
    vc_set(a, zero, one, zero)
    call plane(min_corner, a, coeff)
    face1 = define_surface(coeff, T)
    call add_surface(face1, 0, T)
    call plane(max_corner, a, coeff)
    face2 = define_surface(coeff, T)
    call add_surface(-face2, 0, T)
    if (symmetry ≠ geometry_symmetry_plane_hw) then
        call init_identity(tx)
        vc_set(x0, zero, -(max_corner2 - min_corner2), zero)
        call geom_translate(x0, tx)
        call add_transform(face1, face2, tx)
        call invert(tx, tx)
        call add_transform(-face2, -face1, tx)
    end if
end if

do i = 0, 3
    vc_copy(xi, x1)
    vc_copy(xi+1, x2)
    if (vector_compare(x1, x2) > 0) then
        vc_set(x3, x11, one, x13)
    else
        vc_set(x3, x21, one, x23)
    end if
    call planea(x1, x3, x2, coeff)

```

```

    face = define_surface_a(coeff, x1, x2)
    if (i ≡ 3)
        face1 = face
    if (i ≡ 1)
        face2 = -face
    if (face > 0) then
        vc_copy(x1, surface_points0, face)
        vc_copy(x2, surface_points1, face)
    end if
    call add_surface(face, 0, T)
end do

if (symmetry ≡ geometry_symmetry_oned) then
    call init_identity(tx)
    vc_set(x0, zero, zero, -(max_corner3 - min_corner3))
    call geom_translate(x0, tx)
    call add_transform(face1, face2, tx)
    call invert(tx, tx)
    call add_transform(-face2, -face1, tx)
end if

vol = (max_corner1 - min_corner1) * (max_corner2 - min_corner2) * (max_corner3 - min_corner3)
end if

universal_cell_vol = vol

return end

```

25 Find the centroid of a triangle

This is computed using the fact that the centroid (equivalently, the center of gravity and the intersection of the medians) is at trilinear coordinates $\frac{1}{a} : \frac{1}{b} : \frac{1}{c}$, where a , b , and c are the lengths of the sides of the triangles (for additional information see <http://mathworld.wolfram.com/TrilinearCoordinates.html>).

⟨ Functions and subroutines 8 ⟩ +≡

subroutine *triangle_centroid*(*triangle*, *center*)

implicit_none_f77

implicit_none_f90

real *triangle*_{0:3,g2_x:g2_z} // Input

vc_decl(*center*) // Output

real *area*, *k*, *a*, *b*, *c*, *alpha*, *gamma*, *den*, *lc* // Local

vc_decl(*a_vertex*)

vc_decl(*b_vertex*)

vc_decl(*c_vertex*)

vc_decl(*a_vec*)

vc_decl(*b_vec*)

vc_decl(*c_vec*)

vc_decl(*a_hat*)

vc_decl(*c_hat*)

vc_decl(*bc_cross*)

vc_decl(*c_perp*)

vc_decl(*neg_y*)

vc_decl(*m_test1*)

vc_decl(*m_test2*)

vc_decls

/* Associate point 0 with vertex A, 1 with B, and 2 with C. Here are the lengths of the sides. */

vc_set(*a_vertex*, *triangle*_{0,g2_x}, *zero*, *triangle*_{0,g2_z})

vc_set(*b_vertex*, *triangle*_{1,g2_x}, *zero*, *triangle*_{1,g2_z})

vc_set(*c_vertex*, *triangle*_{2,g2_x}, *zero*, *triangle*_{2,g2_z})

/* Vectors for two of the sides and the corresponding unit vectors. */

vc_difference(*b_vertex*, *a_vertex*, *c_vec*)

vc_unit(*c_vec*, *c_hat*)

vc_difference(*c_vertex*, *b_vertex*, *a_vec*)

vc_unit(*a_vec*, *a_hat*)

/* Perpendicular to $\hat{c}$, define using cross product to be sure it's pointing the right direction. */

vc_set(*neg_y*, *zero*, *−one*, *zero*)

vc_cross(*c_hat*, *neg_y*, *c_perp*) /* Lengths of all 3 sides. */

vc_difference(*a_vertex*, *c_vertex*, *b_vec*)

a = *vc_abs*(*a_vec*)

b = *vc_abs*(*b_vec*)

c = *vc_abs*(*c_vec*) /* Area by using the cross product: */

vc_cross(*b_vec*, *c_vec*, *bc_cross*)

area = *half* * *vc_abs*(*bc_cross*) /* Use α , β , γ to denote the trilinear coordinate values. A constant of proportionality k relates these to the distances of the points to the triangle sides. k can be determined for any particular set of coordinates via the area of the triangle.

$$k \equiv \frac{2\Delta}{a\alpha + b\beta + c\gamma},$$

In the case of the centroid with $\alpha = 1/a$, etc., the denominator is just 3. */

alpha = *one* / *a*

gamma = *one* / *c*

k = *two* * *area* / *const*(3.) /* The vectors used in the Web page referred to in the introduction to this routine do not appear to be a consistent set. The vectors we use here along the sides trace out the triangle in a clockwise direction. The Cartesian coordinates of the point of interest are written in terms of some distance l_c along side c (starting at vertex A) plus a distance $k\gamma$ along $\hat{c}_\perp$ (by definition of γ). A similar expression is written going from vertex C along $-\hat{a}$, and the two are solved for the two distances along the sides. We only need one here:

$$l_c = \frac{-k\alpha + \gamma k(\hat{a}_1\hat{c}_1 + \hat{a}_3\hat{c}_3) + \hat{a}_3(A_1 - C_1) + \hat{a}_1(C_3 - A_3)}{\hat{a}_1\hat{c}_3 - \hat{a}_3\hat{c}_1}.$$

Note: l_c can be negative for a really stretched out triangle. */

den = *a_hat*₁ * *c_hat*₃ - *a_hat*₃ * *c_hat*₁

assert(*den* > *zero*)

lc = (-*k* * *alpha* + *gamma* * *k* * (*a_hat*₁ * *c_hat*₁ + *a_hat*₃ * *c_hat*₃) + *a_hat*₃ * (*a_vertex*₁ - *c_vertex*₁) + *a_hat*₁ * (*c_vertex*₃ - *a_vertex*₃)) / *den*

/* Finally, the Cartesian coordinates are given in general by

$$\vec{x} = \vec{A} + l_c\hat{c} + k\gamma\hat{c}_\perp.$$

*/

vc_xvt(*a_vertex*, *c_hat*, *lc*, *center*)

vc_xvt(*center*, *c_perp*, *k* * *gamma*, *center*)

/* Verify that this point is indeed inside the triangle. */

vc_difference(*center*, *a_vertex*, *m_test1*)

vc_cross(*m_test1*, *c_vec*, *m_test2*)

assert(*vc_product*(*m_test2*, *neg_y*) > *zero*)

vc_difference(*center*, *b_vertex*, *m_test1*)

vc_cross(*m_test1*, *a_vec*, *m_test2*)

assert(*vc_product*(*m_test2*, *neg_y*) > *zero*)

vc_difference(*center*, *c_vertex*, *m_test1*)

vc_cross(*m_test1*, *b_vec*, *m_test2*)

assert(*vc_product*(*m_test2*, *neg_y*) > *zero*)

return

end

26 Compute the center of a quadrilateral

```

⟨Functions and subroutines 8⟩ +≡
  subroutine quad_center(quad, center)

    implicit_none_f77
    implicit_none_f90

    real quad0:4,g2_x:g2_z     // Input
    vc_decl(center)
    // Output /* Use this as a temporary approximation. To be useful, zone must be convex.
    vc_set(center, const(0.25) * (quad0,g2_x + quad1,g2_x + quad2,g2_x + quad3,g2_x), zero,
    const(0.25) * (quad0,g2_z + quad1,g2_z + quad2,g2_z + quad3,g2_z))

    return
end

```

27 Compute area of a closed polygon

This routine computes the area of a plane polygon using the formula employed in *polygon_volume*. That routine deals with the more general problem of computing the volume in both rectilinear and cylindrical geometries. The intended application of this more basic routine is to determine whether the points of a polygon are ordered clockwise (positive result) or counterclockwise (negative). As such, a simpler, self-contained routine is suitable.

There are $n + 1$ points in the polygon. The numbering starts at 0 and ends at n . The coincidence of those two points is checked. This formula requires few other assumptions. Duplicate points can appear. Segments may be colinear. “Holes” can even be accounted for as long as they are specified as part of a single list of connected points. I.e., this is the same manner used for specifying for Triangle polygons with holes.

```

⟨Functions and subroutines 8⟩ +≡
  function polygon_area(n, x)
    implicit_none_f77
    implicit_none_f90
    real polygon_area     // Function
    integer n     // Input
    real x0:n,g2_x:g2_z
    integer i     // Local
    real area     /* Use a local variable to facilitate debugging. */

    assert(x0,g2_x ≡ xn,g2_x)
    assert(x0,g2_z ≡ xn,g2_z)
    area = zero
    do i = 0, n - 1
      area += half * (xi,g2_x + xi+1,g2_x) * (xi,g2_z - xi+1,g2_z)
    end do

    polygon_area = area

    return
end

```

28 Reverse order of polygon points

The objective of this routine is to transform a counter-clockwise oriented polygon into one with clockwise orientation. While those designations make sense only for closed polygons, the routine will, of course, work on an arbitrary list of points. The input integer n gives the number of sides of the polygon; there are $n + 1$ points, numbered 0 through n .

⟨ Functions and subroutines 8 ⟩ +≡

```

subroutine reverse_polygon( $n$ ,  $x$ )
  implicit_none_f77
  implicit_none_f90

  integer  $n$     // Input
  real  $x_{0:n,g2\_x:g2\_z}$ 
  integer  $i$     // Local
  real  $x\_temp_{g2\_x:g2\_z}$     /* This should work for both even and odd  $n$ . */

  do  $i = 0, (n - 1) / 2$ 
     $x\_temp_{g2\_x} = x_{n-i,g2\_x}$ 
     $x\_temp_{g2\_z} = x_{n-i,g2\_z}$ 
     $x_{n-i,g2\_x} = x_{i,g2\_x}$ 
     $x_{n-i,g2\_z} = x_{i,g2\_z}$ 
     $x_{i,g2\_x} = x\_temp_{g2\_x}$ 
     $x_{i,g2\_z} = x\_temp_{g2\_z}$ 
  end do

  return
end

```

29 Find endpoints in a set of elements

This routine examines a set of DG-type elements (*poly-elements*, consisting of n elements), presumably connected, to find the nodes belonging to a single element, i.e., the endpoints of the element chain. The number of endpoints *num_ends* is returned, as are the corresponding node numbers (*end_nodes*), pointers into the input *poly-elements* list (*end_elements*), and tips of those elements (*end_tips*).

⟨ Functions and subroutines 8 ⟩ +≡

```

subroutine find_endpoints(elements_list, n, poly-elements, num_ends, end_nodes, end_elements,
    end_tips)
    implicit none f77
    implicit none f90

    integer n      // Input
    integer elements_list*, element_start:element_end, poly-elementsn

    integer num_ends      // Output
    integer end_nodes*, end_elements*, end_tips*

    integer i, tip, test_node, match, j      // Local

    num_ends = 0
    do i = 1, n
        tip = element_start, element_end
        test_node = elements_listpoly-elementsi,tip
        match = FALSE
        do j = 1, n
            if ((j ≠ i) ∧ ((elements_listpoly-elementsj,element_start ≡
                test_node) ∨ (elements_listpoly-elementsj,element_end ≡ test_node))) then
                /* Should only be one match! */
                assert(match ≡ FALSE)
                match = TRUE
            end if
        end do
        if (match ≡ FALSE) then
            num_ends++
            end_nodesnum_ends = test_node
            end_elementsnum_ends = i
            end_tipsnum_ends = tip
        end if
    end do
    end do      /* Number of endpoints should be even! */
    assert((2 * (num_ends / 2) - num_ends) ≡ 0)
    return
end

```

30 Convert elements to a polygon

This routine transforms a set of DG-type elements, *poly_elements*, into a linear list of points, a “polygon” as used elsewhere in this code, *poly_x*, consisting of *poly_num_points*. The elements need not be all consistently oriented (i.e., so that each interior node is the “end” of one element and the “start” of another). But, the routine does expect to finish with a single chain (two endpoints) or a closed polygon. In the former case, the input pointer to the initial element (*initial_element*) and tip (*initial_tip*) must correspond to one of those endpoints.

"definegeometry2d.f" 30 ≡

@m point_loop #:0

⟨ Functions and subroutines 8 ⟩ +≡

subroutine *elements_to_polygon*(*nodes*, *elements_list*, *n*, *poly_elements*, *initial_element*, *initial_tip*,
poly_num_points, *poly_x*)
implicit none_f77
implicit none_f90

integer *n*, *initial_element*, *initial_tip* // Input

integer *elements_list**,*element_start:element_end*, *poly_elements*_{*n*}

real *nodes**,*g2_x:g2_z*

integer *poly_num_points* // Output

real *poly_x*0:*,*g2_x:g2_z*

integer *i*, *j_match*, *node*, *tip*, *j*, *match*, *tip_match* // Local /* The task here is to determine the desired ordering of the nodes in the input element set, i.e., their connectivity. We assume that the caller has provided the initial point in the output polygon. We then try to find a node of another element in the set that matches its opposite tip. If we do, we add it to the output polygon. The *point_loop* continues in this way until we are left with a point that has no match. This is then one end of the open loop. If that doesn't occur (*j_match* = 1), the loop is closed.

assert(*initial_element* ≥ 1)

assert(*initial_element* ≤ *n*)

assert((*initial_tip* ≡ *element_start*) ∨ (*initial_tip* ≡ *element_end*))

poly_num_points = 0

i = *initial_element*

j_match = 0

node = *elements_list*_{*poly_elements* *initial_element*, *initial_tip*}

*poly_x*_{*poly_num_points*, *g2_x*} = *nodes*_{*node*, *g2_x*}

*poly_x*_{*poly_num_points*, *g2_z*} = *nodes*_{*node*, *g2_z*}

poly_num_points++ /* This restriction may not apply in general. Leave for now until we can replace it with a corresponding assertion in the calling program. */

assert(*poly_num_points* ≤ *g2_num_points* - 1)

if (*initial_tip* ≡ *element_start*) **then**

tip = *element_end*

else

tip = *element_start*

end if

point_loop: **continue**

node = *elements_list*_{*poly_elements* *i*, *tip*}

*poly_x*_{*poly_num_points*, *g2_x*} = *nodes*_{*node*, *g2_x*}

*poly_x*_{*poly_num_points*, *g2_z*} = *nodes*_{*node*, *g2_z*}

poly_num_points++ /* See above comment */

```

assert(poly_num_points ≤ g2_num_points - 1)
match = FALSE
/* Look through all of the other elements in the DG polygon for a matching point. Need to try
both ends of each candidate element! For this reason, must exclude current element. */
do j = 1, n
  if ((j ≠ i) ∧ (elements_list_poly_elements_j,element_start ≡ elements_list_poly_elements_i,tip)) then
    assert(match ≡ FALSE)
    match = TRUE
    j_match = j
    tip_match = element_end
  else if ((j ≠ i) ∧ (elements_list_poly_elements_j,element_end ≡ elements_list_poly_elements_i,tip)) then
    assert(match ≡ FALSE)
    match = TRUE
    j_match = j
    tip_match = element_start
  end if
end do
if (j_match ≠ initial_element) then
  if (match ≡ TRUE) then /* Got a match. Reset the bookkeeping indices so that the node gets
added to the output polygon and we can begin the search again for a match to the node at
the other end of this DG polygon element. */
    i = j_match
    j_match = 0
    tip = tip_match
    go to point_loop
    /* Otherwise, we have reached the other end point. Check that the number of points in the
polygon is as expected. As is the convention elsewhere in this code, the poly_num_points
entry in the poly_x array is reserved for the next entry in the array. The original usage of
this algorithm was for cases using all of the input points, corresponding to the equality. */
  else
    assert((poly_num_points - 1) ≤ n)
  end if /* If the match is with the initial_element, we are also done. In this case, though, we
need to reduce poly_num_points by 1 since this is already a closed polygon (first and last
points are same). */
else
  assert((poly_num_points - 1) ≤ n)
  poly_num_points --
end if
return
end

```

31 Count the number of continuous sections in set of elements

Namely, a set of elements should consist of one or more continuous sections, or curves. Some of these will be “open”, i.e., having two unmatched endpoints. The others will be “closed”. The primary objective of this routine is to find the latter. The routine returns the number of curves found and characterizes each as “open” or “closed”. For convenience, a second array identifies an element belonging to that curve, *curve_start*. Another array labels each element according to the curve it belongs to.

(Functions and subroutines 8) +≡

```
subroutine count_curves(elements_list, n, poly_elements, num_curves, curve_type, curve_start,
    curve_start_tip, element_curve_num)
    implicit_none_f77
    implicit_none_f90

    integer n      // Input
    integer elements_list*, element_start:element_end, poly_elements_n

    integer num_curves      // Output
    integer curve_type(dim_curves), curve_start(dim_curves), curve_start_tip(dim_curves),
        element_curve_num*

    integer i, i_init, j_match, tip, node, match, j, tip_match,      /* Local */
        num_ends, i_tot
        /* Dimensioning these as in specify_polygon. Yes, these are ridiculously large. */
    integer end_nodes(2*g2_num_points), end_elements(2*g2_num_points), end_tips(2*g2_num_points)

    num_curves = 0
    assert(n > 1)      /* Initially label each curve with “0” to indicate that it hasn’t been checked. */
    do i = 1, n
        element_curve_num_i = 0
    end do      /* By making a little extra effort, we can get all of the open curves correctly labeled, as
        well as finding the closed ones. The idea is to start with “endpoint” elements first, finding them
        with find_endpoints: */
    call find_endpoints(elements_list, n, poly_elements, num_ends, end_nodes, end_elements, end_tips)
    assert(num_ends ≤ 2 * g2_num_points)
        /* We will add them at the beginning of the main loop. In this case, we take care to get the tip
        right. The rest of the loop is over all remaining elements. Presumably, these are all on one or
        more closed curves. For these, the choice of tip should not matter. In either case, if an element
        has not already been labeled with a curve number, it must belong to a new curve! */
    do i_tot = 1, n + num_ends
        if (i_tot ≤ num_ends) then
            i_init = end_elements_i_tot
            if (end_tips_i_tot ≡ element_start) then
                tip = element_end
            else
                tip = element_start
            end if
        else
            i_init = i_tot - num_ends
            tip = element_end      // Arbitrarily start with this tip
        end if
        if (element_curve_num_i_init ≡ 0) then
            num_curves++
            assert(num_curves ≤ dim_curves)      /* At this point, the logic closely follows the “forward
            loop” procedure used in elements_to_polygon. */
```

```

i = i_init
element_curve_numi = num_curves
    /* Note that curve_start need not be the first point on an open curve. The real need is
    for a point on a closed curve. Recall that the other end of the current element is already
    considered to be on the curve and, thus, represents the actual starting tip. */
curve_startnum_curves = i
if (tip ≡ element_start) then
    curve_start_tipnum_curves = element_end
else
    curve_start_tipnum_curves = element_start
end if
j_match = 0

point_loop: continue
node = elements_listpoly_elements_i,tip
match = FALSE    /* Look through all of the other non-labeled elements in the DG polygon
    for a matching point. Need to try both ends of each candidate element! For this reason,
    must exclude current element. */
do j = 1, n
    if ((j ≠ i) ∧ (elements_listpoly_elements_j,element_start ≡ elements_listpoly_elements_i,tip)) then
        assert(match ≡ FALSE)
        match = TRUE
        j_match = j
        tip_match = element_end
    else if ((j ≠ i) ∧ (elements_listpoly_elements_j,element_end ≡ elements_listpoly_elements_i,tip))
        then
            assert(match ≡ FALSE)
            match = TRUE
            j_match = j
            tip_match = element_start
    end if
end do
if (match ≡ FALSE) then    /* Reached the end of an open curve. Can go to next i_init. */
    curve_typenum_curves = curve_open
else
    if (j_match ≠ i_init) then    /* Got a match that's not the initial element. Label the new
    element, reset the bookkeeping, and go through the point_loop again. */
        assert(element_curve_numj_match ≡ 0)
        element_curve_numj_match = num_curves
        i = j_match
        j_match = 0
        tip = tip_match
        go to point_loop
    else    /* Got a match and it is the initial element. Label the curve accordingly and go to
    next i_init. */
        curve_typenum_curves = curve_closed
    end if
end if
else
    assert(element_curve_numi_init > 0)
    assert(element_curve_numi_init ≤ num_curves)
end if

```

```

end do
assert(num_curves > 0)
return
end

```

32 Search mesh edges to find the ones corresponding to the input DG label

⟨ Functions and subroutines 8 ⟩ +≡

```

subroutine get_mc_edge_seg(meshcon, meshcon_hv, mesh_edge_num_elements, mesh_edge_dg_label,
    mesh_edge_hv, mc_edge, mc_seg)

    implicit_none_f77
    implicit_none_f90

    integer meshcon1:meshcon_elem_max, /* Input */
    meshcon_hv1:meshcon_elem_max, mesh_edge_num_elementsmesh_num_edges,
    mesh_edge_dg_label*,mesh_num_edges, mesh_edge_hvmesh_num_edges

    integer mc_edge1:meshcon_elem_max, mc_seg1:meshcon_elem_max // Output
    integer i_elem, i_edge, j_seg // Local

    do i_elem = 1, meshcon_elem_max
        mc_edgei_elem = int_undef
        mc_segi_elem = int_undef
        do i_edge = 1, mesh_num_edges
            if (mesh_edge_hvi_edge ≡ meshcon_hvi_elem) then
                do j_seg = 1, mesh_edge_num_elementsi_edge
                    if (meshconi_elem ≡ mesh_edge_dg_labelj_seg,i_edge) then
                        mc_edgei_elem = i_edge
                        mc_segi_elem = j_seg
                    end if
                end do
            end if
        end do
        assert(mc_edgei_elem ≠ int_undef)
        assert(mc_segi_elem ≠ int_undef)
    end do

    return
end

```


33 Attempt to find the closest point on the two input edges to the input point

The full set of mesh data are input as are two of its edges, *iedge1* and *iedge2*. The routine first finds the closest point on each edge to the input point, *poly-p*. It then compares the distance from each to *poly-p* and returns the closer one. Namely, *edge_chosen* is set to *iedge1* or *iedge2* accordingly.

(Functions and subroutines 8) +≡

```
subroutine match_edge_to_poly(vc_dummy(poly_p), mesh_nodes, mesh_elements,
    mesh_edge_num_elements, mesh_edge_elements, iedge1, iedge2, mesh_scratch, edge_chosen,
    closest_node, closest_elements, closest_tips)

    implicit_none_f77
    implicit_none_f90

    integer iedge1, iedge2    // Input
    integer mesh_elements*, element_start:element_end, mesh_edge_num_elements, mesh_num_edges,
        mesh_edge_elements*, mesh_num_edges, mesh_scratch*
    real mesh_nodes*, g2_x:g2_z

    vc_decl(poly_p)

    integer edge_chosen, closest_node    // Output
    integer closest_elements2, closest_tips2

    integer i, j
    integer iedges2, close_node2, close_elements2,2, close_tips2,2
    real dist_p_edge2

    vc_decl(edge_p2)
    vc_decl(delta_p2)

    iedges1 = iedge1
    iedges2 = iedge2
    do i = 1, 2
        do j = 1, mesh_edge_num_elements_iedges_i
            mesh_scratch_j = mesh_edge_elements_j,iedges_i
        end do
        call closest_point(vc_args(poly_p), mesh_nodes, mesh_elements, mesh_edge_num_elements_iedges_i,
            mesh_scratch, close_node_i, close_elements_i,1, close_tips_i,1)
        vc_set(edge_p_i, mesh_nodes_close_node_i,g2_x, zero, mesh_nodes_close_node_i,g2_z)
        vc_difference(edge_p_i, poly_p, delta_p_i)
        dist_p_edge_i = vc_abs(delta_p_i)
    end do
    if (dist_p_edge_1 ≤ dist_p_edge_2) then
        i = 1
    else
        i = 2
    end if

    edge_chosen = iedges_i
    closest_node = close_node_i
    do j = 1, 2
        closest_elements_j = close_elements_i,j
        closest_tips_j = close_tips_i,j
```

§33 [#42] **definegeometry2d** Attempt to find the closest point on the two input edges to the input point 112

```
    end do
    return
end
```

34 Find closest point in a set of elements, *edge_elements*, to the point provided, *poly-p*

closest_node is returned as a pointer to the node representing that point. This node may belong to one or two elements, *closest_elements*, with the corresponding tips given in *closest_tips*. If it's just one element, the two values of *closest_elements* and *closest_tips* are set equal to one another.

(Functions and subroutines 8) +≡

```
subroutine closest_point(vc_dummy(poly-p), mesh_nodes, mesh_elements, edge_num_elements,
    edge_elements, closest_node, closest_elements, closest_tips)

    implicit none
    implicit none

    integer edge_num_elements    // Input
    integer mesh_elements*, element_start:element_end, edge_elements*
    real mesh_nodes*, g2_x:g2_z

    vc_decl(poly-p)

    integer closest_node    // Output
    integer closest_elements2, closest_tips2

    integer num_closest, i, j
    real closest_distance, dist

    vc_decl(edge_point)
    vc_decl(delta)    /* Initialize parameters */
    closest_distance = geom_infinity
    num_closest = 0    /* Loop over elements and the two nodes of each */
    do i = 1, edge_num_elements
        do j = element_start, element_end
            vc_set(edge_point, mesh_nodesmesh_elements edge_elementsi,j, g2_x, zero,
                mesh_nodesmesh_elements edge_elementsi,j, g2_z)
            vc_difference(poly-p, edge_point, delta)
            dist = vc_abs(delta)
            /* If this point is closer to poly-p than any other before, reset all of the output variables. */
            if (dist < closest_distance) then
                closest_node = mesh_elementsedge_elementsi,j
                closest_elements1 = i
                closest_tips1 = j
                num_closest = 1
                closest_distance = dist    /* If the distance is equal to the closest one so far, this element
                    should be sharing the node with the one that gave rise to the current closest_distance.
                    This is checked by the assertion. */
                else if (dist ≡ closest_distance) then
                    assert(mesh_elementsedge_elementsi,j ≡ closest_node)
                    closest_elements2 = i
                    closest_tips2 = j
                    num_closest = 2
                end if
            end do
        end do
    end do
    assert(num_closest > 0)    /* If the closest point is an end point, it will only be on one element. In
        this case, equate the two entries so we don't have to keep track of how many there are. */
```

§34 [#43] **definegeometry2d**Find closest point in a set of elements, *edge_elements*, to the point provided, *poly_p*

114

```
    if (num_closest  $\equiv$  1) then  
        closest_elements2 = closest_elements1  
        closest_tips2 = closest_tips1  
    end if  
    return  
end
```

35 Set up sectors

This code is based on the original in *readgeometry.web*. An extension to handle user-specified, auxiliary sectors has been added.

⟨ Functions and subroutines 8 ⟩ +≡

```
subroutine setup_sectors(num_polygons, polygon_points, polygon_xz, polygon_segment,
    polygon_zone, poly_int_props, poly_real_props, num_aux_sectors, aux_stratum,
    aux_stratum_poly, aux_stratum_points, aux_stratum_segment, y_div, sect_zone1, sect_zone2)
```

```
implicit none_f77
```

```
zn_common    // Common
```

```
sc_common
```

```
implicit none_f90
```

```
integer num_polygons, num_aux_sectors, y_div    // Input
```

```
integer polygon_segment*, 0:g2_num_points-1, polygon_points*, polygon_zone*,
    poly_int_props*, 1:poly_int_max, aux_stratum*, aux_stratum_poly*, aux_stratum_points*,
    aux_stratum_segment*
```

```
real polygon_xz*, 0:g2_num_points-1, g2_x:g2_z, poly_real_props*, 1:poly_real_max
```

```
integer sect_zone1*, sect_zone2*    // Scratch
```

```
integer i_poly, j, sector1, face1, i_aux,    /* Local */
```

```
num_zone1, num_zone2, k_zone2, k_zone1, side1_done, aux1_done
```

```
vc_decl(x1)
```

```
vc_decl(x2)
```

⟨ Memory allocation interface 0 ⟩

```
gi_ext    /* Set up default sectors. These are taken to be at the interface between plasma / vacuum
    and solid / exit zones. The procedure consists of cycling through all of the segments of all of
    the polygons. The find_poly_zone routine returns all zones on both sides of this segment. In
    symmetric problems, there is just one on each side. In problems with resolution in the third
    dimension, more than one can appear on each side. When there are multiple entries on each
    side, the index for the third dimension in zn_index is used to identify zones in the same plane.
    find_poly_zone arranges for the type of zone1 to match that of the polygon. Then, sectors are
    defined at adjacent zones of different types. For example, a target sector is identified by having
    zone1 be a solid and zone2 be plasma. The target sector is then associated with zone1 (and its
    corresponding face, face1); zone1 is passed to define_sector as a check. */
```

```
do i_poly = 1, num_polygons
```

```
  do j = 0, polygon_points_{i_poly} - 1
```

```
    if (polygon_xz_{i_poly,j,g2_x} ≠ polygon_xz_{i_poly,j+1,g2_x} ∨ polygon_xz_{i_poly,j,g2_z} ≠
        polygon_xz_{i_poly,j+1,g2_z}) then    /* This used to be part of find_poly_zone. Note the
        explicit insertion of zero here to match the value used in the process_polygon routines. */
```

```
      vc_set(x1, polygon_xz_{i_poly,j,g2_x}, zero, polygon_xz_{i_poly,j,g2_z})
```

```
      vc_set(x2, polygon_xz_{i_poly,j+1,g2_x}, zero, polygon_xz_{i_poly,j+1,g2_z})
```

```
      face1 = lookup_surface(x1, x2)    /* The find_poly_zone routine ensures that zones associated
        with the current polygon are in sect_zone1; the adjacent zones are in sect_zone2. The
        sign of face1 is such that it is a face of the sect_zone1 zones. That is, flights leaving one
        of these zones will be going out through face1. */
```

```
      call find_poly_zone(face1, zn_type(polygon_zone_{i_poly}), polygon_zone_{i_poly}, y_div,
        sect_zone1, num_zone1, sect_zone2, num_zone2)
```

```
      side1_done = FALSE
```

```
      aux1_done = FALSE
```

```

if ((num_zone1 > 0)  $\wedge$  (num_zone2 > 0)) then
  do k_zone1 = 1, num_zone1
    do k_zone2 = 1, num_zone2 /* The test used in define_sector is not going to be
      able to verify that these two zones are indeed adjacent, only that they share
      a (toroidally continuous) common surface. Instead, we have to rely on the
      bookkeeping we do here to ensure that they are in fact neighbors. */
  @#if 0
    if ((num_zone1  $\equiv$  1)  $\vee$  (zn_index(sect_zone2k_zone2,
      zi_iz)  $\equiv$  zn_index(sect_zone1k_zone1, zi_iz))) then
  @#else
    if ((num_zone1  $\neq$  num_zone2)  $\vee$  (zn_index(sect_zone2k_zone2,
      zi_iz)  $\equiv$  zn_index(sect_zone1k_zone1, zi_iz))) then
  @#endif
    /* TARGET */
    if ((zn_type(sect_zone1k_zone1)  $\equiv$  zn_solid)  $\wedge$  (zn_type(sect_zone2k_zone2)  $\equiv$ 
      zn_plasma)) then
      /* This is the same sector we defined in the previous 2-D only version. The
        main difference is that now we're calling it a plasma sector. Believe that
        this assertion should be true based on the way solid zones are defined. Use
        here and below just in case to prevent duplicate plasma sectors. */
      assert((num_zone1  $\equiv$  1)  $\vee$  (num_zone1  $\equiv$  num_zone2))
      sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
        polygon_segmenti_poly, j, -face1, sect_zone2k_zone2, sect_zone1k_zone1)
      define_sector_plasma(sector1)
      /* This is then the sector based on the zone on the other side. I.e., this zone
        is part of polygon i_poly. In principle, we could have the if-then statement
        check for zone1 being type plasma and zone2 being type solid, but then we
        wouldn't have a temperature associated with that polygon. If num_zone1
        = 1 and num_zone2 > 1, we could have multiple sectors on the plasma
        side, but would need just one on the solid side. The flag side1_done is used
        to prevent duplicate sectors from being defined on the solid side. This
        basically applies to every surface and zone1 combination so that the flag
        needs to be reset with each call to find_poly_zone and needs to be checked
        in each of the sections (since the type of zone2 may vary with k_zone2). */
    if ((num_zone1 > 1)  $\vee$  (side1_done  $\equiv$  FALSE)) then
      sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
        polygon_segmenti_poly, j, face1, sect_zone1k_zone1, sect_zone2k_zone2)
      define_sector_target(sector1, poly_int_propsi_poly, poly_material,
        poly_real_propsi_poly, poly_temperature * boltzmanns_const,
        poly_real_propsi_poly, poly_recyc_coef)
      side1_done = TRUE
    end if /* WALL */
  else if ((zn_type(sect_zone1k_zone1)  $\equiv$  zn_solid)  $\wedge$  (zn_type(sect_zone2k_zone2)  $\equiv$ 
    zn_vacuum)) then
    assert((num_zone1  $\equiv$  1)  $\vee$  (num_zone1  $\equiv$  num_zone2))
    sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
      polygon_segmenti_poly, j, -face1, sect_zone2k_zone2, sect_zone1k_zone1)
    /* Do not currently have a "temperature" associated with vacuum sectors.
      May need one for gas puffing there. */
    define_sector_vacuum(sector1)
  if ((num_zone1 > 1)  $\vee$  (side1_done  $\equiv$  FALSE)) then

```

```

    sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
        polygon_segmenti_poly,j, face1, sect_zone1k_zone1, sect_zone2k_zone2)
    define_sector_wall(sector1, poly_int_propsi_poly, poly_material,
        poly_real_propsi_poly, poly_temperature * boltzmanns_const,
        poly_real_propsi_poly, poly_recyc_coef)
    side1_done = TRUE
end if /* EXIT - PLASMA */
else if ((zn_type(sect_zone1k_zone1) ≡ zn_exit) ∧ (zn_type(sect_zone2k_zone2) ≡
    zn_plasma)) then
    assert((num_zone1 ≡ 1) ∨ (num_zone1 ≡ num_zone2))
    sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
        polygon_segmenti_poly,j, -face1, sect_zone2k_zone2, sect_zone1k_zone1)
    define_sector_plasma(sector1)
    if ((num_zone1 > 1) ∨ (side1_done ≡ FALSE)) then
        sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
            polygon_segmenti_poly,j, face1, sect_zone1k_zone1, sect_zone2k_zone2)
        define_sector_exit(sector1)
        side1_done = TRUE
    end if /* EXIT - VACUUM */
else if ((zn_type(sect_zone1k_zone1) ≡ zn_exit) ∧ (zn_type(sect_zone2k_zone2) ≡
    zn_vacuum)) then
    assert((num_zone1 ≡ 1) ∨ (num_zone1 ≡ num_zone2))
    sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
        polygon_segmenti_poly,j, -face1, sect_zone2k_zone2, sect_zone1k_zone1)
    define_sector_vacuum(sector1)
    if ((num_zone1 > 1) ∨ (side1_done ≡ FALSE)) then
        sector1 = define_sector(poly_int_propsi_poly, poly_stratum,
            polygon_segmenti_poly,j, face1, sect_zone1k_zone1, sect_zone2k_zone2)
        define_sector_exit(sector1)
        side1_done = TRUE
    end if
end if /* Define non-default sectors. The idea is to just define a sector
associated with a specific polygon segment. This sector is defined with the
same orientation as the wall and target sectors above. I.e., a flight leaving the
current polygon will be going out through this sector. Note that aux1_done
works just like side1_done does above. */
if (num_aux_sectors > 0) then
    do i_aux = 1, num_aux_sectors // AUX.
        if ((aux_stratum_polyi_aux ≡ i_poly) ∧ (aux_stratum_pointsi_aux ≡
            j) ∧ ((num_zone1 > 1) ∨ (aux1_done ≡ FALSE))) then
            sector1 = define_sector(aux_stratumi_aux, aux_stratum_segmenti_aux,
                face1, sect_zone1k_zone1, sect_zone2k_zone2)
            aux1_done = TRUE
        end if
    end do // num_aux_sectors
end if // zones line up
end if
end do // num_zone2
end do // num_zone1
end if // num_zone1, num_zone2 > 0
end if

```

§35 [#44] **definegeometry2d**

Set up sectors 118

```
        end do    // polygon_points  
end do    // num_polygons  
    return  
end
```


36 Print data from Triangle

This is taken directly from the *trical* example program that is provided with Triangle.

```

⟨ C Functions 36 ⟩C ≡
    void report(io, markers, reporttriangles, reportneighbors, reportsegments, reportedges,
               reportnorms, reportholes)
    struct triangulateio *io;
    int markers;
    int reporttriangles;
    int reportneighbors;
    int reportsegments;
    int reportedges;
    int reportnorms;
    int reportholes;
    {
    int i, j;
    for (i = 0; i < io→numberofpoints; i++)
    {
        printf("Point_□%4d:", i);
        for (j = 0; j < 2; j++)
        {
            printf("□□%.6g", io→pointlisti*2+j);
        }
        if (io→numberofpointattributes > 0)
        {
            printf("□□□attributes");
        }
        for (j = 0; j < io→numberofpointattributes; j++)
        {
            printf("□□%.6g", io→pointattributelisti*io→numberofpointattributes+j);
        }
        if (markers)
        {
            printf("□□□marker_□%d\n", io→pointmarkerlisti);
        }
        else
        {
            printf("\n");
        }
    }
    printf("\n");
    if (reporttriangles ∨ reportneighbors)
    {
        for (i = 0; i < io→numberoftriangles; i++)
        {
            if (reporttriangles)
            {
                printf("Triangle_□%4d_points:", i);
                for (j = 0; j < io→numberofcorners; j++)
                {

```

```

        printf("%%4d", io→trianglelisti*io→numberofcorners+j);
    }
    if (io→numberoftriangleattributes > 0)
    {
        printf("%%attributes");
    }
    for (j = 0; j < io→numberoftriangleattributes; j++)
    {
        printf("%%.6g", io→triangleattributelisti*io→numberoftriangleattributes+j);
    }
    printf("\n");
}
if (reportneighbors)
{
    printf("Triangle%%4dneighbors:", i);
    for (j = 0; j < 3; j++)
    {
        printf("%%4d", io→neighborlisti*3+j);
    }
    printf("\n");
}
printf("\n");
}

if (reportsegments)
{
    for (i = 0; i < io→numberofsegments; i++)
    {
        printf("Segment%%4dpoints:", i);
        for (j = 0; j < 2; j++)
        {
            printf("%%4d", io→segmentlisti*2+j);
        }
        if (markers)
        {
            printf("%%marker%%d\n", io→segmentmarkerlisti);
        }
        else
        {
            printf("\n");
        }
    }
    printf("\n");
}

if (reportedges)
{
    for (i = 0; i < io→numberofedges; i++)
    {
        printf("Edge%%4dpoints:", i);
        for (j = 0; j < 2; j++)
        {

```

```

        printf("%%4d", io→edgelisti*2+j);
    }
    if (reportnorms ∧ (io→edgelisti*2+1 ≡ -1))
    {
        for (j = 0; j < 2; j++)
        {
            printf("%%.6g", io→normlisti*2+j);
        }
    }
    if (markers)
    {
        printf("%%marker%d\n", io→edgemarkerlisti);
    }
    else
    {
        printf("\n");
    }
}
printf("\n");
}
if (reportholes)
{
    for (i = 0; i < io→numberofholes; i++)
    {
        printf("Hole%%4dpoints:", i);
        for (j = 0; j < 2; j++)
        {
            printf("%%.6g", io→holelisti*2+j);
        }
    }
    printf("\n");
}
}

```

See also section 37.

This code is used in section 7.1.

37 C interface routine to Triangle

This is based loosely on the *tricall* example that is provided with the Triangle package.

⟨ C Functions 36 ⟩^C +=

```

void poly2triangles_(npoints, polygon, area, nholes, hole, refine, ntriangles, triangles, markers)
{
    int *npoints;    // Input
    int *nholes;
    int *refine;
    double *area;
    double (*polygon)2;
    double (*hole)2;

    int *ntriangles;    // Output
    double (*triangles)4,2;
    int (*markers)4;

    {
        struct triangulateio in, mid, out;    // Local
        struct triangulateio *final;
        int i, j, n, nh, pt, jt, k, idup, nunq, ip, munq;
        int pointmapg2_num_points, pointinvmapg2_num_points;
        int segmapg2_num_points,2;

        n = *npoints;    /* Have rewritten this code to eliminate duplicate points and segments from the
                           input polygon. */
        nunq = 0;    // Number of unique points
        for (i = 0; i < n; ++i)
        {
            idup = 0;    // = 0 for a new point
            if (i > 0)
            {
                for (j = 0; j < i; ++j)
                {
                    if ((polygoni,0 ≡ polygonj,0) ∧ (polygoni,1 ≡ polygonj,1))
                    {
                        pointmapi = pointmapj;    // A duplicate point;
                        idup = 1;    // keep track of which it matched
                    }
                }
            }
        }
        if (idup ≡ 0)
        {
            pointmapi = nunq;    // Unique point number as fn. of original
            pointinvmapnunq = i;    // The inverse of that
            nunq++;    // Increment after =, arrays start at 0
        }
    }
    /* Analogous code for segments. Note that the orientation of the segments is unimportant to
       Triangle, so we must test both directions in identifying duplicates. */
    munq = 0;    // Number of unique segments
    for (i = 0; i < n; ++i)
    {
        /* We need to explicitly close our polygon by setting up a segment from the last point to the
           first point. Can do that most transparently by defining ip as: */
        if (i ≡ n - 1)

```

```

    {
        ip = 0;
    }
    else
    {
        ip = i + 1;
    }
    if (pointmapi ≡ pointmapip)
    {
        idup = 1;    // Throw out trivial segments
    }
    else
    {
        idup = 0;
        if (munq > 0)
        {
            for (j = 0; j < munq; ++j)
            {
                if (((pointmapi ≡ segmapj,0) ∧ (pointmapip ≡ segmapj,1)) ∨ (((pointmapi ≡
                    segmapj,1) ∧ (pointmapip ≡ segmapj,0))))
                {
                    idup = 1;    // A genuine duplicate segment
                }
            }
        }
    }
    if (idup ≡ 0)
    {
        // Define map for unique segments
        segmapmunq,0 = pointmapi;    // 0 and 1 just denote the ends
        segmapmunq,1 = pointmapip;
        munq++;    // Again, so arrays start at 0
    }
}    /* Can now allocate and set arrays that will be passed to Triangle. */
in.numberofpoints = nunq;
in.numberofpointattributes = 0;
in.pointlist = (double *) malloc(in.numberofpoints * 2 * sizeof(double));
in.pointmarkerlist = (int *) malloc(in.numberofpoints * sizeof(int));
j = 0;
for (i = 0; i < nunq; ++i)
{
    in.pointlistj = polygonpointinvmapi,0;
    j++;
    in.pointlistj = polygonpointinvmapi,1;
    j++;    /* Point markers are intended to be assigned in a manner analogous to that used above
        for breakup_polygon. However, Triangle only respects nonzero markers. So, add 1 here and
        subtract when transferring back to main code. */
    in.pointmarkerlisti = pointinvmapi + 1;
}
in.numberofsegments = munq;
in.segmentlist = (int *) malloc(in.numberofsegments * 2 * sizeof(int));
j = 0;
for (i = 0; i < munq; ++i)

```

```

    {
        in.segmentlistj = segmapi,0;
        j++;
        in.segmentlistj = segmapi,1;
        j++;
    }

    nh = *nholes;
    in.numberofholes = nh;
    if (nh > 0)
    {
        in.holelist = (double *) malloc(in.numberofholes * 2 * sizeof(double));
        j = -1;
        for (i = 0; i < nh; ++i)
        {
            j++;
            in.holelistj = holei,0;
            j++;
            in.holelistj = holei,1;
        }
    }
    else
    {
        in.holelist = (double *) NULL;
    }
    in.numberofregions = 0;
    in.numberoftriangles = 0;
    in.regionlist = (double *) NULL;
    in.segmentmarkerlist = (int *) NULL;
    @#if 0
        printf("Input_point_set:\n\n");
        report(&in, 0, 0, 0, 0, 0, 0, 1);
    @#endif
    mid.pointlist = (double *) NULL;
    mid.pointmarkerlist = (int *) NULL;
    mid.trianglelist = (int *) NULL;
    mid.segmentlist = (int *) NULL;
    mid.segmentmarkerlist = (int *) NULL;
    mid.edgelist = (int *) NULL;
    mid.edgemarkerslist = (int *) NULL;
    mid.normlist = (double *) NULL;

    triangulate("pzQ", &in, &mid, (struct triangulateio *) NULL);

    @#if 0
        printf("Initial_triangulation:\n\n");
        report(&mid, 0, 1, 0, 1, 0, 0, 1);
    @#endif

    out.pointlist = (double *) NULL;
    out.pointmarkerlist = (int *) NULL;
    out.trianglelist = (int *) NULL;
    out.segmentlist = (int *) NULL;
    out.segmentmarkerlist = (int *) NULL;
    out.edgelist = (int *) NULL;

```

```

    out.edgemarkerlist = (int *) NULL;
    out.normlist = (double *) NULL;

    if (*refine == TRUE)
    {
        char args116 = "rpqa";
        char args2   = "zYQ";
        char areastring9;

        sprintf(areastring, "%8.6f", *area);
        strcat(args1, areastring);
        strcat(args1, args2);
    }
    @#if 0
        The minimum area here should probably be user specified. triangulate("rpqzYQ", &mid, &out, (struct
            triangulateio *) NULL);
    @#endif
    triangulate(args1, &mid, &out, (struct triangulateio *) NULL);

    @#if 0
        printf("Final triangulation:\n\n");
        report(&out, 0, 1, 0, 1, 0, 0, 1);
    @#endif

    final = &out;
    }
    else
    {
        final = &mid;
    }

    if (final->numberofcorners != 3)
    {
        printf("STOP!!! Triangle thinks triangles have %i corners", final->numberofcorners);
    }

    if (final->numberoftriangles > max_triangles - 1)
    {
        printf("STOP!!! Number of triangles %i exceeds dimensions of triangles array!",
            final->numberoftriangles);
    }

    for (i = 0; i < final->numberoftriangles; ++i)
    {
        for (j = 0; j < final->numberofcorners; ++j)
        {
            pt = final->trianglelisti*final->numberofcorners+j;
            jt = final->numberofcorners - j; // Reverse order for decompose_polygon
            trianglesi,jt,0 = final->pointlist2*pt;
            trianglesi,jt,1 = final->pointlist2*pt+1;
            /* Subtract 1 here to undo the shift made prior to the first call. */
            markersi,jt = final->pointmarkerlistpt - 1;
        }
        trianglesi,0,0 = trianglesi,3,0;
        trianglesi,0,1 = trianglesi,3,1;
        markersi,0 = markersi,3;
    }
    *ntriangles = final->numberoftriangles;

```

```
free(in.pointlist);
free(in.segmentlist);
free(in.pointmarkerlist);
free(in.regionlist);
free(in.segmentmarkerlist);
free(mid.pointlist);
free(mid.segmentlist);
free(mid.pointmarkerlist);
free(mid.trianglelist);
free(mid.segmentmarkerlist);
free(mid.edgelist);
free(mid.edgemarkerlist);
free(mid.normlist);
free(out.pointlist);
free(out.segmentlist);
free(out.pointmarkerlist);
free(out.trianglelist);
free(out.segmentmarkerlist);
free(out.edgelist);
free(out.edgemarkerlist);
free(out.normlist);
if (nh > 0)
{
    free(in.holelist);
}
}
```


38 INDEX

- a*: [24](#), [25](#).
- a_hat*: [25](#).
- a_vec*: [25](#).
- a_vertex*: [25](#).
- a_y*: [9](#), [9.4](#), [15](#).
- abs*: [9.4](#), [11](#), [15](#), [16](#), [23](#), [24](#).
- add_surface*: [24](#).
- add_transform*: [24](#).
- alpha*: [25](#).
- aname*: [8](#).
- apex*: [24](#).
- area*: [25](#), [27](#), [37^C](#).
- areal*: [9](#).
- areastring*: [37^C](#).
- args1*: [37^C](#).
- args2*: [37^C](#).
- assert*: [9](#), [9.2](#), [9.3](#), [9.4](#), [10](#), [11](#), [12](#), [13](#), [14](#), [16](#), [17](#), [18](#), [19](#), [20](#), [21](#), [21.1](#), [21.2](#), [21.3](#), [21.4](#), [21.5](#), [21.6](#), [21.7](#), [21.8](#), [21.9](#), [22](#), [22.1](#), [23](#), [24](#), [25](#), [27](#), [29](#), [30](#), [31](#), [32](#), [34](#), [35](#).
- aunit*: [8](#).
- aux_sector_ind*: [9](#).
- aux_stratum*: [1](#), [8](#), [9](#), [9.1](#), [14](#), [17](#), [21](#), [35](#).
- aux_stratum_def*: [21](#).
- aux_stratum_points*: [8](#), [9](#), [9.1](#), [14](#), [35](#).
- aux_stratum_poly*: [8](#), [9](#), [9.1](#), [14](#), [35](#).
- aux_stratum_pts*: [21](#).
- aux_stratum_segment*: [8](#), [9](#), [9.1](#), [14](#), [17](#), [35](#).
- aux_stratum_start*: [21](#).
- aux1_done*: [35](#).
- b*: [9](#), [20](#), [21](#), [22](#), [24](#), [25](#).
- b_vec*: [25](#).
- b_vertex*: [25](#).
- b_y*: [9](#), [9.4](#), [15](#).
- backward_loop*: [21](#).
- bc_cross*: [25](#).
- be*: [37^C](#).
- boltzmanns_const*: [16](#), [35](#).
- boundaries_neighbors*: [9](#).
- bounds*: [1](#), [9.4](#), [19](#), [21](#).
- boxgen*: [9.4](#).
- breakup_polygon*: [1](#), [8](#), [14](#), [21](#), [37^C](#).
- bttopdetector*: [1](#).
- c*: [25](#).
- c_hat*: [25](#).
- c_perp*: [25](#).
- c_vec*: [25](#).
- c_vertex*: [25](#).
- center*: [9](#), [12](#), [14](#), [15](#), [23](#), [25](#), [26](#).
- CHAR*: [9](#).
- char_undef*: [9](#), [20](#), [21](#).
- char_uninit*: [8](#), [17](#).
- check_geometry*: [9](#).
- clear_polygon*: [1](#), [8](#), [14](#), [21](#), [21.6](#).
- close_elements*: [33](#).
- close_node*: [33](#).
- close_tips*: [33](#).
- closed_curve*: [21](#), [21.7](#).
- closest_distance*: [34](#).
- closest_elements*: [33](#), [34](#).
- closest_node*: [33](#), [34](#).
- closest_point*: [33](#), [34](#).
- closest_tips*: [33](#), [34](#).
- coeff*: [9](#), [9.4](#), [15](#), [24](#).
- command_arg*: [8](#).
- common_node*: [21](#), [21.8](#).
- composite*: [24](#).
- conea*: [24](#).
- const*: [9](#), [9.1](#), [9.2](#), [16](#), [19](#), [22.1](#), [25](#), [26](#).
- corner*: [21](#), [21.3](#).
- corner_ptr*: [18](#).
- cos*: [9.4](#), [15](#), [23](#), [24](#).
- cos_y*: [9](#), [9.4](#), [15](#), [24](#).
- count_curves*: [21.7](#), [31](#).
- csec*: [20](#).
- current_int_props*: [9](#), [9.1](#), [12](#), [14](#).
- current_real_props*: [9](#), [9.1](#), [12](#), [14](#).
- curve_closed*: [21](#), [21.7](#), [31](#).
- curve_open*: [21](#), [31](#).
- curve_start*: [21](#), [21.7](#), [31](#).
- curve_start_tip*: [21](#), [21.7](#), [31](#).
- curve_type*: [21](#), [21.7](#), [31](#).
- cylindrical*: [1](#).
- cylindrical_hw*: [1](#).
- cylindrical_section*: [1](#).
- declare_varp*: [9](#).
- decompose_polygon*: [1](#), [12](#), [14](#), [15](#), [37^C](#).
- def_geom_2d_main*: [8](#), [9](#).
- default_diag_setup*: [9](#).
- define_dimen*: [9](#).
- define_sector*: [16](#), [35](#).
- define_sector_exit*: [35](#).
- define_sector_plasma*: [16](#), [35](#).
- define_sector_target*: [16](#), [35](#).
- define_sector_vacuum*: [16](#), [35](#).
- define_sector_wall*: [16](#), [35](#).
- define_surface*: [9.4](#), [15](#), [24](#).
- define_surface_a*: [24](#).
- define_varp*: [9](#).
- definegeometry2d*: [1](#), [8](#).

- def2ddetector*: 1.
- degas2*: 1.
- delta*: 21, 21.9, 34.
- delta_p*: 33.
- den*: 25.
- detector_setup*: 1, 9.
- dg_elements_list*: 8, 9, 9.1, 9.4, 10, 14, 21, 21.4.
- dg_file*: 1, 21.
- dg_loop*: 1, 10.
- dg_poly_ind*: 9.
- dg_poly_meshcon*: 8, 9, 9.1, 9.4, 10, 14, 21, 21.8, 21.9.
- dg_poly_meshcon_elem_ind*: 9.
- dg_poly_meshcon_hv*: 8, 9, 9.1, 9.4, 10, 14, 21, 21.8, 21.9.
- dg_poly_meshcon_ind*: 9.
- dg_poly_num_elements*: 8, 9, 9.1, 9.4, 10, 14, 21, 21.4.
- dg_poly_num_meshcon*: 8, 9, 9.1, 9.4, 10, 14, 21, 21.6.
- dg_polygon*: 1, 21.4, 21.6.
- dg_polygons*: 8, 9, 9.1, 9.4, 10, 14, 21, 21.4.
- diag_grp_init*: 17.
- diag_loop*: 22.
- diag_mult*: 8, 9, 9.1, 17.
- diag_name*: 8, 9, 9.1, 17.
- diag_solid*: 8, 9, 9.1, 17.
- diag_spacing*: 8, 9, 9.1, 17.
- diag_stratum*: 8, 9, 9.1, 17.
- diag_tab_index*: 8, 9, 9.1, 17.
- diag_var_max*: 8, 9, 9.1, 17.
- diag_var_min*: 8, 9, 9.1, 17.
- diag_variable*: 8, 9, 9.1, 17.
- diags_ind*: 9.
- dim_aux_sectors*: 8, 9, 9.1.
- dim_curves*: 21, 31.
- dim_dg_poly*: 8, 9, 9.1, 9.4.
- dim_diags*: 8, 9, 9.1.
- dim_elements*: 8, 9, 9.1, 9.4.
- dim_nodes*: 8, 9, 9.1, 9.4.
- dim_walls*: 8, 9, 9.1, 9.4, 11.
- dim_y*: 8, 9, 9.1, 9.4.
- dim_ym*: 8, 9, 9.1, 9.4.
- diskin*: 9, 14, 21.
- diskin2*: 9, 10, 11.
- diskout*: 9, 20, 21.
- dist*: 21, 21.9, 34.
- dist_p_edge*: 33.
- dummy*: 18.
- e*: 9, 20, 21, 22.
- edge*: 1.
- edge_chosen*: 33.
- edge_elements*: 34.
- edge_num_elements*: 34.
- edge_p*: 33.
- edge_point*: 34.
- edge_reverse*: 21, 21.3.
- edgelist*: 36^C, 37^C.
- edgemarkerlist*: 36^C, 37^C.
- electron_charge*: 22.1.
- element_assigned*: 8, 9, 9.1, 9.4, 10.
- element_curve_num*: 31.
- element_end*: 8, 9, 10, 13, 21, 21.8, 29, 30, 31, 33, 34.
- element_ends_ind*: 9.
- element_ind*: 9.
- element_missing*: 8, 9, 9.1, 9.4, 10.
- element_skipped*: 8, 9, 9.1, 9.4, 10.
- element_start*: 8, 9, 10, 13, 21, 21.4, 21.8, 21.9, 29, 30, 31, 33, 34.
- elements_list*: 29, 30, 31.
- elements_to_polygon*: 21.4, 21.7, 21.9, 30, 31.
- end_aux_stratum*: 1.
- end_detectors*: 9.
- end_diagnostic*: 1.
- end_elements*: 21, 21.4, 29, 31.
- end_faces*: 9, 15.
- end_nodes*: 21, 21.4, 29, 31.
- end_point*: 21, 21.4.
- end_prep*: 1.
- end_sectors*: 9.
- end_tip*: 21, 21.4.
- end_tips*: 21, 21.4, 29, 31.
- end_zones*: 9, 9.1, 15, 16.
- eof*: 9.
- epsilon_angle*: 15, 24.
- erase_geometry*: 9.
- exp_inc*: 9, 10.
- face*: 24.
- facearray*: 8, 9, 9.1, 9.4, 12, 14, 16.
- face1*: 9, 16, 24, 35.
- face2*: 24.
- FALSE*: 9, 9.1, 10, 13, 14, 16, 17, 21, 21.3, 21.9, 22, 29, 30, 31, 35.
- file*: 8, 9, 11, 18, 20, 21.
- FILE*: 1.
- file_format*: 9, 20.
- fileid*: 9.
- FILELEN*: 8, 9, 20, 21.
- filename*: 8, 9.
- final*: 37^C.
- find_endpoints*: 21.4, 29, 31.
- find_min_max*: 9, 19.
- find_poly_zone*: 16, 35.

- FLOAT*: 9.
form: 8, 9, 11, 18.
forward_loop: 21.
free: 37^C.
gamma: 25.
geom_epsilon: 9.4, 11.
geom_infinity: 34.
geom_translate: 24.
geometry: 1.
geometry_symmetry: 23, 24.
geometry_symmetry_cyl_hw: 9, 9.2, 9.3, 9.4, 16, 23, 24.
geometry_symmetry_cyl_section: 9, 9.2, 9.3, 9.4, 15, 16, 23, 24.
geometry_symmetry_cylindrical: 9, 9.2, 9.3, 9.4, 23, 24.
geometry_symmetry_none: 9, 9.1.
geometry_symmetry_oned: 9, 9.2, 9.3, 23, 24.
geometry_symmetry_plane: 9, 9.2, 9.3, 9.4, 23, 24.
geometry_symmetry_plane_hw: 9, 9.2, 9.3, 9.4, 15, 16, 23, 24.
geometry2d: 14.
get_mc_edge_seg: 21.8, 21.9, 32.
gi_common: 9, 23, 24.
gi_ext: 9, 23, 24, 35.
grp_sectors: 9, 17.
g2_common: 9.
g2_ncdecl: 9.
g2_ncdef: 9.
g2_ncwrite: 9.
g2_num_points: 8, 9, 10, 17, 20, 21, 21.1, 21.2, 21.3, 21.9, 23, 30, 31, 35, 37^C.
g2_num_polygons: 8, 9, 9.1, 12, 14, 16, 17.
g2_points_ind: 9.
g2_points_ind0: 9.
g2_poly_ind: 9.
g2_polygon_points: 8, 9, 12, 14.
g2_polygon_segment: 8, 9, 12, 14.
g2_polygon_stratum: 8, 9, 12, 14, 17.
g2_polygon_xz: 8, 9, 12, 14.
g2_polygon_zone: 8, 9, 12, 14, 16, 17.
g2_x: 8, 9, 9.4, 10, 11, 12, 13, 14, 15, 18, 19, 20, 21, 21.1, 21.2, 21.3, 21.4, 21.7, 21.8, 21.9, 23, 25, 26, 27, 28, 30, 33, 34, 35.
g2_xz_ind: 9.
g2_z: 8, 9, 9.4, 10, 11, 12, 13, 14, 15, 18, 19, 20, 21, 21.1, 21.2, 21.3, 21.4, 21.8, 21.9, 23, 25, 26, 27, 28, 30, 33, 34, 35.
half: 23, 24, 25, 27.
here: 37^C.
hole: 37^C.
holelist: 36^C, 37^C.
hweb: 1, 9.
i: 9, 18, 21, 24, 27, 28, 29, 30, 31, 33, 34, 36^C, 37^C.
i_aux: 9, 14, 17, 35.
i_diag: 9, 17.
i_edge: 32.
i_elem: 32.
i_grp: 9, 17.
i_inc_g2: 8, 9.
i_init: 31.
i_mc: 21, 21.9.
i_poly: 9, 16, 17, 35.
i_prop: 9, 12, 14, 21.
i_sect: 9, 16, 17.
i_tot: 21, 21.7, 21.9, 31.
i_2: 9.
i_2_loop: 9.
i_2_range: 9.
idup: 37^C.
iedge: 1, 9, 13, 21, 21.4, 21.6, 21.9.
iedge_loop: 21, 21.5.
iedge_order: 21, 21.5.
iedge_p: 21, 21.9.
iedge_points: 21.
iedge_temp: 21, 21.9.
iedges: 21, 21.5, 21.7, 21.9, 33.
iedge1: 33.
iedge2: 33.
ielement: 9, 10.
ielement2: 9, 10.
iend: 20.
implicit_none_f77: 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35.
implicit_none_f90: 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35.
in: 1, 37^C.
increment_g2_num_polygons: 8, 12, 14.
increment_num_aux_sectors: 8, 14.
increment_num_dg_poly: 8, 10.
increment_num_diags: 8, 9.
increment_num_elements: 8, 10.
increment_num_nodes: 8, 10, 11.
increment_num_walls: 8, 10.
increment_y_div: 8, 9.
ind_x: 23.
ind_z: 23.
index: 10.
init_geometry: 9.4.
init_identity: 24.
initial_element: 30.
initial_tip: 30.
inode: 9, 10, 11, 13, 19, 20.

- INT*: 9.
int_props: 21.
int_undef: 9.1, 10, 16, 21, 21.6, 21.7, 32.
int_uninit: 8, 14, 21, 22.
int_unused: 8, 9.1, 9.4, 10, 16.
invert: 24.
io: 36^C.
iostat: 8.
ip: 37^C.
is_aux_sector: 9, 17.
isec: 20.
iseg: 9, 10, 11, 13, 20.
istart: 20.
itip: 9, 13.
iwall: 9, 10, 11, 20.
ix: 8, 9, 12, 13, 14, 15, 18, 21, 21.3.
ix_max: 9, 13.
ix_min: 9, 13.
ix_start: 21, 21.3.
ix_step: 21, 21.3.
ix_stop: 21, 21.3.
ixi: 21, 21.3.
ixlb: 18.
ixmdp: 18.
ixpt1: 18.
ixpt2: 18.
ixrb: 18.
iy: 23.
iysptrx1: 18.
iysptrx2: 18.
iz: 8, 9, 12, 13, 14, 15, 18, 21, 21.3.
iz_max: 9, 13.
iz_min: 9, 13.
iz_start: 21, 21.3.
iz_step: 21, 21.3.
iz_stop: 21, 21.3.
izi: 21, 21.3.

j: 9, 29, 30, 31, 33, 34, 35, 36^C, 37^C.
j_fin: 21, 21.9.
j_init: 21, 21.9.
j_match: 30, 31.
j_max: 21, 21.9.
j_min: 21, 21.9.
j_seg: 32.
jt: 37^C.
jxz: 8.

k: 9, 25, 37^C.
k_zone1: 9, 16, 35.
k_zone2: 9, 16, 35.
keyword: 9, 21, 22.

lc: 25.

len: 9, 10, 11, 21, 22.
length: 9, 10, 11, 21, 22.
line: 8, 9, 9.2, 9.3, 10, 11, 21, 21.1, 21.2, 21.3, 21.4, 21.5, 22, 22.1.
LINELEN: 9, 20, 21, 22.
lookup_surface: 35.
loop1: 9.

m_test1: 25.
m_test2: 25.
ma_check: 9, 21.
ma_common: 9, 21.
ma_lookup: 9, 21.
malloc: 37^C.
markers: 36^C, 37^C.
mass_in: 17.
match: 29, 30, 31.
match_edge_to_poly: 33.
material: 1.
materials_infile: 1.
max: 8, 9, 11, 16, 19, 20.
max_corner: 9, 9.4, 15, 24.
max_segs: 20.
max_triangles: 8, 9, 14, 37^C.
max_xpts: 18.
maximum: 1.
mc_edge: 21, 21.8, 21.9, 32.
mc_node: 21, 21.8.
mc_seg: 21, 21.8, 21.9, 32.
mem_inc: 9.1.
mesh_corner: 9, 13.
mesh_corner_ind: 9.
mesh_curve_num: 9, 9.4, 14, 21, 21.7.
mesh_east: 8, 13, 21.5.
mesh_edge_dg_label: 9, 9.4, 13, 14, 21, 21.8, 21.9, 32.
mesh_edge_elements: 9, 9.4, 13, 14, 21, 21.7, 21.8, 21.9, 33.
mesh_edge_elements_ind: 9.
mesh_edge_hv: 9, 9.4, 13, 14, 21, 21.8, 21.9, 32.
mesh_edge_num_elements: 9, 13, 14, 21, 21.7, 21.8, 21.9, 32, 33.
mesh_elem: 9, 10.
mesh_elements: 9, 9.4, 13, 14, 21, 21.7, 21.8, 21.9, 33, 34.
mesh_elements_ind: 9.
mesh_h: 8, 10, 13, 21.
mesh_ix_end: 9, 13.
mesh_ix_start: 9, 13.
mesh_ix_step: 9, 13.
mesh_iz_end: 9, 13.
mesh_iz_start: 9, 13.
mesh_iz_step: 9, 13.

- mesh_nodes*: 9, 9.4, 13, 14, 21, 21.7, 21.8, 21.9, 33, 34.
- mesh_nodes_ind*: 9.
- mesh_north*: 8, 13, 21.5.
- mesh_num_edges*: 8, 9, 13, 21, 21.5, 21.9, 32, 33.
- mesh_scratch*: 9, 9.4, 14, 21, 21.7, 21.9, 33.
- mesh_sense*: 9, 12.
- mesh_senw_ind*: 9.
- mesh_south*: 8, 13, 21.5.
- mesh_tot_elements*: 9, 13.
- mesh_tot_nodes*: 9, 13.
- mesh_v*: 8, 10, 13, 21.
- mesh_west*: 8, 13, 21.5.
- mesh_x*: 18.
- mesh_xz*: 8, 9, 9.4, 14, 18, 21, 21.3.
- mesh_xz_ind*: 9.
- mesh_xzm*: 8, 12, 13.
- mesh_z*: 18.
- meshcon*: 32.
- meshcon_elem_max*: 8, 9, 10, 21, 32.
- meshcon_hv*: 32.
- meshcon_loop*: 1, 10.
- meshcon_max*: 8, 9, 10, 21.
- mid*: 37^C.
- min*: 19, 20, 21.9.
- min_corner*: 9, 9.4, 15, 24.
- min_dist*: 21, 21.9.
- minimum*: 1, 37^C.
- miss_elem*: 9, 10.
- mixed_type_loop*: 1, 9.
- mod*: 14, 21.5, 21.9.
- mult*: 9, 10, 22, 22.1.
- multiplier*: 1.
- munq*: 37^C.
- n*: 9, 21, 23, 27, 28, 29, 30, 31, 37^C.
- n_iedge*: 21, 21.9.
- nc*: 9.
- NC_CLOBBER*: 9.
- nc_decls*: 9.
- nc_read_materials*: 8.
- nc_stat*: 9.
- ncclose*: 9.
- nccreate*: 9.
- ncendef*: 9.
- NCOEFFS*: 9, 24.
- neg_y*: 25.
- neighborlist*: 36^C.
- new_diagnostic*: 1.
- new_node*: 9, 13.
- new_other_sector*: 9, 16.
- new_polygon*: 1.
- new_solid_sector*: 9, 16.
- new_zone*: 1.
- new_zone_type*: 9, 14.
- new_zone_type_2*: 9.
- next_surface*: 1.
- next_token*: 9, 9.2, 9.3, 10, 11, 21, 21.1, 21.2, 21.3, 21.4, 21.5, 22, 22.1.
- nh*: 37^C.
- nholes*: 37^C.
- node*: 30, 31.
- node_element_count*: 8, 9, 9.1, 9.4, 10.
- node_ind*: 9.
- node_many_elements*: 8, 10.
- node_mixed_normals*: 8, 10.
- node_no_elements*: 8, 10, 19.
- node_one_element*: 8, 10.
- node_regular*: 8, 10.
- node_type*: 8, 9, 9.1, 9.4, 10, 19.
- node_undefined*: 8.
- nodes*: 8, 9, 9.1, 9.4, 10, 11, 14, 19, 20, 21, 21.2, 21.4, 30.
- normlist*: 36^C, 37^C.
- nout*: 21.
- npoints*: 37^C.
- nsectors*: 16, 17.
- nt*: 9, 14.
- ntriangles*: 9, 14, 37^C.
- NULL*: 37^C.
- num*: 21, 21.3.
- num_aux_sectors*: 8, 9, 9.1, 14, 17, 35.
- num_aux_stratum_pts*: 21.
- num_closest*: 34.
- num_curves*: 21, 21.7, 31.
- num_dg_poly*: 8, 9, 9.1, 9.4, 10, 14, 21, 21.4.
- num_diags*: 8, 9, 9.1, 17.
- num_elements*: 8, 9, 9.1, 9.4, 10.
- num_ends*: 21, 21.4, 29, 31.
- num_h_elems*: 9, 10.
- num_iedges*: 21, 21.5, 21.6, 21.7, 21.9.
- num_mesh_elems*: 9, 10.
- num_new_walls*: 1, 9, 11.
- num_nodes*: 8, 9, 9.1, 9.4, 10, 11, 19.
- num_polygons*: 35.
- num_sections*: 20.
- num_walls*: 8, 9, 9.1, 9.4, 10, 11, 14, 20, 21, 21.2.
- num_y*: 9.
- num_zone1*: 9, 16, 35.
- num_zone2*: 9, 16, 35.
- number*: 1.
- numberofcorners*: 36^C, 37^C.
- numberofedges*: 36^C.
- numberofholes*: 36^C, 37^C.
- numberofpointattributes*: 36^C, 37^C.

- numberofpoints*: 36^C, 37^C.
- numberofregions*: 37^C.
- numberofsegments*: 36^C, 37^C.
- numberoftriangleattributes*: 36^C.
- numberoftriangles*: 36^C, 37^C.
- nunit*: 9, 18, 20, 21, 22.
- nunq*: 37^C.
- nx_nz_max*: 9, 9.4.
- nx_d*: 8, 9, 9.1, 9.4, 12, 13, 14, 18, 21, 21.3.
- nx_d_0*: 9, 13.
- nxpt*: 9, 18.
- nzd*: 9, 9.1, 9.4, 12, 13, 14, 18, 21, 21.3.
- nzd_0*: 9, 13.
- one*: 9.1, 9.2, 9.4, 12, 15, 16, 21, 22.1, 23, 24, 25.
- one_zone_type*: 9, 14, 15.
- oned*: 1.
- open_file*: 8, 9, 10.
- open_stat*: 8, 9.
- order_decreasing*: 21, 21.5.
- order_increasing*: 21, 21.5.
- order_undefined*: 21, 21.5.
- other_face*: 9, 16.
- other_sector*: 9, 16.
- other_seg*: 9, 16.
- other_stratum*: 9, 16.
- other_type*: 9, 16.
- other_zone*: 9, 16.
- out*: 37^C.
- outer*: 1.
- outer_loop*: 21, 21.1.
- p*: 9, 21, 22.
- parse_string*: 9, 10, 11, 21, 22.
- phi_mid*: 23.
- PI*: 9, 9.2, 9.4, 15, 22.1, 24.
- pixel_map_test*: 9.
- plane*: 1, 9.4, 15, 24.
- plane_hw*: 1.
- planea*: 24.
- point_loop*: 30, 31.
- pointattributelist*: 36^C.
- pointinvmap*: 37^C.
- pointlist*: 36^C, 37^C.
- pointmap*: 37^C.
- pointmarkerlist*: 36^C, 37^C.
- poly*: 21, 21.4, 21.6, 21.7, 21.8, 21.9.
- poly_area*: 21, 21.4, 21.7, 21.8, 21.9.
- poly_elem*: 9, 10.
- poly_elements*: 29, 30, 31.
- poly_hole_x*: 8, 9.1, 14, 21.
- poly_hole_z*: 8, 9.1, 14, 21.
- poly_int_ind*: 9.
- poly_int_max*: 8, 9, 12, 14, 21, 35.
- poly_int_props*: 8, 9, 12, 14, 16, 35.
- poly_loop*: 21.
- poly_material*: 8, 9.1, 16, 21, 35.
- poly_min_area*: 8, 9.1, 14, 21.
- poly_num_holes*: 8, 9.1, 14, 21.
- poly_num_points*: 30.
- poly_p*: 21, 21.9, 33, 34.
- poly_real_ind*: 9.
- poly_real_max*: 8, 9, 12, 14, 21, 35.
- poly_real_props*: 8, 9, 12, 14, 16, 35.
- poly_recyc_coef*: 8, 9.1, 16, 21, 35.
- poly_stratum*: 8, 9.1, 14, 16, 21, 35.
- poly_temperature*: 8, 9.1, 16, 21, 35.
- poly_x*: 30.
- polygon*: 9, 21, 21.1, 21.2, 21.3, 21.4, 21.7, 21.8, 21.9, 23, 37^C.
- polygon_area*: 21, 21.4, 21.7, 21.8, 21.9, 27.
- polygon_nc_file*: 1, 9, 9.1.
- polygon_points*: 35.
- polygon_segment*: 35.
- polygon_volume*: 23, 27.
- polygon_xz*: 35.
- polygon_zone*: 35.
- polyoutfile*: 21.
- poly2triangles*: 14.
- poly2triangles_*: 37^C.
- poly4*: 9, 15.
- prep_done*: 9, 9.1, 9.4, 14.
- print_min_max*: 1.
- print_polygon*: 1, 21.
- print_walls*: 1, 9, 11, 20.
- printf*: 36^C, 37^C.
- probably*: 37^C.
- process_polygon*: 9, 14, 21, 21.5, 21.6, 35.
- process_polygon_cylindrical*: 15.
- process_polygon_plane*: 23.
- pt*: 37^C.
- quad*: 26.
- quad_center*: 12, 14, 15, 26.
- quit*: 1.
- read_int_soft_fail*: 10, 11.
- read_integer*: 9, 10, 21, 21.1, 21.2, 21.3, 21.4, 22.
- read_real*: 9, 9.2, 21, 22.
- read_real_scaled*: 10.
- read_real_soft_fail*: 9, 11.
- read_sonnet_mesh*: 1, 9.
- read_string*: 9, 10, 11, 21, 22.
- read_uedge_mesh*: 1, 9, 18.
- readfilenames*: 8.
- readgeometry*: 1, 11, 35.

- REAL:** 7.1^C.
real_props: 21.
real_undef: 9, 9.1, 10, 23.
real_uninit: 8, 14, 21.
real_unused: 9.1.
recyc_coef: 1.
refine: 9, 14, 37^C.
regionlist: 37^C.
report: 36^C, 37^C.
reportedges: 36^C.
reportholes: 36^C.
reportneighbors: 36^C.
reportnorms: 36^C.
reportsegments: 36^C.
reporttriangles: 36^C.
reverse: 1, 21.3.
reverse_polygon: 21, 21.4, 21.7, 21.8, 21.9, 28.
rtest: 9.
- sc_common:* 9, 35.
sc_compare: 16.
sc_diag_angle: 22.1.
sc_diag_energy: 22.1.
sc_diag_name_string: 9.
sc_diag_spacing_linear: 22.
sc_diag_spacing_log: 22.
sc_diag_spacing_unknown: 22.
sc_diag_unknown: 22.
sc_exit: 17.
sc_exit_check: 17.
sc_plasma: 17.
sc_plasma_check: 17.
sc_target: 17.
sc_target_check: 17.
sc_vacuum: 17.
sc_vacuum_check: 17.
sc_wall: 17.
sc_wall_check: 17.
sec_dg_meshcon1: 8, 10.
sec_dg_meshcon2: 8, 10.
sec_dg_poly: 8, 10.
sec_dg_wall: 8, 10.
sec_done: 8, 10.
sec_miss: 8, 10.
sec_p1: 8, 10.
sec_p2: 8, 10.
sec_skip: 8, 10.
sec_undef: 8, 10.
sect_zone1: 9, 16, 35.
sect_zone2: 9, 16, 35.
section: 9, 10.
sector: 1, 9.
sector_break: 1, 14.
sector_break2: 1, 14.
sector_strata_segment: 16, 17.
sector_type_pointer: 17.
sector_zone: 17.
sector1: 35.
segmap: 37^C.
segmentlist: 36^C, 37^C.
segmentmarkerlist: 36^C, 37^C.
set_zn_min_max: 23.
setup_sectors: 9, 14, 16, 35.
should: 37^C.
side1_done: 35.
sin: 9.4, 15, 23, 24.
sin_y: 9, 9.4, 15, 24.
skip_elem: 9, 10.
solid: 22.
solid_face: 9, 16.
solid_faces: 9, 9.4, 14, 15.
solid_sector: 9, 16.
solid_ys: 9, 9.4, 14, 15.
solid_zone: 9, 14, 15, 16.
solid_zone_p: 9, 16.
sonnet_mesh: 1.
SP: 18, 20.
spacing: 1, 22.
specified: 37^C.
specify_diagnostic: 9, 22.
specify_polygon: 14, 21, 31.
sprintf: 37^C.
st_decls: 9, 20, 21, 22.
start: 9, 10, 21, 21.2.
status: 8, 9, 11, 18, 20, 21.
stderr: 8, 20, 21, 21.5, 22, 22.1, 23.
stdout: 9, 21.
step: 21, 21.2.
strata: 16, 17.
strata_segment: 1.
stratum: 1, 22.
strcat: 37^C.
string: 9.
surf_type: 24.
surface_points: 24.
swap_edges: 21, 21.9.
symmetry: 1, 9, 9.1, 9.2, 9.3, 9.4, 15, 16, 24.
tab_index: 22.
table_size: 20.
temp: 21, 21.2, 21.3.
temp_aux_segment: 9, 14.
temp_aux_stratum: 9, 14.
temp_holes: 9, 14.
temp_int_props: 9, 14.
temp_num_stratum_pts: 9, 14.

- temp_polygon*: 9, 14.
- temp_real_props*: 9, 14.
- temp_segment*: 9, 14.
- temp_stratum_pts*: 9, 14.
- temp_triangles*: 9, 14.
- temp_x*: 9, 10.
- temp_z*: 9, 10.
- temperature*: 1.
- test_node*: 9, 13, 29.
- test_vec_1*: 9, 12.
- test_vec_2*: 9, 12.
- test_vec_3*: 9, 12.
- The*: 37^C.
- this_mat*: 9, 16.
- this_node*: 9, 11, 13.
- this_poly*: 9, 16.
- this_rc*: 9, 16.
- this_seg*: 9, 16.
- this_stratum*: 9, 16.
- this_temp*: 9, 16.
- tip*: 29, 30, 31.
- tip_match*: 30, 31.
- tmp_edge*: 21, 21.9.
- tmp_seg*: 21, 21.9.
- tmpfilename*: 9, 10.
- triangle*: 25.
- triangle_area*: 1.
- triangle_centroid*: 12, 14, 25.
- triangle_hole*: 1.
- triangleattributelist*: 36^C.
- trianglelist*: 36^C, 37^C.
- triangles*: 37^C.
- triangulate*: 37^C.
- triangulate_polygon*: 1, 8, 14, 21.
- triangulate_to_zones*: 1, 8, 14, 21.
- triangulateio*: 36^C, 37^C.
- tricall*: 36, 37.
- trim*: 9, 20, 21.
- TRUE*: 9, 9.1, 9.4, 10, 13, 14, 16, 17, 21, 21.3, 21.9, 22, 29, 30, 31, 35, 37^C.
- two*: 9, 9.4, 25.
- tx*: 24.
- uedge_mesh*: 1.
- uniform_ys*: 1.
- unit*: 8, 9, 10, 11, 18, 20, 21.
- universal_cell_max*: 23, 24.
- universal_cell_min*: 23, 24.
- universal_cell_vol*: 24.
- universal_cell_3d*: 9.4, 24.
- update_zone_info*: 12, 14, 15, 23.
- user*: 37^C.
- usr2ddetector*: 1.
- var_alloc*: 9, 9.1, 9.4.
- var_free*: 9.
- var_max*: 22.
- var_min*: 22.
- var_realloca*: 8, 9.
- var_reallocb*: 9, 9.4, 10.
- var_reallocc*: 11.
- variable*: 1, 22, 22.1.
- vc_abs*: 21.9, 25, 33, 34.
- vc_args*: 33.
- vc_copy*: 23, 24.
- vc_cross*: 12, 25.
- vc_decl*: 9, 21, 23, 25, 26, 33, 34, 35.
- vc_decls*: 9, 21, 23, 25.
- vc_difference*: 21.9, 25, 33, 34.
- vc_dummy*: 33, 34.
- vc_product*: 12, 25.
- vc_set*: 9.4, 12, 14, 15, 21.9, 24, 25, 26, 33, 34, 35.
- vc_unit*: 25.
- vc_xvt*: 25.
- vector_compare*: 24.
- vol*: 9, 9.4, 24.
- wall*: 1, 21, 21.2.
- wall_break*: 1, 11.
- wall_chunk*: 20.
- wall_elements*: 8, 9, 9.1, 9.4, 10, 11.
- wall_ind*: 9.
- wall_line*: 20.
- wall_loop*: 1, 10.
- wall_loop2*: 1, 10.
- wall_nodes*: 8, 9, 9.1, 9.4, 10, 11, 14, 20, 21, 21.2.
- wall_segment_count*: 8, 9, 9.1, 9.4, 10, 11, 14, 20, 21, 21.2.
- wallfile*: 1, 21.
- wallfiles*: 1.
- wallinfile*: 9, 11.
- walllfile*: 1.
- walloutfile*: 9, 20.
- wall2d_ind*: 9.
- web*: 1, 24, 35.
- write_geometry*: 9.
- write_poly_nc*: 9, 9.1.
- x*: 24, 27, 28.
- x_max*: 9, 19.
- x_min*: 9, 19.
- x_temp*: 28.
- x_wall*: 9, 11.
- xb_max*: 9, 9.1, 9.2, 9.4, 14, 21, 21.1.
- xb_min*: 9, 9.1, 9.2, 9.4, 14, 21, 21.1.
- xcut*: 1, 21, 21.3.
- x0*: 24.

x1: [24](#), [35](#).
x2: [24](#), [35](#).
x3: [24](#).
y_border: [9](#).
y_div: [8](#), [9](#), [9.1](#), [9.4](#), [12](#), [14](#), [15](#), [16](#), [23](#), [35](#).
y_frac: [23](#).
y_ind: [9](#).
y_indm: [9](#).
y_loop: [1](#), [9](#).
y_loop2: [1](#), [9](#).
y_mat: [9](#), [9.1](#), [16](#).
y_max: [9](#), [16](#).
y_max_zone: [1](#).
y_min_zone: [1](#).
y_p_stratum: [9](#), [9.1](#), [16](#).
y_sect_ind: [9](#).
y_stratum: [9](#), [9.1](#), [16](#).
y_values: [1](#), [8](#), [9](#), [9.1](#), [9.4](#), [12](#), [14](#), [15](#), [23](#).
yb: [9.2](#).
yb_max: [9](#), [9.1](#), [9.2](#), [9.4](#).
yb_min: [9](#), [9.1](#), [9.2](#), [9.4](#).
yhat: [9](#), [12](#).
z_max: [9](#), [19](#).
z_min: [9](#), [19](#).
z_wall: [9](#), [11](#).
zb_max: [9](#), [9.1](#), [9.2](#), [9.4](#), [14](#), [21](#), [21.1](#).
zb_min: [9](#), [9.1](#), [9.2](#), [9.4](#), [14](#), [21](#), [21.1](#).
zero: [8](#), [9](#), [9.1](#), [9.2](#), [9.4](#), [12](#), [14](#), [15](#), [18](#), [19](#), [21](#), [21.4](#),
[21.7](#), [21.8](#), [21.9](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [33](#), [34](#), [35](#).
zi_ix: [23](#).
zi_iy: [23](#), [35](#).
zi_iz: [23](#).
zi_ptr: [16](#), [17](#), [23](#).
zn_common: [9](#), [23](#), [35](#).
zn_exit: [16](#), [23](#), [35](#).
zn_index: [16](#), [17](#), [23](#), [35](#).
zn_plasma: [16](#), [23](#), [35](#).
zn_solid: [16](#), [23](#), [35](#).
zn_type: [16](#), [35](#).
zn_type_set: [23](#).
zn_undefined: [16](#).
zn_vacuum: [16](#), [23](#), [35](#).
zn_volume: [23](#).
zone: [9](#), [9.1](#), [12](#), [14](#), [15](#).
zone_center: [23](#).
zone_max: [23](#).
zone_min: [23](#).
zone_type_array: [8](#), [9](#), [9.1](#), [9.4](#), [12](#), [14](#), [23](#).
zonearray: [8](#), [9](#), [9.1](#), [9.4](#), [12](#), [14](#), [23](#).
zone1: [35](#).
zone2: [35](#).

〈Allocations Initializations 9.1〉 Used in section 9.
 〈C Functions 36, 37〉 Used in section 7.1.
 〈DG Polygon Keyword 21.4〉 Used in section 21.
 〈Edge Keyword 21.3〉 Used in section 21.
 〈End Prep 9.4〉 Used in section 9.
 〈Functions and subroutines 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35〉 Used in section 7.1.
 〈Intelligent Edge Processing 21.6〉 Used in section 21.
 〈Intelligent Edge Specification 21.5〉 Used in section 21.
 〈Memory allocation interface 0〉 Used in sections 35 and 9.
 〈New Polygon 14〉 Used in section 9.
 〈Outer Keyword 21.1〉 Used in section 21.
 〈Process Closed iedge Polygon 21.7〉 Used in section 21.6.
 〈Process DG File 10〉 Used in section 9.
 〈Process Single Mesh Point 21.8〉 Used in section 21.6.
 〈Process and Match Up Mesh Edge 21.9〉 Used in section 21.6.
 〈Read Wall File 11〉 Used in section 9.
 〈Set Bounds 9.2〉 Used in section 9.
 〈Set Mesh Elements 13〉 Used in section 9.4.
 〈Set Mesh Polygons 12〉 Used in section 9.4.
 〈Set Symmetry 9.3〉 Used in section 9.
 〈Setup End Zones 15〉 Used in section 9.
 〈Setup Sector Diagnostics 17〉 Used in section 9.
 〈Setup Toroidally Facing Sectors 16〉 Used in section 9.
 〈Variable Keyword 22.1〉 Used in section 22.
 〈Wall Keyword 21.2〉 Used in section 21.

COMMAND LINE: "fweave -f -i! -W[-ykw700 -ytw40000 -j -n/
 /u/dstotler/degas2/src/definegeometry2d.web".

WEB FILE: "/u/dstotler/degas2/src/definegeometry2d.web".

CHANGE FILE: (none).

GLOBAL LANGUAGE: FORTRAN.